

VECTOR

Orlando '99 Reports ...

- Hoekstra reviews SAX for Linux 48
- Smith & Lucas on Orlando 58, 75
- Sharp APL Operators 85
- Reiter on FFT and J 111
- McDonnell and the Bridge 126
- Neitzel on MIDI in J 130



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.16 No.3 January 2000

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1999-2000

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

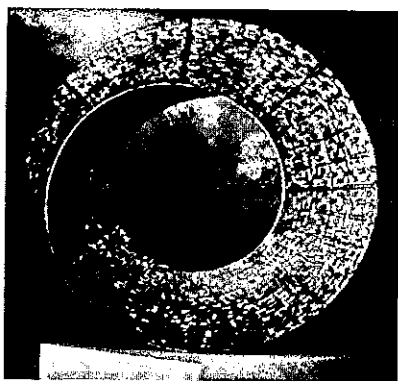
Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

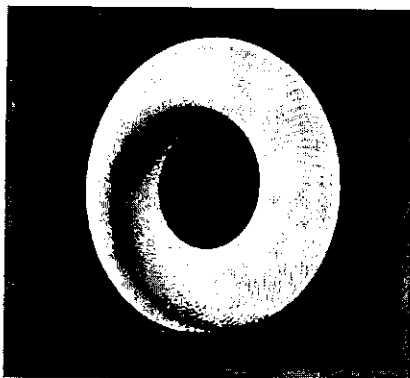
Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385 Email: apl385@compuserve.com

Contents

Editorial	Stefano Lanzavecchia	Page 3
APL NEWS		
Quick Reference Diary		5
News from Sustaining Members		7
APL Product Guide	Gill Smith	12
The Education Vector		
Zark Newsletter Extracts	edited by Jon Sandles	27
J-ottings 23: Just what do they sell at C. and A.?	Norman Thomson	39
Puzzle: Funny Cube	Bernard Legrand	44
REVIEW		
My SAX Experience - Sharp APL for Linux, Beta version 5.1.0	Bob Hoekstra	48
RECENT MEETINGS - APL2000		
APL2000 User's Meeting	Adrian Smith	58
A Conference for Winners	Jim Lucas	75
GENERAL ARTICLES		
Sharp APL Operators	Dan Baronet	85
Functional Implementation of Moments Based Region Descriptors in J	C. Anne Chamley	99
Fast Fourier Transforms, Diffraction Patterns and J	Reiter et al	111
TECHNICAL SECTION		
Hackers' Corner: Making Compressed GIFs in APL+Win and Dyalog APL	Adrian Smith	119
PROPER Capitals	Adam Kertesz	125
At Play with J: We'll Cross that Bridge when we Come to it	Gene McDonnell	126
Setting up a www-server with Dyalog APL and IIS	Janne Mäenpää	131
Sending MIDI Messages to a Sound-card with J	Martin Neitzel	136
Index to Advertisers		143



Umbilic Torus NC by
Helaman Ferguson, 1986



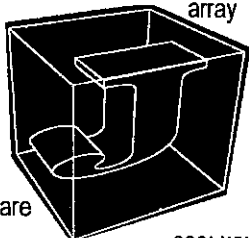
Umbilic Torus created in Jsoftware

For those who see the elegance in an algorithm, Jsoftware offers an OLAP programming language ideal for solving complex problems.

*Jsoftware is the modeling tool of choice
in the art of programming.*

Jsoftware is an OLAP language. It's high performance interactive nature allows a programmer to concisely express algorithms when analyzing complex data sets.

The core language is based on a small set of simple yet consistent rules with many powerful facilities to define new operations. Built-in functions are



highly optimized for operating on whole datasets at a time, allowing you to manipulate data far more easily than with conventional software. Also, sparse array support provides an efficient form of storage for very large data sets.

Make sure to join the JForum for a lively discussion on a variety of interesting J topics,

see: www.jsoftware.com/forum.htm

Strand Software, Inc.
19235 Covington Ct.
Shorewood, MN 55331
(612) 470-7345
Fax (612) 470-9202
info@jsoftware.com

www.jsoftware.com

Editorial

by Stefano Lanzavecchia (stf@apl.it)

"The pace of microchip technology change is such that the amount of data storage that a microchip can hold doubles every year."

Moore's Law

I doubt any computer professional these days ignores the so-called *Moore's Law*, yet it was worth quoting it again in its original form. In 1965 when preparing a talk, Gordon Moore noticed that up to that time, microchip capacity seemed to double each year. The pace of change having slowed down a bit over the past few years, the definition has changed (with Gordon Moore's approval) to reflect that the doubling occurs only every 18 months. Moore's law is often quoted in terms of general computing power doubling at least every 18 months, forgetting that raw power is, in general, not enough. Last summer, my attention was caught by the web site of a researcher working for Microsoft, professor Todd Proebsting. It's not my intention to sponsor the work of the professor, who seems to be doing quite well without my help, at least looking at the projects he's been involved with so far, but I would like to quote here what he calls *Proebsting's Law*, which is a paraphrased version of Moore's law, with a rather more depressing conclusion: *"Advances in compiler optimisations double computing power every 18 years."*

The evidence shown to support this assertion is a simple experiment. I will quote some more directly from its web site for those of you who are reading this text without a cellphone connection to their favourite palmtop to browse the web even during the distressing trip by train from York to Manchester Airport. *Run your favourite set of benchmarks with your favourite state-of-the-art compiler. Run the benchmarks with and without optimisations enabled. The ratio of those numbers represents the entirety of the contribution of compiler optimisations to speeding up those benchmarks. Let's assume that this ratio is about 4x for typical real-world applications, and let's further assume that compiler optimisation work has been going on for about 36 years. These assumptions lead to the conclusion that compiler optimisation advances double computing power every 18 years. QED.* Another way to put it is that, compared to the increase in hardware performance, theoretical studies and practical applications of research on optimising compilers, despite the large amount of resources they consume, contribute only marginally to the effective increase in computing power available to users. Proebsting's conclusion is that Programming Language Research should concentrate on more effective topics than compiler's optimisations. *Perhaps programmer productivity is a more fruitful area.*

Since APL is not a compiled language, how does this law relate to APL programmers? Let me throw another external contribution in, before I come to some conclusions of my own. This morning, while I was collecting my thoughts to write this editorial, I came across another very interesting article, which not only is written in good English, but which puts in a nicely organised way opinions that I mostly share with the author. The article's title is *"Are Very High-Level Languages Really High-Level?"*. Please check the appendix for a reference to the article on the web. In the five dense pages of text, the

author basically expresses the opinion that, while end-users' tools have progressed immensely since the days of teletypes, programmers' tools, that is tools used by programmers and meant for programmers, have not. Many developers are convinced that the simple tools they use allow them to be more productive than if they used anything new, developed since they last learned how to use a text-editor. Good for them. I don't buy into that. The development environment provided by Microsoft Visual C++ is amazingly more powerful than anything that we do-it-yourself hardcore APLers ever even imagined could be done. Yet, more complex is not necessarily better, but this is not the point. In the early 90s a whole lot of new programming languages, known as Very High Level (to distinguish them from the first High-Level ones, like COBOL or Basic or even Pascal) were introduced, to simplify the job of quickly prototyping concepts, but they have not evolved a lot since. In some ways they have regressed by becoming too complex without a good redesign to incorporate all the new ideas.

If I put together the thoughts of the two authors, I can come to these conclusions: a compiler producing optimised code historically does not seem to have helped a lot programmers (who in general pay for the optimisation job with extra wait time) nor end-users; APL is not compiled, and its being interpreted is, in my opinion, a major bonus for a software developer. It allows short code-debug cycles because the developer can at any time not only check the state of the data, but interact with the data, experiment with the data, while running the application. A symbolic debugger like the one offered by Microsoft's Visual Studio permits some of this, but the way in which both code and data can be inspected and modified on-the-fly in an interpreter like APL is still unmatched. Yet, this is not enough to make the life of a developer easy. The development environment of all the commercial implementations of APL is so retro (not to say worse) that any improvement would be well accepted, like the new debugger in APL+Win.

And we haven't even started tackling the issue of language features. In short, APL was no less than two decades ahead of many today's very high-level languages. Somehow the slow pace of improvement of APL interpreters has now put the language back in the group. Are we all really so happy of the way we coded (I wasn't to be sincere, I was more likely trying to learn how to read and write) in the 70s? I will never claim that a fancy GUI makes life easier, but it's true that programmers' productivity is always hardly taken into account when new features are added to a language, and this is also because they don't seem to care. What if they cared?

References

Moore's Law: <http://www.eco.utexas.edu/undergraduate/Forum/ML/index1.htm>

Todd Proebsting's homepage: <http://www.research.microsoft.com/~toddpro/>

Are VHLLs Really High-Level?: http://www.oreilly.com/news/vhll_1299.html

Quick Reference Diary

Date	Venue	Event
24-27 July 2000	Berlin	APL Berlin 2000

APL Berlin 2000

APL Berlin 2000 is an Array Programming Languages Conference, to be held in the Department of Computer Science at the Technical University of Berlin.

The Conference Chairmen are Dieter Lattermann
(dieter_lattermann@compuserve.com) and Conrad Hoesle-Kienzlen
(conrad_hoesle@csi.com).

The Conference has a website at:

<http://stat.cs.tu-berlin.de/APL-Berlin-2000>

mirrored at:

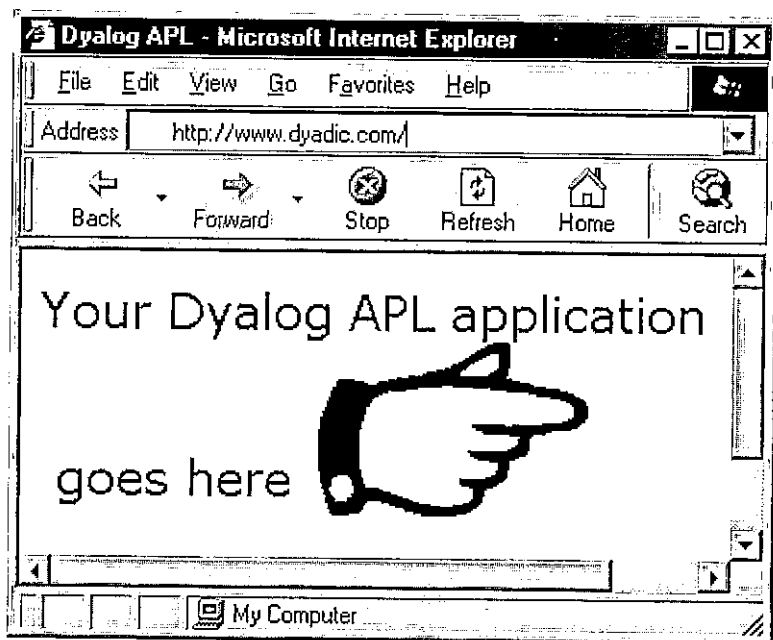
<http://www.lingo.com/APL-Berlin-2000>

Dates for Future Issues of VECTOR

	Vol.16 No.4	Vol.17 No.1	Vol.17 No.2
Copy date	10th March	19th May	8th September
Ad booking	17th March	26th May	15th September
Ad Copy	24th March	2nd June	22nd September
Distribution	April 2000	July 2000	October 2000

dyalog APL

The Definitive APL for Windows™



Using Dyalog APL, not only can you write a powerful, fully functional, **multi-threaded** Web Server, but you can also write ActiveX controls that automatically download and run in the Web browser.

Dyalog APL - the Internet - the Future

Dyalog Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
Hants. RG24 7AL, United Kingdom.

Tel: +44 1256 811125 Fax: +44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

Sustaining Members' News

Soliton Associates

Soliton is pleased to announce the availability of a beta release of the SHARP APL Java Interface (SJI).

The SJI enables SHARP APL to call Java and vice versa. SHARP APL is available for OS/390, AIX, Solaris and Linux. The SHARP APL versions for AIX, Solaris and Linux are commonly called SAX.

The beta release of the SJI for Linux can be downloaded from www.soliton.com/linux. The SJI for Linux, like SAX for Linux, is royalty free for personal use licenses.

GUI and non-GUI gateway

The SJI enables an APL programmer to use all of the available Java GUI classes. However, the SJI is not limited to Java's GUI classes but can access non-GUI classes as well, such as JDBC and Servlets.

A Simple and Timeless Architecture

The architecture of the SJI is simple, consisting of two components: an APL toolkit and a SHARP APL Java Engine. The communication between the toolkit and the Java Engine is socket-based. The Java Engine is a Java Program that can be executed in any Java environment, for example those available for Windows, NT, Unix, Linux and Mac/OS. The Java Engine is small and can also be executed as part of an applet within a browser (Netscape or MS-IE). This architecture is consistent with the server nature of SAX. We expect that many users will have a SAX server under Linux and a LAN-connected GUI client on a Microsoft Windows platform.

The Java Engine listens to a TCP/IP socket port or connects to SAX via the SHARP APL Browser Interface (SBI). Both mechanisms are supported. The SBI is planned for beta distribution in early 2000. The Java engine, being a Java program, can create objects of any Java class, and invoke any method on such objects. This is exactly its purpose. The APL toolkit instructs the Java Engine to create forms, button, toolbars, JDBC objects, etc. When the Java Engine receives information, from a user clicking something or from another process, like JDBC, it forwards this information to the APL application.

The Java Engine and the toolkit are unaware of the available Java classes. This enables the APL application programmer to refer to Java classes and methods using their Java names. However, it does take care to convert APL data structures into Java data structures, and vice versa. That simplicity makes the SJI timeless. As Java evolves, new classes and methods will be available, and these, in turn, will be available to the APL programmers. Moreover, as long as Java supports the classes and methods on the targeted platforms, they will be available to the SJI.

GUI Geometry

Writing applications with a graphical user interface can be tedious, in particular, dealing with how the different GUI elements (buttons, text boxes, labels) are placed and how they move when the user resizes the form. Most application writers have to count pixels and compute the placement of the graphic elements based on the total space that the form has after a resize. Java, being device independent, handles the geometry of GUI elements very well by layering frames in a similar way to HTML forms. In most cases, the form has the expected appearance and behaves as expected during resize.

Distributed processing

With the SJI, you can offload some of the processing from the server platform to the client platform by creating Java expressions to be executed by the Java Engine. A typical example would be data validation by the Java client before a form is submitted to the APL server.

Application as URL

Soliton recognises the need to make APL more available to the Internet. By coupling the SBI (SHARP APL's Web server) with the SJI, APL-Java applications can be accessed through their Internet address or URL. By specifying the URL associated to SJI application, the SBI will send the Java engine to your browser as an applet if not already cached and the application will communicate with the applet to create forms. An application is simply a URL, no need to have DLLs or executables installed.

JAVA calls APL

Not only can APL call Java, but a Java application can also call APL, thanks to some specific classes that are part of the SJI. For example, you may use a J-Application Builder to design a user interface and call APL for data processing

and retrieval. The conversion of data structures between Java and APL is automatic. A 'bean' to facilitate this type of usage is under consideration.

Performance and efficiency

Offering a GUI via a client-server, device and operating system independent mechanism may seem like a 'slow' generic solution. The SJI isn't slow, because most Java commands are buffered and sent as a group to the Java Engine. For example, to create a form, 10 buttons, a menu bar with options and set 'listeners' or callback functions when the buttons are pressed, is done in a single transaction. The TCP connection between the server and the client can introduce a small delay in processing user input. However, in most cases, only the 'Submit' button will cause a TCP/IP transfer to the server, while the Java client will process buttons and fields input locally. To facilitate the work of GUI developers, the SJI can work in diagnostic mode where commands are not buffered but executed at once by the Java Engine.

Beta Limitation

A beta release of the SBI is packaged with the beta release of the SJI for Linux. Only connecting out to the Java Engine is supported: the 'Application as URL' requires the SBI and is not supported yet.

However, with the current beta, you can create GUI or JDBC applications in which APL runs on a Linux server and Java executes on a Linux, Windows or MAC client.

Examples

To illustrate the ease of use of the SJI, here are simple examples:

- 1) Create a frame, give it a name
`frame←Anew 'Frame' ⋮ 'My Application'`
 Create an object of type 'Frame' and give it a title 'My Application'. Create the object at once (Anew) and assign the handle to the variable 'frame'.
- 2) Create a button 'Click Me'
`btn←Anew 'Button' ⋮ 'Click Me'`
 Same as first example but object type is 'Button' and title (label) is 'Click Me'. 'btn' receives the handle for the button.
- 3) Put the button on the frame
`Ninvoke frame ⋮ 'add' ⋮ <btn`
 Add 'btn', the button, to the frame 'frame' as defined in the first example. The 'Ninvoke' means that the operation will be buffered.

- 4) Create a callback, when the Button 'btn' is pressed, call the APL function 'clickme'

```
Winvoke btn>'addActionListener'><NEW>'SJActionListener'>'clickme'
```

For button 'btn', add a Java listener called 'SJActionListener'. SJActionListener is a generic listener for the SJI and in turn, it will tell the APL application to execute the function 'clickme'.

Conclusion

Soliton believes that SJI applications will enable an organisation to distribute APL processing among different TCP connected platforms. It will ease the work of the APL developer by offering a solution that does not need to be rewritten or recompiled every time a desktop operating system is upgraded or changed. Moreover, it will enable users to access APL applications without any need to install additional desktop software as long as a browser is installed.

Insight Systems ApS

Insight Systems is back! After 4 years as part of the Adaytum Group, it was time to return to our roots and become a small company again. On November 1st 1999, Gitte Christensen, Helene Boesen and Morten Kromberg extracted the APL and Statgraphics business from Adaytum, moved about 100 metres and set up our own shop again (with a Linux server this time, rather than NT).

APL Sales and Support

Insight Systems remains the "Array Language Supermarket" for Denmark. We distribute and support APL and J interpreters from Cognos, Dyadic, IBM and Iverson Software.

Products

Most of the products and technologies that we developed in the first half of the nineties are now either in the public domain or OEM'd by vendors (and a few are obsolete, let's face it). Our ODBC interfaces remain the "Industry Standard" under the name SQAPL or APL Link, depending on which vendor you use. We expect the vendors to provide suitable replacements using ADO or other technologies, but until this happens we will continue to support the products and adapt them to new environments if required.

The platform-independent array format we developed for SQAPL and our "APL Pipes" products have been adopted by a group of APL vendors working on a

common TCP-based array transfer protocol. We're entering the new millennium without products of our own, but watch this space!

APL Management Consultancy

If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink-wrapped solutions based on APL.

APL Development and Project Management

Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.

Contact Details

Please see the back cover for our new address and phone numbers.

Causeway Graphical Systems Ltd

The main focus of work since Orlando has been getting the APL2000 releases of both *RainPro* and *NewLeaf* completely back into line with the Dyalog versions. The chart viewer has been modernised with the new CommandBars, and takes advantage of the additional printer properties in APL+Win 3.5 to allow printer selection and paper orientation to be set by the viewer. We now have a single source for the documentation for both platforms, which allows the manuals, help-files and web-based tutorials to be maintained and re-built with minimal effort. From now on, it stays this way!

Also completed over the Christmas break is our Java-applet viewer for *RainPro* graphs and *NewLeaf* tables. This provides a useful alternative to the VML technology (vector graphics support is now solidly available in IE5 and Windows 2000) for non-Microsoft platforms, and it is an essential component of any SAX/Linux implementation of *Rain* graphics. Examples of both formats are on the web-site, where you can also keep up to date with the latest features and fixes. We will be set up for secure web trading (VISA/MasterCard) in the very near future, so purchasing the latest update will get even easier.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to:
Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385,
Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Beautiful Systems	Dyalog APLW for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
The Bloomsbury Software Company	APL+PC Version 11	250	Upgrade to version 11 gives free runtime (£120 from any version)
	APL+Win v3.0	1350	A 32-bit Windows-hosted interpreter that runs under all Windows platforms including Windows 95. Note: Upgrade for £350 from version 1.8 or 2, £920 from version 1.0
	Migration to APL+Win	1060	from APL*PLUS/PC APL*PLUS/DOS any version. from earlier versions of APL*PLUS II
	APL+DOS	1250	APL*PLUS II DOS is renamed to APL+DOS.
	Migration to APL+DOS	760	from APL*PLUS/PC
	APL Link	200	Database Access
	APL Link Pro	500	
	APL*PLUS II for UNIX	poa	APL2000's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dinosoft Oy	Dyalog APLW for Windows	poa	Finnish distributor of Dyalog APL products.

Dyadic	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
IAC/Human Interfaces			
I-APL Ltd	I-APL/Mac	13	Macintosh version of I-APL
	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
IBM APL Products	I-APL/Archimedes	8	As above
	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APIs; TIME facility
	APL2 for Windows version 1.0	\$1500	Product No. 5639-d46, part number 4229558.
	APL2 Runtime environment	\$250	Part No. 39L8419 - Packager and runtime CDs. One additional runtime install \$200, 5 additional \$900, 10 \$1,500.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyalog Systems Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
Iverson Software Inc.	J on the Web online registration ...		
	J Educational Edition	\$95	
	J Standard Edition	\$295	
	J Professional	\$895	
	Books and accessories (discounts for reg users)		
	J Dictionary	\$50	
	J User Manual	\$50	
	J Phrases	\$50	
	J Primer	\$50	

	Concrete Math	\$40	
	Exploring Math	\$50	
	J User Conference Proceedings	\$35	
	Mugs, T-shirts, Mousepads	\$10 each	
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also
	APL+Unix	poa	distributes in Switzerland and Belgium. Call for price quotes.
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Oasis	Dyalog APL	poa	Dyadic Systems
	APL*PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL.
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		

Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for Unix	poa	for IBM AIX and Sun Solaris
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and ISI products	poa	
	USA		
	Dyadic and ISI products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum Software	Adaytum Planning	poa	Full-featured Budgeting and Financial Planning system for medium to large enterprises.
APL Group (see Eventra)			
APL Software/Services			
	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dlick.holt@juno.com.
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
The Bloomsbury Software Company (for VSAPL)	Enhancements & Sharefile	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	Compiler	poa	The First APL compiler!
(for APL2)	Sharefile/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
Causeway	CausewayPro for Dyalog/W	400/\$600	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Functionally compatible across Dyalog/W and APL*Win.

	<i>NewLeaf</i> for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win (1.8 and above)
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	HELM	poa	Decision Support System for top management. Handles large multi-dimensional tables, data analysis, EIS presentations, generates HTML and LaTeX output. Platforms: MS-DOS, LAN, Windows 3.1/95/NT. Ideal for APL customisation (APL*PLUS II and Dyalog APL); more than 150 installed.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/I users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Eventra	Qualedi	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
IAC/Human Interfaces	SPARKS	poa	Educational simulation of electric circuit (for Apple Mac.)
	EPIDEMIC	poa	Educational simulation of spreading infection (for Apple Mac.)
	COINS	poa	Educational simulation (KS3) of coin-tossing experiment with simple stats (for Apple Mac.)
	FIBONA	poa	Educational simulation of Fibonacci's rabbits (for Apple Mac.)
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System (AGSS)	\$250 \$500 \$2500	for DOS, Product Number 5764-009 for Workstations (OS/2, Aix, Solaris), Product Number 6764-092 for CMS, Product Number 5764-011
INFOSTROY	APL*PLUS/Xbase Interface (II386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark <i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>
JAD Software	JAD SMS	poa	Software management system for Dyalog APL (7.2 or later) based on hierarchical databases; includes gui interface and stand-alone functions. Pricing will be per-user. Available fall 1999.
Lescasse Consulting	APL+Win Monthly Training	\$800	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.

	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FRESCO Business Graphics	poa	Fast and easy business graphics DLL
	AP126/PC	poa	GDDM interface for Dyalog APL/W
	AP127/PC	poa	ODBC Interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP Interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	ISAP	poa	MS Windows DLL for calling Dyalog APL/W from web pages via MS Internet Information Server. Visit www.lingo.com for a demo.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +II v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Soliton Associates	LOGOS	poa	Application Development Environment
	VIEWPOINT	poa	Report generator with interfaces to DB2 and OS/390 data
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Ajay Askoolum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.
Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969
AUSCAN Software	Consultancy	poa	APL software development, training
Bloomsbury Software	Consultancy	300-750+VAT	
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems Ltd	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
IAC/Human Interfaces	Consultancy	poa	APL and ergonomics consultancy services.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials, Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.

Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
George MacLeod	Consultancy	poa	Design and programming of new APL applications. Enhancing and maintaining existing APL applications. Porting existing APL applications from one APL system to another. Supporting users of APL applications. Experienced on both mainframe, UNIX and PC APL interpreters.
Mackay Kinloch Ltd	Consultancy	from £40/hr	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Y2K, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning, for all APL versions (10+ years experience)
Object Oriented Ltd	Consultancy	poa	General APL consulting, code recycling U mainframe to PC, performance tuning.
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to APL-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL.
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.

Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
Bloomsbury Software	Training	poa	Contact the company for details.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Soliton Associates	MVSLINK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL	FF350 (private) FF2800 (Company)	
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)

Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join
NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Moscow, Russia		Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qity SI-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION CONTACT

ADDRESS, TELEPHONE, FAX, EMAIL etc.

ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Marlenhoehe 86, 25451 Quickborn, Germany. Tel: +49 4106 60977 Fax: +49 4106 67869 Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: adsys@ibm.net
Adaytum Software	Douglas Rowley	13 Great George Street, BRISTOL BS1 5RR, UK. Tel: 0117-921 5555
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: 100331.644@compuserve.com
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Club Germany	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1451
APL Group (see Eventra)		
APL Software/Services	Dick Hoyt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.hoyt@juno.org
APL Solutions Inc	Enc Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cais.com
Ajay Askoolum	Ajay Askoolum	42 Hanworth Road, Redhill, Surrey RH1 5HT Tel: 01737-771643 Email: ajay@askoolum.freemove.co.uk
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@interlog.com
BACUS	Joseph De Kerf	Roolberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 866-2636; Fax: +1 (215) 866-4888
Bloomsbury Software	Peter Day	Bloomsbury House, 74-77 Great Russell St., London WC1B 3DA, UK. Tel: 0171-436 9481 Fax: 0171-436 0524 Email: pd@bloomsbury-software.co.uk
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: 100612.1057@compuserve.com
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: 100430.740@compuserve.com

Causeway Graphical Systems Ltd	Adrian Smith	The Makings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: causeway@compuserve.com
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 0171-404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652 Email: codework@inrete.it
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375. c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Danish User Group	Helene Boesen	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dinosoft Oy	Pertti Kalliojärvi	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dutch APL Association	Bernard Smoor (Sec)	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, UK. Tel: 01256-811125 Fax: 01256-811130
Dyadic Systems Ltd.	Peter Donnelly	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408) 782-6648 Fax: +1 (408) 782-6627 Email: info@DynArray.com
DynArray Corporation	Dr James Brown	Merritt Crossing, 440 Wheelers Farms Road, Milford, CT 06460, USA. Tel: +1(203) 882-9988. Fax: +1 (203) 882-9946 Email: ssawabini@eventra.com; eshaw@eventra.com
Eventra	Stuart Sawabini	Berteliusvagen 12A, S-146 38 Tuillinge, Sweden Tel&Fax: +46 778 4410 Email: olle.evero@mailbox.swipnet.se
Evestic AB	Olle Evero	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olii.paavola@pyr.fi
FinnAPL	Olli Paavola	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com
First Derivative Analytics Ltd.	Ken Chakahwata	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749328 Fax: 01622 749365 E-mail: martin@gsoft.freereserve.co.uk
General Software Ltd	M.E. Martin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
Godin London Incorporated	Gaëtan Godin	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email: HMWV@4extra.com
H.M.W.Trading Systems	Chris Hogan	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028 Email: bob.hoekstra@khamlin.demon.co.uk
Hoekstra Systems Ltd	Bob Hoekstra	28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
Michael Hughes		IAC Human Interfaces, Unit R21, Auckland New Business Centre, St. Helen Auckland, Bishop Auckland, County Durham, DL14 9TX, UK. Tel: 01388-605544. Email: 100021.3073@compuserve.com
IAC/Human Interfaces	Ian A. Clark	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: 100612.1057@compuserve.com
I-APL Ltd	Anthony Camacho	APL Products, IBM Santa Teresa, Dept M46/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 (=2752) Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
IBM APL Products	Nancy Wheeler	3 S. Tufenin Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: aim@infostroy.spb.su
INFOSTROY	Alexei Miroshnikov	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
Insight Systems ApS	Helene Boesen	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559 Email: info@jsoftware.com
Iverson Software Inc.	Eric Iverson	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124
JAD Software	David Crossley	

Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buld.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022. Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KY00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	42 Queen Annes Road, Bootham, YORK YO30 7AF Email: joh@jaustria.freemove.co.uk
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.rnl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Steven J Halasz	1105 Chicago Avenue, Suite 155, Oak Park, IL 60302, USA. Tel: +1 800 546 4621-1 Email: sjhalasz@interaccess.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: akinloch@globalnet.co.uk
George MacLeod	George MacLeod	37 Newhouse Rd, Bovingdon, Herts, HP3 0EJ, UK. Tel: 01442-834015 Email: GeorgeMacLeod@slimcorp.com
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Millinto Inc.	Dan Baronet	4215 St-Andre, Montreal, CANADA H2J 2Z3. Tel: +1 (514) 529-1434 Email: danb@dsuper.net
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: Ellis@mtlfirm.demon.co.uk
NY Sig	David Siegel	PO Box 2697, New York, NY10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Theo Zwart, Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: info@oasis.nl or rijkse@oasis.nl
Object Oriented Ltd	Walter G. Fil	Am Grendel 2, CH-6004 Luzern, Switzerland. Tel: 41 41 418 70 70 Fax: 41 41 418 70 77 Email: info@object-oriented.com
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	115 Brighton Road, Purley, Surrey CR8 4HE, UK. Tel: 0181-763 2490 Fax: 0181-763 2491
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 706 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 313, Newtown, CT 06470, USA. Tel: +1 (203) 270-9729 Email: rendata@apibooks.cnchost.com or orders@apibooks.cnchost.com
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	413 Cornanthe Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 UK Address: Claridge House, 29 Barnes High St, London SW13 9LW Tel: 0181 8768666 Fax: 0181 8768660 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Laurie Howard	Soliton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email: sales@soliton.com

SovAPL			
Russian Chapter of SIGAPL	Alexander Skomorokhov		PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)31463 Fax: +1 (530) 504 8194 Email: askom@obninsk.com
Strand Software Inc	Anne Faust		19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (612) 470-7345 Email: sales@software.com
Rex Swain	Rex Swain		8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulfhjelrn		Novator Consulting Group AB, Svårdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 C-Serve: 100341,404
Swiss APL User Group			Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr		4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter		PQ Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
Stephen Wynn			8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327236 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist		23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ps.boulder.ibm.com/ps/products/apl2/
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaytum Software	www.adaytum.com/
AFAPL	www.ensmp.fr/~scherer/langlet/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/rb/apljourn.htm
AUSCAN	www.interlog.com/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.inrete.it/cdwk/eng/homee.html
COSY (Bob Armstrong)	www.cosy.com/
Dinosoft Oy	www.dinosoft.fi/
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyadic Systems Ltd	www.dyadic.com/
DynArray	www.dynarray.com/
Eventra	www.eventra.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Hoekstra Systems	www.khamsin.demon.co.uk/about/hsl/noframe.html
IBM APL2	www.torolab.ibm.com/ap/apl/apl2.html
Infostroy	www.insight.dk/infostroy/
Insight Systems ApS	www.insight.dk/
Iverson Software Inc	www.jssoftware.com/
Japan APL Association	www.naska.co.jp/JAPLA/
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	www.users.globalnet.co.uk/~akinloch/akinloch.htm
MicroAPL Ltd	www.microapl.co.uk/
Milinta Inc	www.dsUPER.net/~danb/milinta/
Oasis b.v.	www.oasis.nl/
Renaissance Data	www.aplbooks.cnchost.com/
RE Time Tracker Oy	www.rett.fi/
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Strand Software Inc.	www.jssoftware.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/

Zark Newsletter Extracts

introduced by Jon Sandles

Thank you to the few who noticed that in the last issue we reprinted the same crossword (but with the solution ... so it was a bit easier!) Many apologies for this error, this time we should have a new crossword and a continuation of the tree function analysis puzzles. I once again encourage you all to try and shed new light on these problems, particularly in the context of namespaces or even J locales.

UTILITY CORNER:

Identifiers Within Functions

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In last issue's LIMBERING UP column, we asked you to write a function named *ANALYZE* that analyses the identifiers used in the function whose name is given as the right argument. Consider this function to be analysed:

```

V IND←CHOICES SELECT PROMPT;R
[1]  A Ask for choice:
[2]  ASK:R←CINPUT PROMPT
[3]  A Search choices for 1st char of response:
[4]  IND←CHOICES⍷1←R
[5]  A Ask again if not a valid choice:
[6]  →ASK IF(IND>ρCHOICES)MSG'INVALID CHOICE.'
V

```

Here's an illustration of the *ANALYZE* function:

```

)FNS
ANALYZE SELECT
)VARS
                                     No variables
ANALYZE SELECT
)VARS
ARGS FNS IDS LABELS LOCALS RESULT
ARGS
CHOICES
PROMPT

```

```

      FNS
ANALYSE
SELECT
      IDS
ASK
R
CINPUT
PROMPT
IND
CHOICES
R
ASK
IF
IND
CHOICES
MSG
      LABELS
ASK
      LOCALS
R
      RESULT
IND

```

As you can see, *ANALYZE* builds six global variables. Each of the variables is a character matrix with one name per row. Short names are left-justified. Here's what they represent:

<i>FNS</i>	the names of the functions in the workspace
<i>RESULT</i>	the name of the result variable (0 rows if none)
<i>ARGS</i>	the names of the argument variables (0, 1 or 2 rows)
<i>LOCALS</i>	the names of the variables localised in the header, i.e. preceded by ' ; '
<i>LABELS</i>	the names of the labels, i.e. the names beginning a function line and followed immediately by ' : '
<i>IDS</i>	the names of all identifiers within the body of the function (all lines but the header), ignoring names within comments or quotes; you may include or ignore label names, at your whim.

All six character matrices have the same number of columns, namely as many as the longest name in *FNS*. Any longer names are omitted. Names that begin with the $\square$ symbol, such as $\square IO$ or $\square PW$, are omitted.

Here's one solution to the problem:

```

V ANALYZE FNAME;COLS;CR;HDR;I;LEN;LET;MAX;N;NUM;OK;
  START;VC;WID;IO
[1]  A Analyses the identifiers within the function named
[2]  A in FNAME. In particular, the ANALYZE function
[3]  A creates six global variables: FNS, RESULT, ARGS,
[4]  A LOCALS, LABELS, and IDS. Each of the global
[5]  A variables is a character matrix with one name per
[6]  A row. Short names are left-justified, i.e. padded to
[7]  A the right with blanks. Here's what they contain:
[8]  A
[9]  A FNS      The names of the functions in the workspace
[10] A RESULT   The name of the result variable (0 rows if
[11] A          none)
[12] A ARGS     The names of the argument variables (0, 1,
[13] A          or 2 rows)
[14] A LOCALS   The names of the variables localised in the
[15] A          header, i.e. preceded by '; '
[16] A LABELS   The names of the labels, i.e. the names
[17] A          beginning a function line and followed
[18] A          immediately by a ':'
[19] A IDS      The names of all identifiers within the body
[20] A          of the function (all lines but the header),
[21] A          ignoring names within comments or quotes.
[22] A
[23] A All size matrices have the same number of columns,
[24] A namely as many as the longest name in FNS. Any
[25] A longer names are omitted. Names that begin with the
[26] A IO symbol, such as IO or PW, are omitted.
[27] A
[28] IO=0 A Origin 0 simplifies the logic a bit
[29] FNS=NL 3 A Char matrix of fns in the ws
[30] COLS=1+P FNS A No. cols in each global results
[31] CR=CR FNAME A Canonical rep (char mat) of fn
[32] WID=P HDR+CR[0;] A Fn header and width of CR
[33] A HDR=(HDR\A')+HDR A In APL2, drop any hdr comment
[34] CR=1 0+CR A Body of fn
[35] LEN=('A'+HDR)*HDR\A' A Length of result var if any
[36] A RESULT is 0 or 1 row matrix (0 if too long)
[37] RESULT=((LEN*COLS)\A*LEN),COLS)P COLS+LEN+HDR
[38] HDR=(LEN+*LEN)+HDR A Drop result var and 'A'
[39] HDR=(+/V\HDR*')\HDR A Delete trailing blanks
[40] LEN=HDR\';' A Length of fn syntax
[41] ARGS=LEN+HDR A Select syntax
[42] HDR=LEN+HDR A Drop syntax, leaving semicolon
[43] I=(ARGS=' ')/\LEN A Indices of blanks
[44] N=P LEN+(I,LEN)-0,I+1 A Lengths of names in syntax
[45] MAX=I/\LEN A Length of longest name
[46] A Convert to a matrix:
[47] ARGS=(N,COLS)+(N,MAX)P(LEN*.*MAX)\ARGS-' '
[48] A Select args, not fn name, unless too long:
[49] ARGS=((LEN*COLS)\(N>3 1 2)/1 0 1)F ARGS
[50] I=(HDR\';')/\LEN-P HDR A Indices of semicolons

```

```

[51] N←pLEN+(1+I,LEN)-I+1  a Lengths of localised names
[52] MAX←[LEN  a Length of longest name
[53] a Convert to a matrix
[54] LOCALS←(N,COLS)+(N,MAX)p(,LEN+.>MAX)\HDR-';
[55] LOCALS←(LEN,COLS)/LOCALS  a Ignore long ones
[56] OK←\CR='''  a Flag code NOT within quotes
[57] a Flag code NOT within comments or quotes:
[58] OK←OK^'\OK=CR='a'
[59] a Chars that can begin identifier names:
[60] LET←'ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
      tuvwxysΔ'
[61] a Other valid chars (plus - and _ if allowed):
[62] NUM←'0123456789'
[63] a Flag any valid chars, adding col for ending IDs:
[64] VC←,(OK^CR^LET,NUM),0
[65] a Flag start and end (+1) of each run:
[66] VC←VC*-1+0,VC
[67] N←pIDS+VC/1pVC  a Convert to indices
[68] IDS←((N+2),2)pIDS  a Two-col mat of start/end inds
[69] CR←,CR,' '  a Ravel code and pad to match VC
[70] START←IDS[;0]  a Start indices
[71] LEN←IDS[;1]-START  a Lengths of names
[72] a Must start with a letter, not be too long, and not
[73] a be preceded by a '□' (e.g. □IO):
[74] OK←(CR[START]≠LET)^(LEN,COLS)^(□)*CR[0[START-1]
[75] START←OK/START  a Squeeze out bad ones
[76] N←pLEN-OK/LEN
[77] IDS←(N,COLS)p' '  a IDS in raveled form
[78] a I←(LEN[0]),(LEN[1]),(LEN[2]),...:
[79] I←LEN/1+0,+LEN
[80] I←(1pI)-I
[81] a Insert IDS where they belong
[82] IDS[I+LEN/COLS*N]+CR[I+LEN/START]
[83] IDS←(N,COLS)pIDS  a Reshape to a matrix
[84] a Labels start a line and are followed by a colon:
[85] LABELS←((0=(WID+1)|START)^CR[START+LEN]=':')/IDS

```

v

Though this function is extensively commented, some additional remarks are in order. We described the $\square CR$ (canonical representation) and $\square NL$ (name list) system functions in the last issue of the 'News. The former returns a character matrix image of the function and the latter returns a character matrix of the names of the functions in the workspace (for $\square NL$ 3). Let's trace some of the important lines:

```

[29] FNS:  ANALYZE      These are the only functions
      SELECT          in the workspace

```



```
[70] START:43 47 49 56 129 133 141 143 216 220 223 228 236
[71] LEN: 3 1 6 6 3 7 1 1 3 2 3 7 3
[74] OK: 1 1 1 1 1 1 0 1 1 1 1 1 1
[75] START:43 47 49 56 129 133 143 216 220 223 228 236
[76] LEN: 3 1 6 6 3 7 1 3 2 3 7 3
```

```
[80] I: 0 1 2, 0, 0 1 2 3 4 5, 0 1 2 3 4 5, 0 1 2,
      0 1 2 3 4 5 6, 0, 0 1 2, 0 1, 0 1 2,
      0 1 2 3 4 5 6, 0 1 2
```

```
[82] CR[I+LEN/START]
      ASKRCINPUTPROMPTINDCHOICESRASKIFINDCHOICESMSG
```

```
[83] IDS: ASK
        R
        CINPUT
        PROMPT
        IND
        CHOICES
        R
        ASK
        IF
        IND
        CHOICES
        MSG
```

```
[85] LABELS: ASK
```

Incidentally, the $\backslash$ function, which converts pairs of 0s to runs of 0s, and the $\wedge$ function, which converts all 1s after the leading 1s to 0s, can be replaced by the $\neq$ function, which converts pairs of 1s into runs of 1s, and the $\vee$ function, which converts all 0s after the leading 0s to 1s. Here are the two sets of logic:

Current logic:

```
OK $\leftarrow$ \CR $\neq$ '''' a Flag code NOT within quotes
a Flag code NOT within comments or quotes:
OK $\leftarrow$ OK $\wedge$  $\wedge$ \OK $\leq$ CR $\neq$ 'A'
a Flag any valid chars, adding col for ending IDs:
VC $\leftarrow$ , (OK $\wedge$ CR $\in$ LET, NUM), 0
```

Alternative logic:

```
NOK $\leftarrow$  $\neq$ \CR='''' a Flag code within quotes
a Flag code within comments or quotes:
NOK $\leftarrow$ NOK $\vee$  $\vee$ \NOK $\leq$ CR='A'
a Flag any valid chars, adding col for ending IDs:
VC $\leftarrow$ , (NOK $\leq$ CR $\in$ LET, NUM), 0
```

What happens if you use the *ANALYZE* function to analyse a function named *START*?

It won't work. Why? Because the name *START* has been localised in the header of the *ANALYZE* function and is used therein as a variable. Hence, the global definition, the *START* function, is "shadowed" by the local *START* variable while *ANALYZE* is running. In particular, the *INL* system function (line [29]) sees *START* as a variable, not a function, and so does not return it in its list of function names. Likewise, the *OCR* system function (line [31]) returns an empty character matrix since its argument is the name of a variable, not a function.

This shadowing problem exists for all the names local to *ANALYZE*, including the argument *FNAME* and the names of any labels (luckily there aren't any). How can this problem be avoided?

It can't. But you can minimise its impact by using bizarre variable names - names that aren't likely to be used as function names. A name like *Δ67ΔXX* is pretty unlikely to be used as a function name. Unfortunately, screwball names like *Δ67ΔXX* will tend to make the *ANALYZE* function pretty unreadable. A good compromise is to use names that are just a little screwy, like *ΔSTART* or *ΔΔSTART* or *ΔSTARTΔ*.

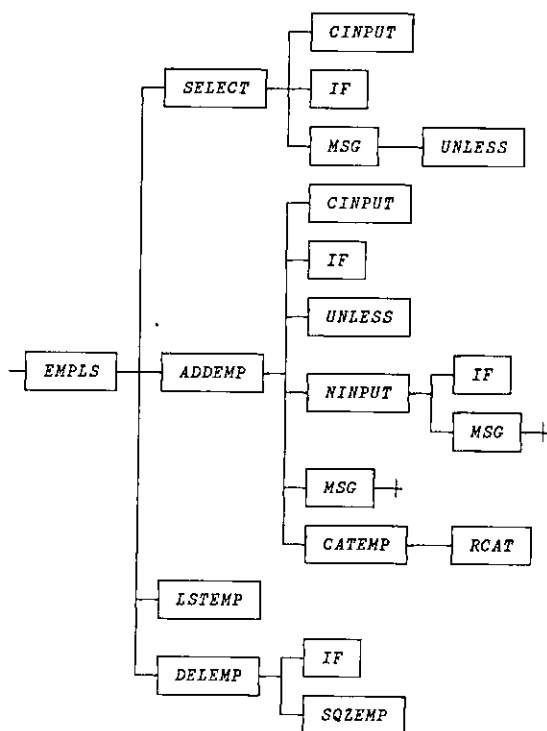
For now, we'll leave the *ANALYZE* function as it is, and pray that we won't need to analyse any functions with shadowed names.

LIMBERING UP: Identifiers Within Functions

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

Two issues ago, we introduced a utility function name *FNTREE* that displays a tree diagram of the subfunctions (their subfunctions) called by a specified function in the workspace. The right argument of *FNTREE* is the name of the function to be diagrammed. The result is a character matrix of the tree diagram. See over

FNTREE 'EMPLS'



Last issue, we discussed the logic for constructing the tree diagram. This issue (in the UTILITY CORNER column), we discuss the logic for analysing the identifiers used within a function. Next issue, we'll present the finished product, in which we put it all together. In the meanwhile, one sticky problem remains.

Given the global variables *FNS*, *RESULT*, *ARGS*, *LOCALS*, *LABELS*, and *IDS* generated by the *ANALYZE* function, you still need to determine which of the identifiers represent subfunctions. In particular, the names in *IDS* that are found in *FNS*, but are not found in *RESULT*, *ARGS*, *LOCALS*, or *LABELS*, are subfunctions.

We'd like to express these subfunctions as a vector of row indices into *FNS*. But how?

If these six global variables were vectors of identifying numbers, rather than matrices of identifying names, the logic could proceed like this:

```

a Make the identifiers distinct:
IDS+((IDS\IDS)=1pIDS)/IDS
a Eliminate those that are local:
IDS+IDS~RESULT,ARGS,LOCALS,LABELS
a Consider only those globally defined as functions:
IDS+((IDS=FNS)/IDS)
a Convert to indices into FNS:
IDS+FNS\IDS

```

Straightforward but worthless. The six variables are matrices, not vectors. The dyadic $\vee$, ϵ , and $\sim$ functions work on vectors, not matrices.

Your task is to write a set of APL expressions that generate the desired result. In particular, incorporate the expressions in a function named *SUBFNS*, which accepts all six variables (*FNS*, *RESULT*, *ARGS*, *LOCALS*, *LABELS*, *IDS*) as global arguments and which returns as an explicit result the row indices into *FNS* of the distinct rows of *IDS* not found in *RESULT*, *ARGS*, *LOCALS*, or *LABELS*, but found in *FNS*.

To illustrate, suppose the six variables have the values indicated earlier in this newsletter, except suppose *FNS* has five rows:

```

      FNS
ANALYZE
CINPUT
IF
MSG
SELECT
      SUBFNS
2 3 4

```

The result (2 3 4) indicates that the only names in *IDS* that are in *FNS* but not in *RESULT*, *ARGS*, *LOCALS*, or *LABELS* are the names found in *FNS*[2 3 4;].

The *SUBFNS* function should be as efficient as possible. If your APL system supports nested arrays and you believe a nested solution is the most efficient, you're welcome to use nested constructs. In APL2 use $\leftarrow [2]MAT$ to convert a matrix to a vector of vectors; in APL*PLUS, use $\leftarrow MAT$; in Sharp APL use $\leftarrow MAT$.

Please send your solutions (on paper or disk) to:

Vector Production, Brook House
Gilling East
York YO63 4JJ, UK

or via email to Vector Production at apl385@compuserve.com.

We also welcome your version of the completed *FNTREE* function. Look over the discussions in the last few issues of the Zark APL Tutor News and decide how you would put it all together.

Finally, we'd like to make a request to the artist in you. Those of you who use the Zark APL Tutor are familiar with the faces of the "APL Creators." Throughout the course, the faces pop up to debate various design objectives and implementation issues.

We recently received a note from a Tutor user who wanted to give our Creators a face-lift. He offered some new faces. Below are the Creator faces currently in use or under consideration. Please drop us a note voting for your favourites. We even welcome any new faces you design. Some restrictions though:

1. The face must fit within a 7-row, 11-column character matrix (including the "APL CREATOR NUMBER n" label).
2. Since the Tutor runs under APL*PLUS PC, APL*PLUS II, and APL2 mainframe, you should try to use only characters that are common to these (and future) platforms. In other words, avoid characters that are too strange (like © on the PC) or that aren't supported on some display terminals (like = and on the mainframe.)
3. The face should not look too much like the face of an actual APL creator, lest he sense the frustrated vibes of a Tutor voodoo user. (Say THAT five times fast!)

```

  /-----\
  ( |  @   @  | )
  |   A   |
  |  \_/  |
  \-----/
  APL CREATOR
  NUMBER 1

```

```

  /=====\
  (  @   &  )
  |   >   |
  |  -  /  |
  \-----/
  APL CREATOR
  NUMBER 2

```

```

  /-----\
  <| '0' '0'|>
  |   ^   |
  | (\_/) |
  \-----/
  APL CREATOR
  NUMBER 3

```

```

  ,~~~~~,
  {{{{ _}
  {{ C <@ \
  {{{ ._|"
  {{____}
  APL CREATOR
  NUMBER 4

```

```

  /-----\
  ( | + ~ ~ | )
  |   A   |
  |  u--  |
  \-----/
  APL CREATOR
  NUMBER 5

```

```

  ^^^^^^^^
  ( -o o- )
  |   u   |
  |  ~  /  |
  |   _   |
  APL CREATOR
  NUMBER 6

```

```

  //////////////
  |  _  _  |
  | (-)~(-)|
  |   @   |
  |  IIIII |
  \-----/
  CREATOR 7

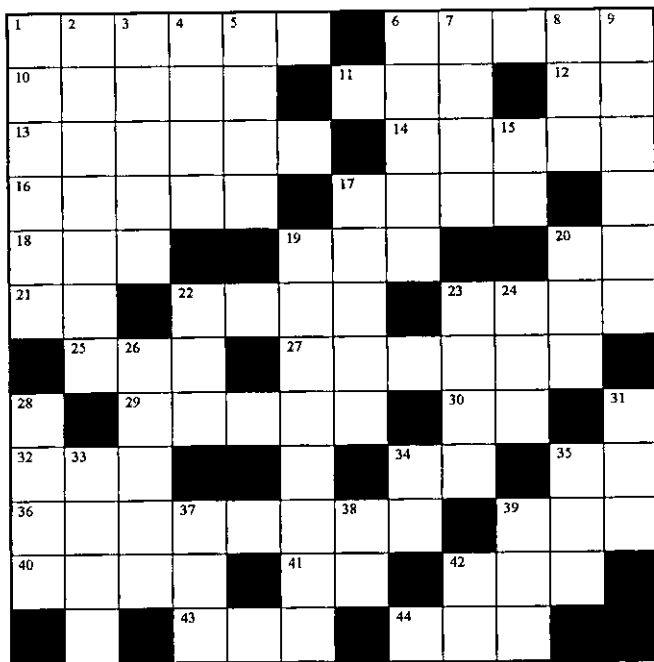
```

```

  \\\\\\\
  |  _  _  |
  | (-)~(-)|
  |   u   |
  |  IIII  |
  \-----/
  CREATOR 8

```

Crossword



Across

1. In the beginning...
6. Where did these come from?
10. The indices of the 1 bits in the vector P
11. Wait awhile
12. $'(15) \times 5' \sim '(5)'$
13. Half of S, rounded up, times C
14. $(R + 19 - R), 1R$
16. Where to → to get to L5 when OK=1
17. The end
18. $-L - P$
19. The last line of a loop that begins $LP: \rightarrow (N \& 1 t; I + I + 1) / 0$
20. The remainder when dividing by Y
21. The quotient when Y is the numerator
22. Where L0 is first located in the vector N
23. $A \leq B$ for Boolean A and B

25. $2 \uparrow M$ where $(\rho, M) \geq 2$
27. Round A to the nearest integer, but go DOWN if the fraction is .5
29. $-N - 2N - 3N \dots -DN$ (origin 1)
30. $\phi 2 + 1 \phi 2 + ' \& l t ; s = z \& g t ; \# '$
32. Look at a function under program control
34. $(\sim \square IO) + . + 14$
35. What all APL*PLUS file functions have in common
36. Do $(\sim A \geq 36) [I]$ most efficiently
39. APL's Trojan Horse
40. Very important person power
41. $3 + . \times 15 \circ . + 0 \ 1$
42. Needed, to tell you if a variable or function exists
43. $(\times / 19) \div (\times / 12) \times (\times / 19 - 2)$
44. The source of all randomness

Down

1. Retrieve with caution
2. $(\sim 1 \ 1) [\square IO + C / K]$
3. $(\psi L) [\square IO]$
4. E copies of the single letter SL
5. $AP \times . 2$
6. This workspace is no longer needed
7. $(WL + F), WL + F$ for positive WL
8. $' \rho (I / 1 \rho I), \square IO + \rho I$
9. The magnitude of the first nine elements of the matrix A, scaled by D
15. $+ \backslash F \rho 1$
17. Retrieve a stored workspace
19. Go to line L if X is less than 99 (where $L \neq 0$)
20. $(Y \neq 5) / Y$
22. Multiply each value in the matrix NM by ...
23. $\sim B 0$
24. Mismatch
26. $(\rho NR) + (\rho P)$ for vectors NR and P
28. How many different characters are there?
31. Define a function under program control
33. The Ith value in the vector C
34. $2 * 2 * 2$
35. Where the tough go ($\rightarrow$) after tough going
37. $I \times I$
38. $\times / \sim 1 + \times \backslash 2 + \phi + \backslash 2 \rho 1$
39. $) VARS$ or $) FNS$, under program control
42. $\square + R + ' \square + R ' \sim ' + '$

J-ottings 23: Just what do they sell at C. and A.?

by Norman Thomson

Are you the sort of person who shuffles things around, papers on your desk, people in your department and so on? If so, read on, this is for you. A *permutation* is just a shuffle of items within an object. It can be described in two ways, either as what you *do* in the course of the shuffle (e.g. switch items four, five and two around cyclically, and also items one and three), or as a display of the end result (e.g. items finish in order three, four, one, five, two). The former type of description, that is of the *operation*, is logically prior to the latter. Any permutation is describable as a product of non-overlapping *cycles* (see the following paragraph for what this means), whereas the end result is the vector of positions which results from the execution of these cycles.

In J, the verb C. transforms either of these two descriptions into the other. The first kind is always boxed which can be ensured by using either of the "box" or "link" verbs. For example, <7 3 6 4 should be interpreted as

"item 7 is replaced by item 3 which is replaced by item 6 which is replaced by item 4 which is replaced by item 7",

in other words think of "is replaced by item" as being inserted in the gaps in a cycle vector as in the initial preparation for evaluating "function insert".

The C. verb thus forces an explicit distinction between a permutation in the *operational* sense, that is a set of non-overlapping cycles, and the *image* of such a permutation, which unfortunately is also popularly referred to as a permutation. Denote by n the number of items available to be permuted (this is called the *order* of the permutation). If $n = 3$, then, using the definitions just given, <2 1 0 is a permutation whose image is 2 0 1. If $n = 4$ the image of the same permutation is 2 0 1 3. In other words the image of an operational permutation depends on its order. Images can be used as indexes, for example with the "from" verb as in 2 0 1 { 'ABC' which evaluates to 'CAB'.

Permutations are in general *products* of disjoint cycles so that a cycle 2 0;3 1 has as its image C.2 0;1 3 which is 2 3 0 1, and conversely the cycle representation of 2 3 0 1 is C.2 3 0 1 which is 2 0;3 1. The one-to-one correspondence between cycles and images can be guaranteed (and is so guaranteed in J) by making the highest index leftmost within each cycle, and

ordering the cycles in increasing value of leftmost elements. Thus the permutation 1 3;2 0 is identical to 2 0;3 1, which is how it is best to think about it, although C. 1 3;2 0 evaluates correctly to 2 3 0 1.

The dyadic form of C. , which is called "from direct", is very useful for shuffling items within objects. For example, if ten items in an object, which could be either upper-case letters (say $\text{ten} = (.65 + i.10)\{a.\}$), or rows of a character matrix, represent ten people, and it is desired to switch the fourth and fifth, and also the second and eighth, use

```
(4 3;7 1)C.ten
AHCEDFGBIJ
```

The left argument can be either a permutation or a permutation image, and it is applied directly to the items of the right argument.

There is an important distinction between "from direct" and explicit "from". This is demonstrated by

```
(C.4 3;7 1){ten
AHCEDFGB
```

Now consider A. which is a verb called "atomic permute". The force of the word "atomic" is that all the permutation images of i.3 say can be listed in increasing numerical order by

```
(i.13)A.i.3
0 1 2
0 2 1
1 0 2
1 2 0
2 0 1
2 1 0
```

Replacing each image by its ordinal number reduces it to an integer from the set i.13 - think of this as an "atom". If the order n is large this can be a useful shorthand. Monadic A. supplies this shorthand, so that for example A. 3 2 1 0 (the final permutation of order 4 in numerical sequence) is 23. Image numbering depends on the order n, so that the reverse process, that is moving from atomic number to image provides an obvious niche for dyadic A. :

```
23 A. i.4
3 2 1 0
```

A more general way of looking at the dyadic form of A . is to view its left argument as an atomic number identifying a permutation to be applied to the smallest applicable sub-vector at the right hand end of the right argument y . For example 1 $A. y$. shuffles the last two items in an object, 2 $A. y$. through 5 $A. y$. shuffle the final three items, 6 $A. y$. through 23 $A. y$. shuffle the final four items, and so on.

It is easy to construct the algebra of composing successive permutations of a given order as tables of atoms, for example

```
each=.8>
pans3=:3 :0      NB. table of compositions of permutations of
:                NB. order 3 in sequence by atomic number
A.(x.A.t){y.A.t=.i.3
)
>pans3 each/~ i.6
0 1 2 3 4 5
1 0 3 2 5 4
2 4 0 5 1 3
3 5 1 4 0 2
4 2 5 0 3 1
5 3 4 1 2 0
```

The leading diagonal has just two non-zero entries, which suggest the cyclic nature of atoms 0, 3 and 4. They form a subgroup (see J-ottings 22) in their own right; reorganising the table, and editing its output shows its fundamental structure:

```
>pans3 each/~ 0 3 4 1 2 5
0 3 4 1 2 5
3 4 0 5 1 2
4 0 3 2 5 1
.
1 2 5 0 3 4
2 5 1 4 0 3
5 1 2 3 4 0
```

This non-symmetric group is isomorphic to the dihedral group D_3 (again see J-ottings 22) which describes the symmetries of the equilateral triangle. 0, 3 and 4 correspond to rotations by 0, $\pi/3$ and $2\pi/3$, and 1, 2 and 5 to reflections in the three perpendicular bisectors of the sides. The subgroup formed by (0,3,4) is called the *cyclic group* C_3 .

The isomorphism between permutations of order 3 and the symmetries of the equilateral triangle is simple to envisage. The step from order 3 to order 4 however involves a large leap in the size of the underlying groups since there are

now 24 permutations and so the group table has 576 entries. The table is obtained by `>pans4 each/~ i.24 .` (`pans4` is the same as `pans3` except that '3' is replaced by '4'), and an edited subsection of it is given below.

	6	7	8	9	10	11	12	13	14	15	16	17
	...											
12	14	20	0	21	1	15	8	22	2	23	3	9
13	15	21	1	20	0	14	9	23	3	22	2	8
14	..	12	18	2	23	4	17	6	19	0	21	5
15	13	19	3	22	5	16	7	18	1	20	4	10
16	17	23	4	18	2	12	11	21	5	19	0	6
17	16	22	5	19	3	13	10	20	4	18	1	7

The full table is like a multiplication table which you would look up to find the effect of applying any two permutations of order 4 in sequence. For example 13 A. 7 A.i.4 has the same permutation image as 21 A.i.4, and permutation 10 has the effect of reversing permutation 13. The table is non-symmetrical as the second six by six block above shows, but it is full of highly intertwined internal symmetries.

Inverses: reshuffling back to where you were

The image permutation which restores the natural numerical ordering of the image permutation γ is $/:\gamma$. So if the ten people represented by upper case characters `ten = 'ABCDEFGHIJ'` find themselves in the order `ICEDHFB AJG`, the operational instructions to restore them to natural order are

`C. /: ten i.'ICEDHFB AJG'`

which is the permutation `3;5;9 8 0 7 4 2 1 6`. The instructions therefore are to tell the fourth and sixth people (that is numbers 3 and 5) to stay put; the person in the ninth position is to move to the tenth, the person in the first position to move to the ninth, and so on.

A permutation consisting only of 2-cycles is self-inverse. For $n = 4$, there are ten of these, one containing no 2-cycles, six containing a single 2-cycle corresponding to the 6 ways in which a pair of items can be chosen from four, and three containing two pairs of 2-cycles, viz. `3 2;1 0`, `3 1;2 0` and `3 0;2 1`. There are eight permutations with 3-cycles which form four mutually inverse pairs such as `<3 0 2` and `<3 2 0`, and finally the number of 4-cycles is six, since a 4-cycle must have 3 as its first item and there are six ways in which the remaining three items can be permuted. Permutations can be classified as odd or even according to whether they are attainable as the result of an odd or an even number of successive two-item swaps from `i.n`. For example, `0 1 2 3->0 1 3 2->3 1 0 2`, so

3 0 1 2 is the image of an even permutation. Analogously to the equilateral triangle, even permutations of $i.4$ correspond to the non-reflective symmetries of the tetrahedron, that is those which are physically realisable within three dimensional space.

The first six permutations in atomic ordering have 0 in their first position, the next six have 1 and so on. However apart from this, labelling by numerical order does not reflect logical structure within the set of all permutations of order 4. The table below classifies these permutations. In the properties column, O means Odd, S means Self-inverse, and D indicates a Derangement, that is a permutation in which *every* item has moved from its home position. Subgroups of order 2 can be formed by combining 0 with any of the nine non-identity self-inverse permutations. The following form subgroups of orders 4, 8 and 12:

{0,7,16,23} {0,2,21,23} {0,1,6,7} {0,5,14,16}
 {0,7,16,23,6,1,17,22} {0,7,16,23,14,9,5,18}
 {0,3,4,7,8,11,16,12,15,23,19,20}

of which the last is the set of even permutations, called the *alternating group* A12.

A. no	Image	Permutation	Properties	Inverse
0	0123		S	
1	0132	<32	O S	
2	0213	<21	O S	
3	0231	<312		4
4	0312	<321		3
5	0321	<31	O S	
6	1023	<10	O S	
7	1032	32;10	S D	
8	1203	<201		12
9	1230	<3012	O D	18
10	1302	<3201	O D	13
11	1320	<301		19
12	2013	<210		8
13	2031	<3102	O D	10
14	2103	<20	O S	
15	2130	<302		20
16	2301	31;20	S D	
17	2310	<3021	O D	22
18	3012	<3210	O	9
19	3021	<310		11
20	3102	<320		15
21	3120	<30	O S	
22	3201	<3120	O D	17
23	3210	30;21	S D	

And this is only the beginning of what they have at C. and A. - why don't you pop round to the store and explore a bit for yourself?

Puzzle: Funny Cube

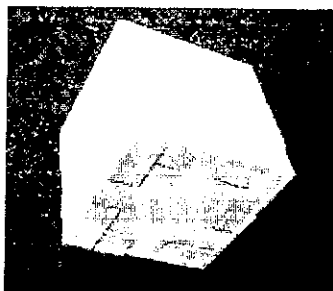
by Bernard Legrand

During a recent exhibition, PROGILOG, held in Paris last November, the company Euriware gave me a funny advertising object: a little 3D puzzle.

The puzzle is made of six pieces, which are in fact the faces of a cube. The aim is to build the cube by embedding the pieces one into the other.

Only 6 pieces: it looks very easy!

In fact the multiple possibilities make the trick more complex than one might think.

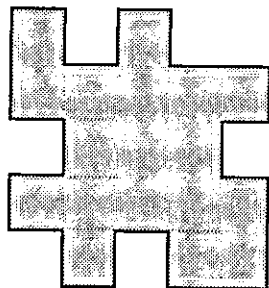


That gave me the idea of a little problem that I am offering both to the members of the French APL association (AFAPL), and to the readers of *Vector*.

Representations

Each piece is a 5 by 5 square, the sides of which are notched as shown here.

The piece thickness is equal to 1, which makes it possible to fit pieces.



In APL, one can represent such an object by a binary matrix, like this:

```
1 0 1 0 0
1 1 1 1 1
0 1 1 1 0
1 1 1 1 1
0 1 0 1 1
```

Of course one may rotate that piece by multiples of $\pi/2$, to obtain four configurations, which will be named "State 0" to "State 3", depending on the number of $\pi/2$ rotations applied.

State 0	State 1	State 2	State 3
1 0 1 0 0	0 1 0 1 1	1 1 0 1 0	0 1 0 1 1
1 1 1 1 1	0 1 1 1 1	1 1 1 1 1	1 1 1 1 0
0 1 1 1 0	1 1 1 1 0	0 1 1 1 0	0 1 1 1 1
1 1 1 1 1	0 1 1 1 1	1 1 1 1 1	1 1 1 1 0
0 1 0 1 1	1 1 0 1 0	0 0 1 0 1	1 1 0 1 0

But: it is strictly forbidden to turn the piece over.

Input Data

You are given the 6 matrices which represent the six pieces, each given in its original position (State 0) ; here they are:

Piece 1	Piece 2	Piece 3
1 0 1 0 0	0 0 1 0 1	0 1 0 1 0
1 1 1 1 1	0 1 1 1 1	1 1 1 1 0
0 1 1 1 0	1 1 1 1 0	0 1 1 1 1
1 1 1 1 1	0 1 1 1 1	1 1 1 1 0
0 1 0 1 1	0 1 0 1 1	1 1 0 1 1
Piece 4	Piece 5	Piece 6
0 0 1 0 0	1 1 0 1 0	1 0 1 0 0
1 1 1 1 0	0 1 1 1 0	1 1 1 1 0
0 1 1 1 1	1 1 1 1 1	0 1 1 1 1
1 1 1 1 0	0 1 1 1 0	1 1 1 1 0
0 0 1 0 0	0 1 0 1 0	0 0 1 0 0

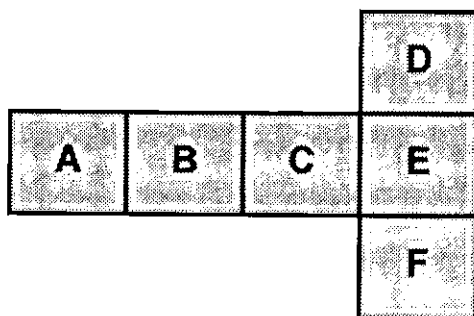
That set will be represented, in APL by a unique binary array, *Bin*, of dimensions $6 \times 5 \times 5$. Because one could imagine many different shapes for the pieces, that binary array *Bin* will be our input data.

The Game

You just have to find out ALL the valid possible ways of building the cube (maybe there are more than one, who knows ?)

But how can we represent each solution? Here again, we need conventions. We have decided on the following conventions:

1. Among the different representations of a cube, we have chosen the following one, where the faces have been named "A" to "F":



2. It is also decided that the face "A", in the leftmost position, will be occupied by the piece used as an example in the first paragraph, in its original state (piece number 1, in 0 state). It will be called "*Base piece*".
3. In order to represent each element of the solution, just give the piece number (1 to 6), followed by its rotation state (0 to 3).

With these conventions, any solution can be represented as shown here:

(but, sorry: this is **not** a solution)

Place	Piece	State	
A	1	0	(Base piece)
B	4	2	
C	3	0	
D	6	1	
E	5	1	
F	2	3	

1 0

4 2

3 0

6 1

5 1

2 3

Each solution can simply be represented by a 6x2 APL matrix.
The "solution" above would give:

Your Program

Your (monadic) program should take, as its right operand, the $6 \times 5 \times 5$ binary matrix *Bin*, given above, and produce as its result an $N \times 6 \times 2$ numeric array, where N is the number of solutions, each of which is represented as a 6×2 matrix, according to the conventions given in the previous section.

Extension

The problem can be extended if you are allowed to turn one or more pieces over, except the *Base piece*, which must remain unchanged (just to avoid symmetrical solutions).

To have the same representations for everybody, I suggest two additional conventions:

- the pieces will be turned over using a "reverse" along the last dimension: ϕ
- the rotations will be made after a piece has been turned over.

You can write a second program which will have an optional left operand, like this:

0 : pieces cannot be turned (the default)
1 : pieces can be turned upside down.

This new program will give a result of 3 columns:

- Piece number (1 to 6)
- Has it been turned over (0 or 1) ?
- Number of $\pi/2$ rotations applied (0 to 3)

Send your solutions (in any APL, J, K, O, R, ...) to *Vector*, of course, [email: apl385@compuserve.com. Postal address c/o Vector Production, see inside back cover], but please also email them to the French APL Association secretary, Mrs Ludmila Lemagnen, at the following address:

lemagnen@aol.com

I hope you will have fun with the game!

Bernard Legrand, Legrand Consultants
President of the French APL Association AFAPL

My SAX Experience – Sharp APL for Linux: Beta version 5.1.0

reviewed by Bob Hoekstra (Bob.Hoekstra@khamhsin.demon.co.uk)

Abstract

Some readers may be familiar with SAM, the mainframe version of I.P. Sharp's APL interpreter. For some years SAX, the Solaris and AIX variant of this interpreter, has been establishing itself as a serious force. Now a version of SAX is becoming available for Linux, the free operating system that is rapidly growing in popularity.

Introduction

I have written before about my fanaticism regarding the combination of APL and UNIX [1] or Linux [2], so I won't repeat this. While at APL98 I heard the rumour that Soliton was porting SAX to Linux, I got very excited and collared their representative on the spot, demanding the full scoop and pledging my support (and a *Vector* review) if I could have a copy soon!

Linux

I imagine that everyone has heard of Linux [2] by now. For those that haven't, it is an alternative (i.e. non-Microsoft) UNIX-like operating system for the PC. Its development and growth has been very rapid with many people, possibly thousands, having a hand in it. Until quite recently there were relatively few applications available for Linux, but this is rapidly being rectified with the support of some large and well known software houses [5].

In the absence of good applications, Linux on the desktop was mainly of interest to those who wanted to get involved in the development of the operating system itself. However, it has been an excellent platform for providing file and print services to clients running many operating systems for a long time (in computer terms). As the applications appear, Linux is rapidly becoming a suitable platform for developing all sorts of applications in many different languages as well as providing general "office productivity" functions (word processor, spreadsheets, etc). Linux is very efficient and will support multiple simultaneous users on even quite modest hardware. With the release of SAX there is another good reason for

APLers to run Linux (currently J is available for Linux, as are some "non-industrial strength" APLs).

Linux has been ported to many hardware platforms besides those based on Intel processors. In the current release, SAX is only available for the Intel platform. This may change, but I do not know Soliton's intentions in this regard.

It is my personal hope that SAX will also create some interest amongst those who have not used APL until now. I am just as interested in enticing the current Linux user to investigate APL as encouraging APLers to use Linux. I have always believed that the properties of UNIX complement those of APL very well and that the two provide a powerful combination.

Can I use SAX?

In their announcement in *Vector*[3], Soliton announced that "...Personal licences for SHARP APL for Linux will be available free of charge...", all you have to do is download the files from their web site [6]. Note that commercial licences will not be free. I am not sure of the exact pricing, so I will not repeat rumours I have heard. However, I am confident that the price will be acceptable.

So the answer to the question is yes: anyone can use SAX. This sounds too good to be true, and it is, for the following reasons:

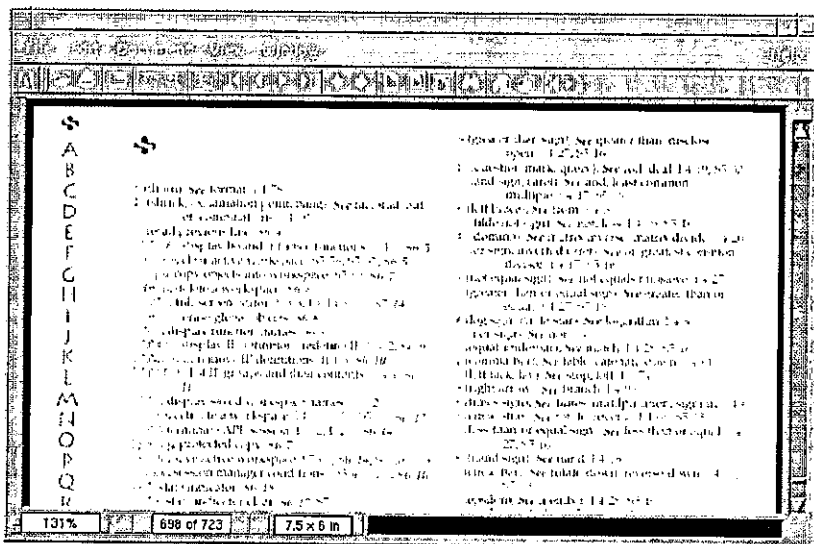
Firstly, those who have not dipped their toe into the Linux waters yet will have to install the operating system. This is not a trivial operation. However, I will not devote much space to this task (this is an APL review after all) but I will make a few suggestions:

- Try to get one of the mainstream variants of Linux to install. I would recommend the Red Hat [7] or SuSE [8] distributions as easy to install. By contrast, the Debian [9] distribution is not meant for the first time user. Downloading a complete copy from the net is impractical, but distributions on CD are easy to come by. A how-to book might also be a good idea. In Europe I can recommend The Linux Emporium [10] for all Linux requirements, but there are other suppliers.
- Anyone doing this for the first time should have a spare hard disk to install on, and take the disk containing the current operating system out of the machine (unless you are happy to lose the information on it).
- It might be useful to get some help. Most experienced Linux users are only too happy to help a newbie (maybe you can get them interested in APL). Remember to install and configure an X server, and lastly to make a new kernel to suit your particular hardware configuration.

Secondly, the SAX download can take quite a while. There is the software itself, a single gzipped tar file of around 2.5MB and an instruction file of about 1KB. You will also want the documentation, a pdf file of 6.1MB (see below). I had access to a T3 internet connection, but someone downloading with a conventional modem could easily take more than an hour to do this.

The Supplied Documentation

The file `sax5.pdf` contains the manual. This is an impressive document: 723 pages! At times I have struggled to find what I was looking for, but in the end found the information required. This is where a powerful machine pays off, as SAX will run quite happily on a pedestrian PC with Linux installed, but using Adobe's Acrobat reader to do string searches in a document of this size is a different matter! The documentation is in general well written and the pdf file is well supplied with links, so navigation is not a problem most of the time. In fact, I got to rather like this part of SAX: `acroread` is often easier to use than a book.



Installing SAX

Assuming you have Linux installed, the installation of SAX is easy. All one has to do is follow the instructions in the instruction file, `install.txt`. This file gives clear and sensible instructions on the installation, and within a few minutes it's all done.

I have just a few negative comments about these instructions:

- Using `gunzip` on the downloaded file `sax-5.1-linux.tgz` is not necessary. If we have the GNU version of `tar` (and most Linux distributions do), we can `unzip` and `untar` in one command as follows:

```
$ tar xvzf sax-5.1-linux.tgz
-rw-r--r-- root/root      2611828 1999-10-19 00:59 bundle.gz
-r--r--r-- root/root       2543 1999-10-21 15:26 README
-rw-r--r-- root/root       3067 1999-10-18 22:58 disclaimer
-rwxr--r-- root/root       5529 1999-10-21 16:55 SAXLinstall
```

The reason for doing this is that it saves some disk space, though in this case the saving is surprisingly small:

```
# ls -l sax-5.1-linux.tgz
-rw-rw-r-- 1 root root      2612398 Dec 14 11:49 sax-5.1-linux.tgz
# gunzip sax-5.1-linux.tgz
# ls -l sax-5.1-linux.tar
-rw-rw-r-- 1 root root      2631680 Dec 14 11:49 sax-5.1-linux.tar
```

This probably indicates that the file is compact and does not compress very effectively.

- More importantly, I disagree with their last instruction regarding the setting of globals. This tells the installer to adjust every user's environment by adding an environment variable `SAXDIR` and to extend the `PATH` by adding the binary subdirectory of the installation directory (`/usr/sax/rel/bin` by default). This is not always good practice, and furthermore the instruction would only work for those using the Bourne shell or its derivatives and not for those using the C-Shell family.

For those new to Linux: Soliton doesn't say which file to edit in their instructions. This should be `/etc/profile` and corresponding adjustments should be made to `/etc/.login` if it exists. Alternatively, use a "wrapper" as I do.

I firmly believe in "wrappers". A wrapper in this context would be a shell script in a common directory (I invariably use `/usr/local/bin`) with the same name as the required executable which sets any necessary environment variables before calling the executable. My wrapper in this case would contain:

```
#!/bin/bash
#####
# /usr/local/bin/sax # Wrapper script to start SAX/L
# Bob Hoekstra, 1999.12.14
#####
export SAXDIR=/usr/sax/rel
export PATH=${SAXDIR}/bin:$PATH
exec ${SAXDIR}/bin/sax
```

I won't go into all the advantages of using wrapper scripts as I don't want to get into the details of UNIX administration, so I will leave this subject here.

I am always interested to see the amount of disk space used by a new product. `du -s` gives, by default, the number of 1KB blocks used by files or directories:

```
# du -s /usr/sax/*
4203    /usr/sax/alpha
8848    /usr/sax/local
6630    /usr/sax/re1
```

Others will not have the `alpha` directory: this contains the alpha release, and is shown here to indicate that the software has grown significantly to this beta release (dated 18 October 1999). The `local` directory is not really necessary (it only contains the downloaded files and they could be elsewhere) but this is as per Soliton's installation instructions. This means that a minimum of 9MB space is required in the disk partition where you install, with 16MB being realistic following the installation instructions. Bear in mind that you will want to use the documentation, which (following Soliton's instructions) is in `local/sax5.pdf`.

Starting SAX

Having completed the installation, I looked at the README, noting that SAX had been compiled on Red Hat Linux with a kernel version 2.0.36, whereas I run Debian Linux with a 2.2.1 kernel. This caused no problem as the `libc` and `libncurses` libraries were the same version.

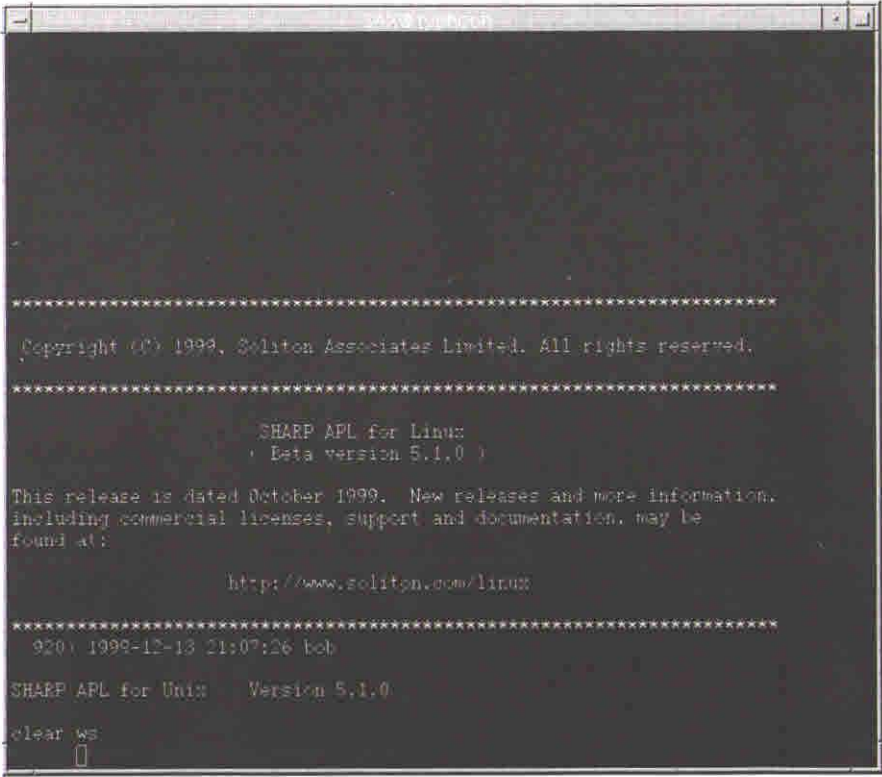
Executing my wrapper script described above brought up an interpreter window almost instantly (see Figure 1). The interesting things to note in the window are

- The copyright statement. This clearly indicates that this is not a final release, but a beta version. It also gives the URL to find updates.
- The fact that the process number is given at startup. This is useful as separate APL processes use this number to identify each other for communication purposes. As a last resort, runaway processes can also be killed using this number. It indicates the UNIX parentage of the product. This is followed by a timestamp and the name of the user owning the APL process. See the line

```
920) 1999012013 21:07:26 bob
```

- Lastly, immediately before the `clear ws`, is another line showing that this is "SHARP APL for Unix". I am curious as to whether this is
 - a hangover from the UNIX-to-Linux port,

- a mistake,
- an indication that Soliton thinks "Linux=UNIX", or
- an indication that lessons learned in this version are to be implemented in future UNIX versions.



```
-----SAX on Linux-----
*****
Copyright (C) 1999, Soliton Associates Limited. All rights reserved.
*****

      SHARP APL for Linux
      ( Beta version 5.1.0 )

This release is dated October 1999. New releases and more information,
including commercial licenses, support and documentation, may be
found at:

      http://www.soliton.com/linux

*****
920 1999-12-13 21:07:26 bob
SHARP APL for Unix  Version 5.1.0
clear ws
_
```

The interpreter interface is implemented in an `xterm` window. This pleased me as it means that SAX should run well across a network, running the interpreter on one machine and the interface controlled by an X server on another. This is my preferred environment: SAX running on my Linux PC with me using it from a high quality monitor and keyboard on my Sun Ultra 5.

SAX should also work fine while run from a `telnet`-type session from a PC, although such an interface may need some fiddling to get the keyboard working correctly. Unfortunately I have not been able to test this.

Running SAX

In earlier communications, Soliton expressed satisfaction with the performance of SAX. I do not want to start comparing execution times between SAX and other APLs, as the comparison is not fair if the operating system is different.

However, I am happy to say that SAX is very fast. At no time did I feel that the system was sluggish, although users should be made aware of the fact that the start-up option `f sync`, if left at its default value of 2, can affect file writes rather drastically. Setting this to 0 switches off file syncing and makes great improvements, if at the cost of some security. This is mentioned in the README file and inadequately described in the documentation, although I did find a better explanation of this in the *Intrinsic Functions Manual* section on `unix.filesync` (page 252). Essentially (as I understand it) setting this to 0 allows full buffering of files, hence the speed improvement, which means that a crash before a buffer flush could result in an unusable file. My confidence in Linux lets me take this risk.

SAX was also quite careful with system resources. I tried to see how many sessions I could open: with 60 simultaneous open SAX sessions, each of 0.5MB, I had about 3MB of my 96MB memory still available and I had not started to use any of my 280MB swap space. The script to start these sessions took about a minute to run on my PC, based on a Cyrix P150+ processor. This indicates the suitability of SAX as a multiuser development platform (on more powerful hardware, of course).

The APL

I found it a little difficult to get into SAX. It has been a few years since I last used Sharp APL under MVS on a mainframe, and I find the differences between SAX and most of the other mainstream APL variants significant. Nevertheless, this is an extremely powerful implementation of the language with much to recommend it.

Of course, anyone can write "old" APL code (users of VS/APL will feel right at home) but modern features are implemented quite differently. Fortunately for me, Dan Baronet's article [11] means I don't have to go into details regarding the differences between this APL and the others.

The SAX "variant" of APL shows its heritage: of all the "standard" APLs it is definitely the most like J. The concepts of *rank*, *frame* and *cell* have much less meaning (if any) in other APLs, whereas in SAX they are an integral part of the language. In certain cases the importance of axes is reversed. Then there are the

differences between boxed (SAX) and nested arrays (most other APLs). See Dan's article [11] for more detailed information in this regard.

One of my favourite language extensions the Sharp APL family has is the extension of the logical *and* and *or* functions to provide *lowest common multiple* and *greatest common denominator* functions for non-boolean integers. This seems so logical that I cannot imagine why other APLs haven't adopted it.

Lastly, the mainframe background also shines through in the continued support for different task types. While these were developed for the mainframe, they fit naturally into the UNIX environment. As an example, starting an *S task* (this is a background APL process controlled by shared variables) using `AP 1` which loads a workspace, executes a trivial latent expression function and captures the result is almost instantaneous. Although workspaces of up to 2GB are supported (mainframe workspaces are typically much smaller) subject to available swap and memory, the technique of packaging is also still there, allowing the developer to create really complex programs while keeping memory usage at sensible levels.

After a while, SAX started to come almost naturally and I began to appreciate its features. One might conclude that the language is fully featured and includes several powerful extensions to the ISO standards. Initially there is some learning to do for those not used to it, but the effort is more than justified.

The Interface

Right now, the interface might be disappointing to those accustomed to the Dyalog APL, APL2000 or APL2 interpreters running under Microsoft Windows. The use of Windows has allowed vendors to provide a very productive development environment, which is difficult to do without once you get used to it. SAX provides nothing like this, in fact it even compares unfavourably to Dyalog APL/X version 6 (now several years old, but could be considered a comparable product) as this made much more use of the windowing services provided by the X server.

However, I'm confident that this will be addressed soon. Soliton is preparing an interface between their APL and Java which will include an integrated development environment. This will be included in a future release. While I have not yet seen any hint of this yet, it must be better than the raw session manager they currently have.

The function editor should also be mentioned. My feelings about full screen editors are subjective and fairly personal, but I must confess that I did not like

Sharp's editor on the mainframe and, sadly, the editor has not been improved in SAX. As far as mainframe editors are concerned, I thought IBM did it better with VS/APL (along with a little help from GDDM I believe), as did STSC's mainframe product, which used the same editor. As far as I'm concerned APL2 was no improvement in this regard. Sharp APL's editor is a large step backward, and this is essentially the editor that comes with SAX. Perhaps the Java IDE mentioned above will provide an improvement.

A last point on this subject is the keyboard definition tables. While these are adjustable, one can hardly imagine a more boring and tedious job. In the long run, I believe Soliton would encourage customer loyalty if they addressed these niggles.

The Future

Soliton is working on further developments. I suspect several companies are keen to port tools to this new environment (I know that Causeway are porting RainPro). I think this product still has a little maturing to do between beta and final release. All in all, I am sure that this will become an important interpreter. It is the first of the serious APLs to be ported to Linux and with its innovative licensing policy, I believe it deserves to succeed, and given its excellent reputation on the mainframe, I'm sure it will.

However, Soliton is unlikely to have the field to itself for long. Dyadic Systems are probably going to announce their Linux product soon [4] and with IBM's support for Linux, I would not be surprised if APL2 was not far behind.

Soliton has intimated that this port is a byproduct of the POSIX port of the Solaris and AIX products. I am keen to get my hands on the Solaris version and compare that to its competitors on an even footing.

Conclusion

Soliton has a real rough diamond here: the language is great once one gets used to it, but the development environment can do with some improvement.

I would recommend that all individuals, especially contractors/consultants, get their personal copy of SAX as soon as possible. I believe that our clients will be using this product in anger before long and we should be ready. Certainly those currently using SAX under Solaris or AIX or SAM on their mainframe should be looking at this product very seriously.

As for corporate buyers, my advice would be to look very seriously at this product. I cannot really say that it is quite ready for development of new systems right now, but I am quite confident that it soon will be. The production version has been promised for first quarter 2000 [3], so we don't have long to wait. When this arrives, we can use relatively cheap hardware and a free but stable operating system to create world beating products!

Notes, References and Web Sites

- [1] Refer to Dyalog APL for Motif Version 8.1 Release 2 for Sun Solaris 5.5, by Bob Hoekstra, Vector Vol. 14 No. 3.
- [2] Read the letter from Bob Hoekstra in *Correspondence*, Vector Vol. 16 No. 1
- [3] Soliton made an announcement of the impending release of SAX/L in *Sustaining Members' News*, Vector Vol. 16 No. 2
- [4] Dyadic Systems indicated an "expectation to announce" a Linux port in early 2000 in *Sustaining Members' News*, Vector Vol. 16 No. 2.
- [5] To get an idea of growing corporate support for Linux, search for "Linux" on IBM's web site (<http://www.ibm.com/>).
- [6] The Soliton Associates web site is at <http://www.soliton.com/> and their Linux web page is <http://www.soliton.com/Linux/>
- [7] Red Hat's web site is <http://www.redhat.com/>
- [8] SuSE's web site is <http://www.suse.de/>
- [9] Debian's web site is <http://www.debian.org/>
- [10] The Linux Emporium provide an excellent Linux service at their web site: <http://www.linuxemporium.co.uk/>
- [11] *Sharp APL vs. APL2 coding style*, by Dan Baronet, Vector Vol. 16 No. 2.

APL2000 User Conference Orlando, November 1999

reported by Adrian Smith (causeway@compuserve.com)

Background

This was the second time I have been fortunate enough to visit the APL2000 users' meeting; the organisation was as slick as ever, the attendance was excellent (comfortably over 100) and the atmosphere was strongly positive. It bothers me a little that the 'official' APL summer gathering seemed mostly interested in Graphpak retrospectives, with very little leading-edge development work to interest the next generation. By contrast, this vendor-run event was remarkably forward-looking - many of the talks were of general technical interest and you felt that this was a young crowd who were doing some seriously exciting stuff with the Internet. What's more they were willing to share ideas which were early in development, but could well be out there as products this time next year.

Venue

Orlando in November is an ideal US target for Europeans as well as Americans. The climate is friendly (cool and clear in the mornings and evenings, high 20s over the middle of the day), and flights are cheap. The hotel was simple and functional, although the room prices were a little steep at \$119 per night for something which was basically a vertical motel with a conference centre strapped on the side.

The layout of the meeting rooms was good - we had three adjacent rooms with a central area for coffee breaks where you could easily wander out of doors and sit around under the palm trees for a chat. The projection equipment was ever-present and worked first time, every time. As usual, almost no presenters had tested their sessions on a real audience, and the hopeful cry of "can you increase the font size please" was heard more often than it should have been. I can't comment on the social programme - on a trip this short I try to stay close to my natural time zone, so my early-to-bed and up before dawn regime cut me out of the LaserQuest outing and the Chinese meal.

Training Day One

I think about 40 people attended the four training sessions - I spent the day with Gary Bergquist learning about ZarkWin, but there were also sessions from Ray Polivka (an Introduction to APL), Rick Butterworth (APL Gui Programming), Patrick Parks (COM objects) and Fred Waid (APL on the Internet). Any of these would have been good value.

Gary Bergquist on ZarkWin

I went to this talk a little puzzled about the claim that I would receive "software worth \$7,500 completely free" if I attended. Was the price a misprint? No, as Gary explained "I put an outrageous price on it because I have no interest in selling it! This was not built to sell, it was built because I would die without it".

Gary's major line of work is in bringing APL+DOS systems over to Windows - this means generating lots of rather simple forms in a short time. He quickly discovered that what took him minutes in DOS could take weeks in Windows, and that "❏WI was way too painful" for this kind of work. Interestingly, he started out by trying to build a toolkit which worked in the same way as his DOS tools for driving ❏WIN - this failed as he realised it would be cheating the user, and if you really wanted to go down this road it would be far better just to make a ❏WIN emulation and leave the system with a DOS look and feel.

The next cut took 9 months, it was much better, and at the end of that time he again threw it away and started over. In the light of the inventors' motto "*It takes three times*" the third cut can evolve and grow because the basic design is right. Gary has kindly agreed to let Vector run his tutorial as a complete article, so rather than repeating what he says, I shall just add a few notes on what I found most interesting about his approach, and leave it at that.

- like Causeway, he keeps the form definition in a nested data structure and has a design function to update it. Like Gary, I find it hard to see how you would do it any other way, and yet there are major production systems out there with massive 'build' functions which are just reams and reams of ❏WI 'New' code with wired-in numbers for the position and size.
- all the 'objects' Gary allows you to add to the base form are defined by custom classes by functions such as `zwButtons` which has all the stuff necessary to make and manipulate a 'button group' object. His designer simply trawls the workspace for things with the right sort of type and name and offers these to you. It is fascinating how PlusWin programmers have learned to live without namespaces! The trick is simply to treat a function as a namespace, using the right argument as the 'function' so you end up with either a huge Select-Case structure or in Gary's examples, lots of lines like:

```
+('New' 'GrpSize' 'GrpNew' 'NewLoc' ∈ Δp) / Δ4, Δ4, Δ17, Δ17
```

What I find interesting here is that this apparently horrible kludge does have some advantages - you can top and tail this function with common code very easily, and it puts all the rats in one trap which moves easily out to file, has a single timestamp and so on. I can see why the PlusWin world is not crying out for namespaces - if you already have a function-based filing system, the last thing you will want to face is the prospect of adding in the extra level of complexity needed to walk the namespace tree to look for everything changed today, and so on. The addition of control structures (and now of bookmarks in the editor) makes finding your way around 400-line functions quite bearable, so the kludge is not quite so kludgy after all.

- Gary's approach to form design is very novel and quite fascinating. He gives you almost no direct control over the positioning of objects, and things like labels determine their own size from the font and text. What you set is the *relationship* between objects, using a very powerful set of alignment and spacing constructs so you can say 'A is centered below B which is to the right of C' and so on. Objects can be put in groups which are part of other groups and so on, so once you get a feel for laying out forms with these rules you can make basic data-entry forms very quickly. What is more, the rules are re-evaluated on resize of the main form (assuming at least one of the objects was sensitive to form size) so you get very effective resize behaviour - way beyond anything you can do in DyalogAPL with the 'Attach' property.

This approach solves two horrid problems, only one of which Gary was aware of! The first one is 'large fonts' - people who run your application on big screens with a big font size. His forms just cope! The second one is translatability - someone translates your application into Finnish and on average the text labels get 50% longer ... he copes magnificently with this one too and he didn't even realise! The final advantage of using a rule-based design strategy is that if you want to push out HTML forms you traditionally do it with tables, putting the various input fields into table cells and using *colspan* to get rough control over width. Of course you can cheerfully nest the tables, so groups of controls typically end up in deeply nested tables - this fits so beautifully with Gary's rules that I could see his designer as an excellent starting point for an HTML forms generator!

Watching the audience reaction to all this, I fear Gary may be crying in the wilderness. I can see the merit of what he is doing, and so will anyone else who is faced with converting 100 forms in a week, or *not getting paid!* Those of his audience who had not yet had to face up to this kind of pressure clearly were itching to get hold of the radio group by the corners and drag it across a few pixels, line those buttons up *just there* and so on. I think he is right - these metaphors are not for mixing but if I can sneak some of his automatic alignment rules into the Causeway designer without treading on its purely visual approach, I will certainly have a go.

Conference Day One

This ran pretty solidly from Eric's welcome at 9:00 to the end of the closing session at 6:15. My only regret was that there was only one of me, so it was a matter of chasing around in the coffee breaks and getting presenters to promise to send us their talks for Vector. I also managed a few previews or postviews on the balcony over a doughnut or lunch.



Eric Lescasse chatting with Bill Rutiser over an outdoor lunch

Jim Sinclair - using APL at Cognos Corporation

For those of you who have been asleep for the last year, after various false starts, Lex2000 and the associated APL business were acquired by Cognos Corporation early this year. Lex2000 has been re-badged and updated as "Cognos Finance" and APL+Win is now in serious use as a development tool elsewhere in the Cognos product set.

Jim Sinclair is a VP in development, so he is both a user of APL and to an extent the link from the APL2000 group into the Cognos management structure. He gave us an interesting history of the company, and of his role in it - he also offered some very useful (and possible controversial) observations on the relative merits of APL against other development environments.

He first met APL at the University of Waterloo, then left it for 25 years to work in system development in a variety of 'conventional' languages. He again became interested in APL as a development tool when Cognos began to explore extending their product range towards the financial consolidation and budgeting areas. He quickly became aware that many of the market leading applications seemed to be written in APL – perhaps this was no accident? He started out with IBM's APL2 (because IBM had to be the best, and he knew no better), gave up on it rather quickly and bought a copy of Dyalog APL instead. Having used this to build some prototypes, he investigated Lex2000 which turned out to be written in this other APL ... ! He attended the APL2000 conference in 1998 and found it hard to swallow some of the things which were being claimed about the language. Never mind – Cognos bought Lex2000 and Jim now mostly works in APL+Win.

Having run through the history (and some very amusing corporate videos), Jim expanded on his personal experiences with APL. Clearly, having an experienced C++ developer (who also knows Dyalog APL) as both user and owner has been a serious kick up the rear-end for the +Win group – you can already see the consequences feeding through in the new code-walker and in a variety of simple and well-overdue improvements in the environment. Here are some of his comments:

- **debugging** – compared with C/C++ debugging in APL is a darn sight easier than in any other language we use. I used to like the Dyalog debugger better, but now I like CodeWalker.
- **implementing math algorithms** – it seems to work right first time!
- **stability** – I've used Microsoft products for years and years and they blow up. This blows up too, but not very often.
- **support** – in all the years of working with Microsoft support, I've never talked to a real person. APL2000 have been very good at getting stuff into the product (for example to make MDI work as it should) that I felt was really necessary.
- **development environment** – OK, but plenty of other stuff competes very well.
- **programmer productivity** – I don't think that APL is any more productive than C++. It depends more on who you have and what you use it for. If you are working in the right sweet-spot for the language, you are very productive.

Naturally the last comment drew a fairly strong audience reaction! I think I agree with him (stand by for mass burning of Vector by APLers the world over) ... I have a bunch of little C programs that I use for things like generating FTP scripts from all the files I touched today. I use an ancient command-line C compiler and simple text editor and I am definitely more productive with these tools than I

would be trying to hack this stuff out in APL. Watching Richard (age 16yrs, APL experience approx 13yrs, Java experience approx 18 months) develop simple multi-user TCP/IP games in APL and in Java, I reckon he is at least as productive in Java as in APL. In particular the Gui development tools (as shipped by any of the APL vendors) are light-years behind Delphi-5, J-Builder or Visual Basic.

Some of Jim's dislikes were:

- **writing Gui code** – atrocious – I don't like the process. I tried some of the screen painters – it was like being back in Windows2 – give me C++ or VB! The resulting Gui application is not as good as applications built in other Guis – e.g. in +Win 3.0 you couldn't build an MDI form that behaved in the right way.
- **data modelling** – used to having this in the environment and having it build the code for you.
- **performance** – you can't get APL to sing like C++.
- **memory management** – in C++ 90% of your bugs are in here. In APL you have no control over it (but no bugs).

Top of his list of 'want to see' was "a sans serif APL font that actually looks good". This is a tough one! *APL2741* has probably run its course, but getting a fixed-pitch font to work well both on screen and paper is not easy. Adam Kertesz showed a rather nice one called Andale Mono in his talk later that morning, but of course it had no APL symbols in it. I will keep playing with APL Sans until it looks right – comments welcome on the readability of the code in *Hackers' Corner* in Vector 16.2 which uses it.

Breakout Sessions

I was lucky to get my own talk out of the way early, which also gave people time to catch me later on in the conference. However I missed out on two papers on porting mainframe applications, which was a great pity as both papers were well-written with lots of pragmatic advice.

We then had a very nice talk from Mark Osborne on how floating-point calculations can produce surprising results when you push *PPP* to the limits. I particularly liked his 'measuring-rod' analogy – say you are trying to report the length of a sheet of paper and you have a set of 5 rods in the ratio 1 : 1/2 : 1/4 : 1/8 : 1/16 : 1/32. The snag is that you are only allowed to use a consecutive set of 3 rods, so you report your length as (index into the set),(3-element boolean) for example 2,1 0 1 would be 5/8. Clearly you can do addition and multiplication in such a system of measurement, and clearly you will rapidly generate inaccuracies.

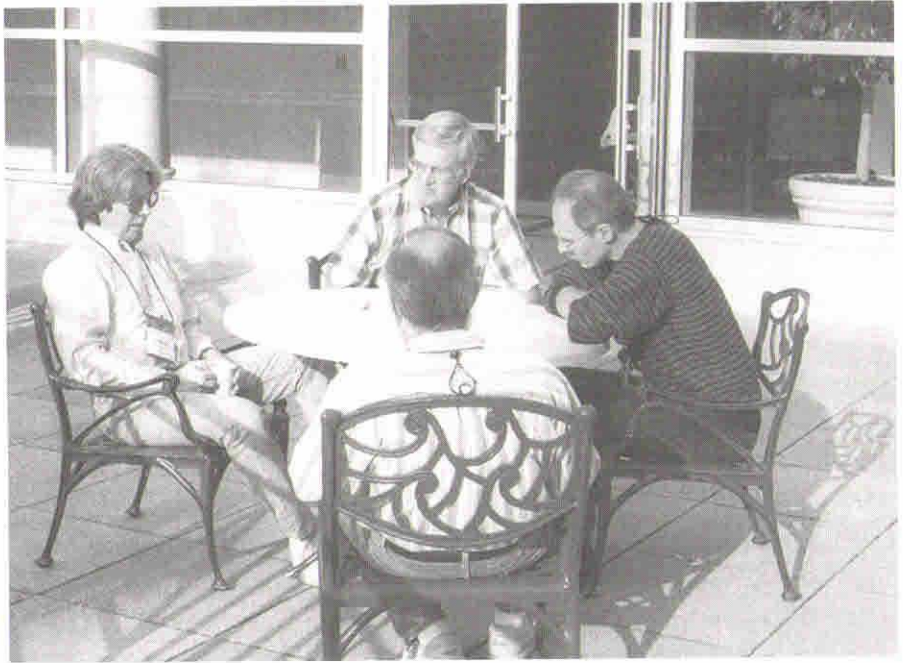
Now imagine that you are allowed to pick 53 contiguous rods from a set of (say) 1024 and you have all the same problems, just pushed a little further to the right of your boolean vector. Well, this is just what the IEEE floating-point specification (as implemented by the Intel FPU) is doing, so now you can see why things don't always come out as you expect.

After lunch I had a hard choice between **John Walker** on getting CD album titles over the internet, and **Bill Rutiser** on the inner secrets of Unicode. I hope John will let us run his paper in Vector (along with all the code) as it looks like a really good practical education both in multimedia and in basic Internet techniques. Bill's talk attempted to traverse a hugely tricky subject and unfortunately he ran out of time before he ran out of material. I spent nearly 2 hours with him on the final day, and left Orlando with a great deal of precious knowledge (and the full table of where all the APL symbols are in the Unicode bible) which I could not have acquired any other way. It would be good to see this material written up one day, but I have to admit that fonts are something of a minority interest.



Finally, I went to see **Fred Waid** who is responsible for the www.openhere.com portal, which is driven entirely by APL+Win. I left feeling just a touch uneasy about the technical tricks needed to get NT to kick off APL as a process, and the way this was backed on to Microsoft IIS – but very convinced by Fred's argument that when you want to write 'neat stuff' on the client side APL is a superb environment to build in. This is particularly true for applications like spiders – increasingly these are being written for clients who are using them to watch competitor's sites or to get the best moment to bid in on-line auctions. The competitors soon get wind of this and start to take defensive measures, maybe simply changing the format of the site to confuse the spider. With APL you stand a

chance of rewriting the spider nearly as fast as they reformat the site! I would like to be equally convinced about the server-side, but I have yet to see the 'killer app' that APL excels in. OK, you can write a great chat-room in APL, but is it really that much more productive than C++ here? Time will tell.



Chris Lee (SoftMed, left) and friends between sessions

Conference Day Two

One of the really nice touches about APL2000 conferences is that they give you breakfast! This gets everyone assembled early and is yet another opportunity to chat to APLers in pleasant surroundings. Eric's introduction to the day was followed by Don Segal of SoftMed Systems, and then another run of parallel sessions.

Don Segal - Building a Company on APL

Don is President and CEO of the biggest and most successful medical-records company in America. He is in the rather unique position of having originally attacked the problem as a user, and gradually evolved from a set of solutions which fulfilled his needs to the much more generic and scalable solution which makes a successful product. His recipe "understand what the user wants better than the user" was essential, as his users are typically cogs in a vast machine and have little control over the system as a whole. Rival products were based on

asking the users what they needed – they invariably creaked at the second site and failed soon after under the weight of ad hoc patches. “We saw a lot of competitors having to keep customising from client to client – when you reach 50-60 clients you are in quicksand.” Don also observed “The software business is not as easy as a lot of people seem to think – the product must be designed to give leverage and be generic across scales”.

He actually started on the IBM 5110 “It switched to APL first, so I used that. I couldn’t spell, so I never used BASIC – a good business decision I think!”. This marvellous old machine ran a production system for 5 years on the 5" monochrome screen. From here he made steady progress in understanding the problem and developing generic solutions which gradually gained acceptance in the medical records business. It was interesting to hear his views on the dangers of going to market with a badly designed solution “When you sign up clients your problems begin – if your design is weak the support can eat you alive”. It was also interesting to get a slightly different perspective on the productivity balance between 15 APLers and 9 C++ developers in his group: “On balance we get a factor of 3 between the groups. We have one product in FoxPro – with lots of clients – that product is *terrible* – it still cannot do what we wanted it to do. It will be replaced with APL and SQL even though 150 clients will need to be moved to the new platform as a result.”

Breakout Session - morning

Using a Resource DLL Compiler – Brian Chizever

Fortunately for me, Brian re-ran his earlier session on using a resource DLL compiler to make a simple-to-ship collection of icons, bitmaps and sounds for use in a runtime +Win 3.6 application. This was one of those sessions which was just plain useful, and came with a nice freeware resource compiler and linker to make it easy to use in the comfort of our own portables. The following notes are more or less as I took them down during the session, with minor corrections to bring them into line with his PowerPoint slides.



Brian sharing the front desk with Sonia Beekman

I think Brian had the easiest selling job of anyone at the conference – external resources are a major pain with any Windows system. You can quickly accrete a huge collection of icons, little bitmaps, cursors and maybe even special sounds. These make it very hard to deliver your application without key elements going missing, or just reverting to the ‘placeholder’ set that you drew in a hurry 6 months ago. On balance I feel it is always better to have Windows construct these for you, rather than taking the DyalogAPL option of storing them in the workspace as numeric arrays (for bitmaps and icons) and creating the objects directly from these. By having Windows manage them you save on workspace (the memory is allocated externally), and you will have fewer problems with colour maps, changes to the object format (e.g. animated cursors) and so on.

What Brian suggested was ‘putting all the rats in one trap’ by compiling all your resources into a single DLL and using this as the source for the APL Gui. This technique also prevents the end-user fiddling with your icons and generally changing the style of the application in unexpected ways. He started by illustrating a selection of ways for setting the ‘icon’ property of your main form:

```
ff' [WI 'New' 'Form'
ff' [WI 'icon' '..\icons\basket'
```

This sets ‘basket.ico’ as the small icon in the top left of the form, and it also appears in the ‘Alt-tab’ selection panel and on the taskbar. A more interesting choice might be any of ...

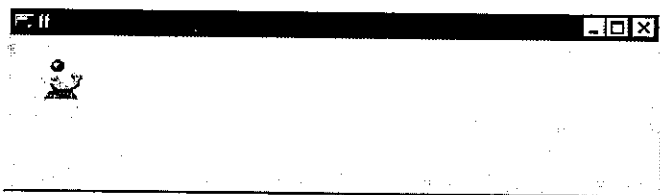
```
ff' [WI 'icon' 'notepad.exe'
ff' [WI 'icon' '>ICO 2 notepad.exe'
```

The top line simply picks up the default icon from the notepad application - it does not need an explicit directory as Notepad resides in the Windows directory which is always on the search path. The second line is more explicit - it requests the second icon resource from the same .exe file. If you check out your start-menu shortcut to Notepad you will indeed find that it is hiding two icons in there! Another example might be:

```
'ff' [WI 'icon' 'calc.exe'
```

You can also set the 'pointer' of the form in a very similar way, for example to have an extremely silly seal bouncing a ball on his nose:

```
'ff' [WI 'pointer' '..\cursors\seal.ani'
```



Sounds are a little more complex, as you first have to load the DLL into memory and get its handle:

```
[+L+[WCALL 'LoadLibrary' 'conf.dll'
27787264
[WCALL 'PlaySound' 'cat' L 'snd_async snd_resource'
1
[WCALL 'FreeLibrary' L
```

... or you can do it the easy way from a sound file if you prefer (1=async):

```
[WCALL 'PlaySound' '..\sound\curlew.wav' 0 1
```

N.B. Don't try this one in the airport lounge on the way home!!

Having whetted our appetites with a rather nice spinning globe and a barking dog, Brian continued by taking us through the process of making our own resource DLL complete with icons, cursor, sounds and so on. By the end, anyone with a portable on their lap had a working CONF.DLL with an assortment of new odds and ends included - this makes for a very satisfying workshop as you can be pretty confident that it will all still work when you get home.

Basically this is a 3-stage process:

1. Combine your assorted icons and bitmaps into a single resource file. This is done by writing a script (typically called a .RC file) and running it through a resource compiler to make a .RES binary file. A freeware compiler was included on the conference diskette.

xx.RC + (xx.ICO xx.CUR ... etc) =====> xx.RES

2. Generate a stub of compiled object code (probably a tiny C program that does nothing) – Brian kindly provided a suitable one called CONF.OBJ

xx.C =====> xx.OBJ

3. Run a linker to strap the output from stage-1 on the back of stage-2 to make a functional DLL. Again, a freeware linker was included.

xx.OBJ + xx.RES =====> xx.DLL

The entire process can easily be automated with a simple batch file. Here are the first few lines of Brian's CONF.RC file ...

```
//
// Icons
//

CD          ICON      "pics\\cd.ico"
MEDIA       ICON      "pics\\media.ico"
```

Comments have the usual C++ style and blank lines are ignored. The 'live' entries give either a number or a name to the resource, the resource type and the path to the actual resource file – note the doubled backslashes here.

```
//
// Sound Files
// a bug in ALINK.EXE 1.6 drops the first letter of any
// resource with a resource type "WAVE"
//

xCAT         WAVE      "pics\\cat.wav"
xDOG         WAVE      "pics\\dog.wav"
```

Brian even documented the bugs in the supplied freeware compiler! The final section of the script sets up the information which Windows will show in the property sheet for your DLL:

```

1 VERSIONINFO
  FILEVERSION 3,6,0,0           // your file version (returned by DLL)
  PRODUCTVERSION 3,6,0,0       // your product version
  FILEFLAGSMASK 0x3fL          // which FILEFLAGS are valid
  FILEFLAGS 0x0L               // 0=normal, else debug, private, etc
  FILEOS 0x40004L              // VOS_NT_WINDOWS32
  FILETYPE 0x2L                // DLL
  FILESUBTYPE 0x0L             // no sub-type
BEGIN
  BLOCK "StringFileInfo"
  BEGIN
    BLOCK "040904b0"
    BEGIN
      // these entries appear at the top of the property sheet from explorer
      VALUE "FileVersion",      "3.6.b\0"
      VALUE "FileDescription",  "APL+Win Resource File\0"
      VALUE "LegalCopyright",   "Copyright © 1999 APL2000 Inc.\0"
      // All remaining entries appear alphabetically in the listbox at the bottom
      // of the property sheet. Add as many as you need.
      VALUE "CompanyName",      "APL2000 Inc.\0"
      VALUE "OriginalFilename",  "resource.dll\0"
      VALUE "ProductName",       "APL+Win\r\nResources\0"
      VALUE "ProductVersion",    "3.6\0"
      VALUE "Reason",            "1999 APL2000 User Conference\0"
    END
  END
  BLOCK "VarFileInfo"
  BEGIN
    VALUE "Translation", 0x409, 1200
  END
END

```

He recommended leaving the upper entries alone (except for your product version) and adding any useful documentation to the lower set. Note that all the strings are 'null terminated' in good C fashion and you may use `\r\n` to insert newlines here. To run this, you need a good ol' DOS prompt (yes, it does still exist):

```

D:\ARCHIVE\APL2000\resource>gorc -r conf.rc
GoRC.Exe Version 0.53 - Copyright Jeremy Gordon 1998/9
Output file: conf.res

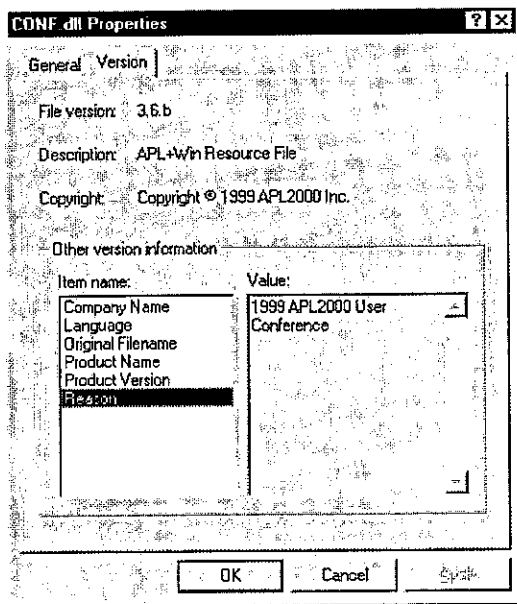
```


This generates as output a file called `conf.res`, which can then go through into the linker:

```
D:\ARCHIVE\APL2000\resource>alink -oPE -dll conf.obj conf.res
ALINK v1.6 (C) Copyright 1998-9 Anthony A.J. Williams.
All Rights Reserved

Loading file conf.obj
Loading file conf.res
Loading Win32 Resource File
matched Externs
matched ComDefs
Generating PE file conf.dll
Warning, no entry point specified
```

The final warning is fine – this DLL doesn't do anything, and the linker thought it ought to tell us! A quick check in Explorer proves Brian wasn't lying!



That really is all there is to it. As a final check we can copy the updated DLL over to our working directory and use one of those icons on the main form:

```
'ff' [WI 'icon'
'>ICO CD conf.dll'
```

Note the use of a name, rather than a number here – this makes things fairly self-documenting.

Brian did point out one annoying difference between Windows95 and WindowsNT – NT orders icons as they appear in the .RC file, whereas 95 orders them alphabetically. Simple enough to work around ... just be sure to arrange them alphabetically in your resource

script and then they will both behave the same. As I said at the start, it all just works, even back in the cold, dark UK. Many thanks for a most useful 40 minutes.

Breakout Sessions - afternoon

After lunch it was a hard choice between **Carl House** on a toolbox for generating a fully cross-indexed website, **Warren Baelen** on ADO and **Patrick Parks** on interfacing to commercial code-management systems such as Visual SourceSafe. All these were well-covered in the proceedings, so in the end I went to find out about ADO, how it is taking over the role of ODBC and how to drive it from APL. Again, this was a helpful talk with all acronyms well explained, and again I hope we can cover it in detail in a future Vector - there was far too much detail to include here.



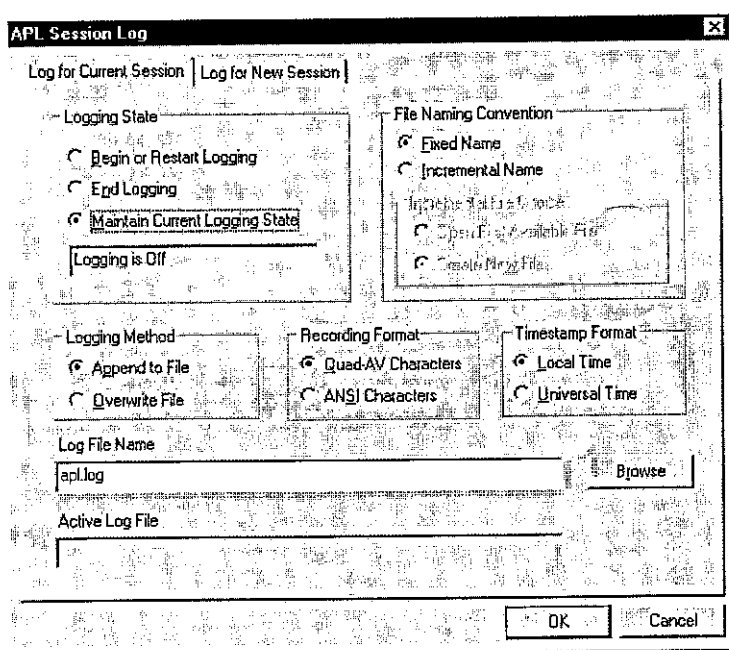
Carl House in a hurry!



Patrick Parks

The next session was an interesting case-history, related by Colyn Phillips – the Gui dialogue in the +Win options screen which controls the session log files. This was partly about Gui design, but looking at this from the outside I felt it was more about trying to put a brave face on a piece of over-designed functionality. It is trying to cope with the problem of two concurrent APL sessions both logging at the same time. Dyalog simply duck (sic) the problem – last one out saves the log! More realistically you name the logfile in your shortcut so you have one file associated with each workspace.

In +Win you have a wonderfully confusing set of buttons to press, spread over two almost identical tabs ...



One day I may find out how to work it – then again I never did work out how to operate the old debugger, always resorting to the standard APL hack of inserting lines of dominos to make things stop where I wanted them to. Hopefully the same command-line approach will work here too so I can be selective about which workspaces I log and where.

The last session I attended was from Chris Lee of SoftMed which was mostly about code-management, utilities and development philosophy. Again, I was

struck by the fact that introducing namespaces to these systems could cause more problems than it would solve. The day closed with a talk by Luis Caro on APL in the Columbian banking system. I am sorry to report that by now I was too brain-dead to attend, so if anyone took good notes here, it would complete the picture nicely to have them.

Training Day Two

The conference closed with a final tutorials day - I think around 30 people were still around to the end, although some of them were probably just kicking about waiting for flights home. I grabbed Brian Chizever's notes on `WCALL` which have an excellent coverage of the common Windows calls you might need from APL. I also had an excellent impromptu tutorial from Bill Rutiser on a Russian-built package called FontLab which actually seems to understand the TrueType font format, unlike Fontographer which just presents page after page of meaningless options with no help whatsoever on what they do. I am now a little more hopeful of being able to make an `APL2741` font that works correctly in the majority of modern web browsers, as well as in Word documents and APL sessions!

Other sessions on offer were from Rick Butterworth (using the complex classes like ListViews, and running APL+Win as an ActiveX object); Eric Baelen on driving Excel from an APL+Win application, and Eric Lescasse on his APL+Win 3.5 object collection. This is a library of over 70 custom classes which have been built using the new user-defined objects capability.

Wrap-up

If you can take time out round about the 3rd week of November next year, this looks like a very worthwhile trip. Even if you mostly develop in some other flavour of APL, you can benefit from most of the ideas and it really helps to have some of the general enthusiasm rub off. As Eric said in his opening address, when you go back home and start working, it often feels like you are the only one left. Getting together with a bunch of like-minded APLers is an essential part of keeping the flame alive!

A Conference for Winners

reported by Jim Lucas (jel@danbbs.dk)

The *APL2000 Users' Conference* (Orlando, Nov. 1999) had "Winner" written all over it. From the quality jacket (I wear mine everywhere, now), *full proceedings*, and significant software included with the registration handouts to the superb lunches (also included) to the layout of the presentation rooms around a central mini-lobby, which encouraged intercommunication as well as providing easy access to all activities. Everything was well planned, well organized, and well managed.



The Pool, from the Balcony outside the Conference Area

The tone of the conference was also a winning one. No moaning about the demise of APL, in either past or future tense. No bombastic rhetoric about the inherent superiority of APL, either. Everybody was too busy saying, "Hey, look what I've done. Let me show you how I did it." With enough detail that you could reproduce it directly, not just an outline that required you to figure out all the important details yourself. And the speakers were *SPEAKERS*, apparently chosen for their skill in live presentation, not just for the topics or content they were presenting.

Who were the attendees? Plenty of experienced — and successful — “old timers”, but also a refreshing scatter of not only new, but also young faces. The advance attendee list had 97 names representing 13 states and 11 countries; 59 different companies were represented. I believe there were enough late registrants to push attendance comfortably over 100. There also seemed to be a good mix between independent entrepreneurs and corporate programmers.

I attended all four days, the two Conference Days, plus the two optional Training Days, one before and one after the Conference Days. Adrian Smith was also there, as both speaker and attendee, so please read his much more detailed review in this same issue. I won't try to duplicate his technical reports; he took much better notes, and skipped the evening entertainment and bull sessions to write them up each night. I'll just try to say what it felt like to be a participant.

The Program

The format for the Training Days was simple, a three-hour session in the morning and another in the afternoon. The only difficulty was in choosing which of four concurrent streams to attend. These were intensive sessions, with three of the streams each day being actually six-hour classes interrupted by lunch, and only one stream each day having a different topic in the afternoon than in the morning.

Conference Days were crowded with shorter presentations, but peppered with fifteen-minute breaks for mingling, discussion, and refreshment. In all there were seven non-technical plenary sessions, ranging in length from fifteen minutes to an hour and a quarter. Each day there were four hour-long technical presentations in each of three parallel streams. Topics ranged from the effects of APL's internal representations on precision and efficiency, to the many ways to get what you need out of the morass of Windows® internals, to designing and managing web sites. I don't think there was a single time slot in which I didn't have to make a difficult choice.

I won't take space here to list all the session topics. As I write this, the information is still available at <<http://www.apl2000.com/ejblet12.htm>> and its subsidiary links on the APL2000 web site.

The Materials

I already mentioned the “proceedings”. It's actually a bit more than that, a binder containing the Agenda (a copy of each day's schedule, including brief description of each session), the (advance) Attendee List, the papers for each of the Breakout Sessions, and papers for two of the Training Sessions, as well. The loose-leaf

binder is a bit bulky, but that format made it easy to insert in their proper places late-arriving pages, specifically two papers, two addenda to papers already present, and a corrected copy of one paper where the original had printed APL code using the wrong font.

In the same binder — in a plastic page with snug pockets for the purpose — were five diskettes, all containing self-extracting compressed files: Disks 1 & 2 of the *APL+Win 3.5.03 Patch* (3.5 is the current version), the *APL+Win 3.6 Beta* release, and *Demos and Utilities and Help Files* to accompany several of the presentations and make it simple to reproduce the details after leaving the conference. These included ten workspaces, six help files, two databases, and various other files, from bitmaps to executables.

Software handouts were also provided to attendees of three of the Training Sessions, so I also got the ZarkWin software. Attendance was higher than Gary Bergquist had anticipated, and he hadn't brought enough diskettes for everyone, but that was no problem, as several of us copied the contents of the diskette onto our notebook computers and passed it down the line.

The Presentations

On the first Training Day I attended Gary Bergquist's ZarkWin class. Adrian has described this in great detail, with a deep perspective coming from his own experiences in developing Causeway. There is no point in my trying to add to this. He has also reported well on the plenary presentations, and again I won't try to add to what he has said.

Conference Day 1

These are the Breakout Sessions I attended on the first Conference Day:

Eric Baelen - Using APL+Win 3.6

This was an overview of what's new in the 3.6 Beta. Mainly powerful new debugging tools and enhancements the Windows interface. Nothing new on the "raw APL" front this time around.

Brian Chizever - Resource DLL Compiler

This was a powerful and detailed demonstration of what Windows resources are, how to view them, use them, and even create your own. The explanations and examples were both detailed and clear.

John Walker - APL + CD

Everything you need to implement your own CD player in APL, *including* accessing the CD's descriptive details from *CDDb*, a database of CD information on the internet.

Fred Waid - Open Here

I attended Fred's workshop at the *APL99* conference and felt I learned a lot. But for someone new to web-oriented design and programming it was just too much to digest in one go, so I attended this session in hopes of reinforcing some of what I had already heard. The shorter presentation was less technical; still, it filled in some gaps in my understanding. What does seem clear is that programming a web site or web tools in APL is no more difficult than programming anything else in APL. Think about that. And his demonstration shows that by various measures APL is a viable foundation for web-oriented programming.

Conference Day 2

These are the Breakout Sessions I attended on the second Conference Day:

Jeremy Main - APL+Win and MS Access®

Not a "how to" workshop on how to write detailed code, but a detailed view of the progress in Microsoft® database technology and the various current alternative paths for communication between APL and Access, with recommendations.

Carl House - Fast Track to a Good Web Site

You want to create a web site fast? A *good* web site? Then don't try to reinvent the wheel. (Now where have I heard that expression before?) Search out sites you think are well-designed and use them as templates. Then experiment a bit to see which do the best job of presenting your personal content in an informative and usable format. Carl showed us how he did this, explaining and illustrating his choices. He also talked about related issues of website maintenance.

Eric Baelen - An Excel® Front End for APL+Win Applications

How to use an Excel Workbook as a front end to an APL+Win application. With enough detail for you to go home and do it again yourself, then proceed to doing it with your own workbooks and applications.

Tom Atkins - APL and Powerpoint®

Frankly, I attended this session to learn about Powerpoint. I learned a lot both about Powerpoint's capabilities and about how to construct and control Powerpoint presentations from APL+Win.

Training Day 2

On the second Training Day I attended only the first half of what was an all-day class in order to attend a different stand-alone presentation in the afternoon.

Brian Chizever - WCALL (part 1)

Although I skipped part 2 of this class to attend the next one, I feel I learned a great deal about the Windows API in all its variety and how to use it from APL, including using it to retrieve information on how to use it. Invaluable.

Eric Baelen - Excel

Complementary to using Excel as a front end for APL applications is using APL to construct, populate, manipulate, and interact with Excel Workbooks and Worksheets. I.e., instead of making APL available as a tool to Excel, use Excel as a user interface for APL. This was a step-by-step introduction including all the important steps, with advice on pitfalls and quirks, and style recommendations to make maintenance easier.

Presentations I Missed

If I attended eight different Breakout Sessions, then I missed fifteen. (Brian Chizever's *Resource DLL Compiler* presentation was offered twice.) Here's where the contents of the binder were really valuable. First of all, browsing the papers helped me with last-minute choices of which sessions to attend. But I could also learn a significant amount from reading the papers alone. And doing so gave me a basis for private discussion with both those who *had* attended these talks and the presenters themselves.

Adrian Smith's *Introducing VML...* piqued my interest and gave me enough to start me working. This is clearly something I'll be looking at more closely in the future.

Since I live in Denmark and have contacts in other countries (Poland, Finland, Russia, Japan,...) multi-language support is of great interest to me. Though I missed Bill Rutiser's talk, I learned a great deal by reading his paper and talking

with him in the breaks. That I don't feel Unicode has solved my problems is a separate issue.

The papers on floating-point precision and nested-array internals were comforting in confirming what I thought I already knew. Rick Butterworth's *Managing Long Running Calculations*, Patrick Parks' *Managing APL Code*, and Colyn Phillips' *Creating a User Interface...* all address issues that I have solved in my own ways. It's always useful to compare approaches.

Though I'm not currently involved in such work, it was interesting to compare the two papers on migrating applications to APL+Win from older environments. And instructive to note both the similarities and differences in their stated approaches. I was also intrigued by the contrast between Walter Fil's (*Taking the Migraine Out of Migration*) and Gary Bergquist's (the ZarkWin Class) philosophies of the best way to upgrade from DOS-based to Windows-based user interfaces.

Beyond the Presentations

Everything at the conference was arranged to encourage people to interact with each other, sharing ideas, helping each other, and just getting acquainted. All the presentation rooms opened onto a central mini-lobby, where could be found both refreshments and the central desk — for registration, information, and assistance of all sorts. Across the hall — if that's the right word for a glass-sided walkway large enough to accommodate a transport truck — was a balcony terrace overlooking the swimming pool, with tables and chairs as for a café. People could easily congregate in groups of various sizes, but all in close proximity, so shifting among groups was also easy... and so was reaching out for something to eat or drink without abandoning one's conversation.



Lunch every day was provided by the conference, and was not only ample, but excellent. The only thing comparable I've experienced at an APL conference was APL99 in Rome. One day we ate in the spacious dining room on the floor below the presentation rooms, but the other days lunch was on the terrace beside the swimming pool. Some people did look for shady tables, but I don't think anyone actually complained about the sun.

The banquet was included in the conference fee. No point in trying to find cheaper alternatives. The room was spacious, with plenty of room between the tables. The food was overwhelming, with myriad choices representing several cuisines, from pasta to carvery to sushi. I think even the vegetarians had to make choices. Unnecessary luxury? I had the distinct impression that the luxurious atmosphere had a functional effect. Once again, the atmosphere encouraged people to relax, and to share on both personal and professional levels. No one seemed to feel that stiff formality was required. There were a few speeches and presentations, but they were brief. Clearly, we were intended to pay attention to each other, and we did.

More Beyond

The one area where one might have wished for improvement was the rooms. \$119 per night as the *special conference rate* for what more than one person described as a vertical motel seemed a bit steep. I do think it was a big plus to have all the main events in the same hotel where most of the participants were staying. Unfortunately, not all attendees managed to get rooms at that hotel. Fortunately, the area is a hotel truck farm, so I think everyone managed to find rooms within walking distance. I think I was the most distant, a mile away. After finding no room at the inn, I managed to book a room at a real motel, at half the price. And that mile walk was all past a single building... Orlando's huge, official conference centre. It was a pleasant walk, with plenty of grass, palms, ponds, and even a couple of live egrets.



On the social-activities front, we're once more in positive territory. For when people were on their own, there were numerous restaurants and shopping and entertainment opportunities a short cab ride or even a walk away. (Not the Disney World theme parks. They're much to big and expensive to consider visiting for just an evening.) I don't think many of the conference goers went to the hotel's disco (*too* loud) or restaurant, but the bar in the lobby saw several groups, and discussions both quiet and animated.

Then there was Wonderworks! This was another official conference activity, free to all participants. Like everything else, it was conveniently located, just down the street from

the hotel. The building looked to me like "classic courthouse", pillars and all,... but upside down! An architect's statement that what you'll find inside is entertainment. I'm not sure what's on the other floors, though I think some of it may be exploratorium-like. We all got whisked to the top floor, which was ours alone for the night. It was a games arcade, and everything was free! Video games of all sorts... Shoot the bad guys; pick your flavour. Race your car, skis, kayak, etc. Boxing and karate. But also shooting basketballs, kicking soccer balls, air hockey, and the ultimate... laser tag! There was a cash bar, and we were provided with an incredible free dessert buffet, more than we could possibly consume.



The Dessert Buffet!

The one officially optional activity with a separate price tag was a group outing to a ritzy Chinese restaurant. Personally, I thought the price was a bit high, the tastes were a bit on the touristy side, and I know where to get better for less in New York City. But it wasn't New York; the food *was* excellent; and once again I thought the shared feeling among the participants was such that I don't begrudge the price I paid to be among them.

In Closing

This conference was instructional. Nothing was theoretical; it consisted entirely of demonstrations of how to use APL to get things done, including and especially using APL as a consistent and powerful tool for accessing and manipulating resources outside of APL. Too pricey? Not at all! Even without the excellent food and refreshments you would be hard pressed to find training of this quality on any subject at a comparable price, much less the broad range of subject matter

covered here. And while nothing explicit was said about personal networking, all the details provided an atmosphere that encouraged it. This conference was valuable, and I plan to return next year.

In one evening discussion Roy Sykes challenged us to come up with solutions to certain problems in APL, using very few characters. I won't steal his best one, but I will leave you with another nice one to puzzle out (those who haven't seen it before): if V is a vector of numbers, all positive, give a four character expression which returns the average of V . Have fun.



And finally, I'm including some photos I took at the conference. I won't bother with captions or explanations, except for two: Those of you who saw Eric Baelen present his *APL Man* computer game (written entirely in APL+Win) at APL99 may recognize what appears to be the original character on which the game was based, namely Eric himself, playing laser tag. The other is a



picture of some characters who were seen at the Hotel Peabody, across the street from our conference site. Contrary to vicious rumour, they are *not* spies for Dyadic Systems, but permanent residents, who have lived at the Peabody for generations and have never set foot in England.

I recommend this conference to anyone who works in APL. Even if your dialect is not APL+Win, you will probably learn a lot which is useful. Consider attending next year.

GENERAL ARTICLES

This section of Vector is intended for general readers who have a working knowledge of APL or J. Authors are encouraged to submit articles which illustrate effective APL applications.

SHARP APL Operators

by Dan Baronet (danb@dsuper.net)

Introduction

Following Soliton's announcement to unleash their APL into the Linux community I decided to review the differences between SHARP APL and APL2 style APLs (IBM's APL2, Dyalog, APL+Win). I have divided the report into several articles, each with a different topic.

This article is the second of the series. In the first one I showed the differences between the languages and enclosed arrays handling. This one is specific to the operators in SHARP APL.

Not having a current version of IBM's APL2 and APL+Win on hand I have been making my observations based on Dyalog APL using `⎕ml=2`.

When reading this text keep in mind the date it was written (September 1999) as languages change and may render this information incorrect in the future.

Conventions

In the following text *SAPL* refers to Sharp APL, including SAM which is SAPL under MVS and SAX which is SAPL under Unix. SAX is the one also running under Linux and compared to APL2 here.

Variables and workspace names are enclosed in 'quotes'. Functions and files are enclosed in <angle brackets>.

Italicized words usually precede their definition.

The term *atom* refers to a numeric or character scalar only. The term *item* refers to any scalar.

Whenever possible I will show examples in SAX and APL2 side by side.

To better understand this article you should first read the preceding article titled "SHARP APL vs. APL2 coding style" (see Vector 16:2 p.87).

Because I will use many examples involving monadic and dyadic functions I will use <mx> to denote a function used in a monadic way (where 'x' may be a

number as in <m1>, <m2>, etc.) and <dx> to denote a dyadic function as in $m_3 \times d_2 y$.

I use the terms 'list' and 'vector' in the same way. Same with the terms 'table' and 'matrix'.

A Bit of History

All of the composition operators described here were introduced in the late 70s. Unfortunately they were never used to their full potential because of implementation restrictions in SAM. With the arrival of SAX those barriers were removed.

An important distinction between SAPL and APL2 is that is it not possible to create your own operators in SAPL.

Types of Operators

There are several kinds of operators. They all use a mixture of data and functions as arguments.

The main category discussed here covers the composition operators using functions only as arguments. Before I discuss these, I will introduce another operator which I will use in examples.

With with data arguments

There is a way to *glue* constants to a dyadic function to change it into a monadic function. For example, consider the function "catenate 1 2 3 before each item":

SAX	APL2
1 2 3'', ''>x	1 2 3°, ''x

Here '' is first used with a numeric (data=1 2 3) left argument to create such a function. Same with ° for *Dyalog*. An equivalent statement would have been

(<1 2 3), ''>x	(<1 2 3), ''x
----------------	---------------

It is also possible to use the data to the right, as in "divide by 3 and 5":

÷''3 5''>x	÷°3 5''x
------------	----------

for which an equivalent statement could be

(<3 5)÷<''>x	(<3 5)÷÷''''x
--------------	---------------

or, without *swap* (the $\text{SAX} \leftarrow$ operator),

$$x \div '' < 3 \ 5$$

$$x \div '' \leftarrow 3 \ 5$$

When a data argument is used with '' (or $\circ$) the entire data list is used. Notice what happens next:

$$4 = \rho \div '' 3 \ 6'' > 14$$

$$4 = \rho \div \circ 3 \ 6'' 14$$

SAX applied the function "divide by the vector 3 6" and returned 4 results (of 2 elements each). *Dyalog* applied the same function to *each* one of 14 yielding 4 enclosures of 2 results each also. If we remove the ''> in SAX the 4 results are reassembled into a 4 by 2 table (matrix):

$$4 \ 2 = \rho \div '' 3 \ 6 \ 14$$

Notice how the '3 6' is part of the $\div ''$ and the interpreter does not attempt to do $3 \ 6 \ 14$ instead. SAX also "knows" that *divide* is done on scalars whereas in *Dyalog* the '' has to be specified in order for the function to be applied on *each* item.

Swap, ($\leftarrow$) monadic use

Swap, is used to swap arguments to a function. Its monadic use implies the argument is the same to the left. For example, if $b \perp b$ returns the number of contiguous 1s to the right of boolean 'b' then so does $\perp \leftarrow b$. To remove trailing spaces in a series of enclosed words we can try

$$(- \perp \leftarrow \circ > ' ' = '' > w) \downarrow '' > w$$

Composition operators

There are 3 composition operators in SAPL: *with*¹ (''), *on* (ö) and *upon* (ö). Their syntax is always $x \text{ f'' } g \ y$ and they are ambivalent. As described in the preceding article, *with* differs from *on* only in applying the *inverse* of *g* after *f* and *g* and requires knowing the inverse of *g*. Ex:

$$\rho '' > x$$

$$\rho '' x$$

¹ In SAX the term used is *under* but I will stick with *with* here

Here, for each cell, we first *disclose* the cell, apply *shape* then apply the inverse of *disclose*, namely *enclose*, to the result. In APL2 the *disclose/enclose* is implicit and never specified. Nor can any other function be used with the `''` operator.

To get the shapes of all elements together in a simple array in the preceding example one must either *disclose* the final result:

$$>\rho''>x \qquad \qquad \qquad \supset\rho''x$$

or, in SAPL, use *on* instead:

$$\rho\ddot{\circ}>x$$

Those 2 operators (*with* and *on*) assume *g* is monadic and *f* is ambivalent. Like in APL2 where *f* is ambivalent.

The individual results are assembled the same way, whether they are disclosed together using `>f''>` or as a result of doing `f2>`.

The ability to specify a function different from *disclose* in SAPL leads to interesting cases. For ex^2 :

$$\overline{\tau}''\phi x \qquad \qquad \qquad (\overline{\tau}^2+1 \ 1, \rho x)\rho x$$

Another is

$$\vee\backslash''\phi b \qquad \qquad \qquad \phi\vee\backslash\phi b$$

to scan backwards.

Upon (ö)

This operator differs from *on* in the sense that this time *g* is ambivalent and *f* is monadic. On monadic calls, it doesn't matter whether you use *on* or *upon*. That is $\rho\ddot{\circ}>y$ and $\rho\ddot{o}>y$ are the same.

What really is interesting is when you have a sequence of them. For example, assume we want to apply three functions in sequence to each cell of an array.

² only valid for rank 2 or less objects

We can do

$m1'' > m2'' > m3'' > y$
 or we can do
 $m1 \circ m2 \circ m3'' > y$

$m1'' m2'' m3'' y$
 (*Dyalog* only down here)
 $m1 \circ m2 \circ m3'' y$

The end result will likely be different. In the first case $<m3>$ is called ρy times, then $<m2>$ is called ρy times then $<m1>$ is called ρy times. In the second case the sequence $<m3, m2, m1>$ is called ρy times.

In *Dyalog*, $\circ$ is used to glue functions together, having the same effect in this monadic case only. I will often refer to *Dyalog* for examples in the following text as IBM's APL and APL+Win do not have this feature.

Chaining functions

Chaining functions using the operators seen so far is not that obvious at first. We have to remember the meaning of $''$, $\circ$ and $\circ$ and keep in mind they have, like any other operator, full left scope unless modified by pairs of parentheses.

The monadic case is the same for both SAX and *Dyalog*. Each function is called in sequence.

In the dyadic case chaining functions is interpreted quite differently.

first *Dyalog*

In *Dyalog* all but the leftmost function are assumed to be monadic. The monadic functions are called sequentially until the last one which MAY be called dyadically. For example

$x \ d1 \circ m1 \circ m2 \circ m3'' y$

is the same as doing, for each corresponding cell

$x[i] \ d1 \ m1 \ m2 \ m3 \ y[i]$

As seen earlier, using constants with dyadic functions makes them monadic and is allowed. For example:

$x \ d1 \circ m1 \circ (2 \ 3 \circ d2) \circ (d3 \circ 'ds') \circ m2'' y$

then SAX

In SAX the monadic functions are applied to **both** corresponding cells until the dyadic function is reached. The use of *on* and *upon* or parentheses around the functions may achieve the same or different result. Ex:

```
x  d1◊m1◊m2◊m3◊>y
x  d1◊(m1◊m2◊m3)◊>y
```

is the same and equivalent to

```
<(m1 m2 m3 >x[j]) d1 (m1 m2 m3 >y[j])
```

for each corresponding cell.

upon (◊) should be put just BEFORE the (only) dyadic function if it's not the leftmost function, as in

```
x  m1◊m2◊d1◊m3◊m4◊>y
```

which is, for each corresponding cell,

```
<m1 m2 (m3 m4 >x[j]) d1 (m3 m4 >y[j])
```

Of course clarity may suffer a bit. By the way, this is all in accord with what J does.³

Interpretation of this last example

Remember operators bind "naturally" to the left. Reading from right to left we see that

- ◊ tells us to apply the entire sequence to its left to each corresponding *disclosed* element of 'x' and 'y'
- the rightmost ◊ tells us to apply <m4> monadically to >x[j] AND >y[j] and pass these two results as arguments to dyadic function <m1◊...m3> whose
- rightmost ◊ tells us to apply <m3> monadically to each argument and pass these two results as arguments to dyadic function <m1◊...d1> whose
- rightmost ◊ tells us to use <d1> dyadically and pass its result as argument to monadic function <m1◊m2> whose

³ in J, ◊ is &., ◊ is & and ◊ is @ for function arguments

- $\circ$ tells us to use monadic $\langle m2 \rangle^4$ and pass its result as argument to monadic function $\langle m1 \rangle$
- which returns its result to ``
- which returns the result of the entire sequence after *enclosing* it

Personally I'd recommend breaking down the sequence *if possible*. Or write another function that does the same and call only *that* function, as in $x \text{ myfn}'' > y$.

Dyalog's dynamic functions can achieve the same result by doing

```
x {m1 m2 (m3 m4 α) d1 m3 m4 ω}''y
```

Examples

1: to perform integer division (*floor of divide*) of two lists of numbers we can code

```
x l÷ y           lx÷y
```

The difference here, is that the former is done cell by cell whereas the latter is done function by function (it cannot be done cell by cell in APL2).

2: let's put a few features together. In this example we want to take the first "n" rows of each table in a list and then transpose them:

```
n ⍷÷''>m           ⍷''n+[1]''m
```

Here we have *upon* with short *take with disclose*. 'n' being a series of scalars (vector of integers).

3: find the rank of each cell:

```
ρ÷ρ''>x           ρ∘ρ''x
```

This is more likely to be the kind of code you will encounter.

⁴ it doesn't matter which operator (on or upon) is used from then on since functions can only be called monadically

Scalar functions

In APL2, the use of the *each* operator is assumed for scalar functions (like +). There is no need to specify it⁵. Ex:

$x + '' > y$	$x + y$
$+ '' > / x$	$+ / x$
$+ / '' > x$	$+ / '' x$

Rank

Often functions apply to cells of rank 0 (scalar functions like + or -) or of rank infinite (like <). I only know of one other function, *matrix divide*, which is of a different rank, namely 2.

Normally, *matrix divide* applies to matrices. If you try to use it on a 3d array APL2 complains (RANK error). In SAX it won't. Try

```
4 3 5 = ⍱ 4 5 3 p i * . + i + i 4
```

Each 5 by 3 matrix (4 of them) was inverted to give four 3 by 5 matrices. ⍱ is said to be of rank 2. Each "cell" to which ⍱ was applied was of rank 2. All were 5 by 3 matrices. There were four of them. A 4-element vector of 5 by 3 matrices.

In SAPL this concept of "frame" (the vector in this case) and "cell" (the matrices) is of utmost importance. It breaks up data into piecemeal sections to be fed to functions.

We can view this 4 by 5 by 3 array as a scalar (the frame) containing a 4 by 5 by 3 matrix (the cell) OR as a four-element vector (the frame) of 5 by 3 matrices (the cells) OR as a 4 by 5 matrix of three-element vectors OR as a 4 by 5 by 3 matrix of scalars.

Most functions have a default rank. The function rank determines the rank of the argument cells. For monadic ⍱ it is 2, a matrix. For + is 0, a scalar. For < it is infinite. Any structure will do.

For *dyadic* functions two ranks exist: the left rank and the right rank. Function ε has two ranks (it doesn't have a monadic rank): a left hand rank of 0 (for each item) and a right hand rank of infinite (membership is checked against all of it).

⁵ this is consistent with the definition that all scalar functions apply *disclose* BEFORE the operation and *enclose* AFTER, with the fact that scalars CANNOT be *enclosed*.

SAX allows you to modify the rank of primitives and to specify the rank of YOUR functions *when you use them*.

More examples

We've all been facing this problem: we need to add a vector of numbers, either to each line of a as in

$m + (\rho m) \rho v$ (example 4)

or add a different *scalar* to each *line* of 'm' as in

$m + \phi (\phi \rho m) \rho v$ (example 5)

These are fine but there are sometimes situations that require more thinking than you'd like to do. If we ponder this we see that in the expression ' $m+v$ ' the *add* function is trying to add numbers 1 by 1 (scalar by scalar) and is running out of numbers on one side resulting in a length error of some sort.

We can change that in SAPL. We can re-specify its behaviour by saying: "break down your work into a vector (*frame* of shape 5) of vectors (*cell* of shape 3)" (assuming 'm' is a 5 by 3 and 'v' is a 3 number list).

For example 4 this becomes:

$m + \circ 1 v$ $m + [2] v$ (as in APL+Win)

We modified + to do 5 additions of vectors of 3 elements each. We end up with 5 results of 3 elements each, a 5 by 3 matrix result.

Here the $\circ$ with a numeric right argument is called the *rank* operator and modifies its left argument (here +). The 1 to its right specified how many trailing axes (1 here, vectors) of the arguments were to be part of the cell sections. This made a left hand frame shape of $(\neg 1 + \rho m)$ cells, or 5, and a left hand cell shape of $(\neg 1 + \rho m)$, or 3. The same ' $\circ 1$ ' also made a right hand frame shape of $(\neg 1 + \rho v)$, or 10 (a scalar), and a right hand cell shape of $(\neg 1 + \rho v)$ or 3. Cell sizes are equal (3) and frame shapes are acceptable (5 elements + scalar). All is well.

In example 4 we divided the 5 by 3 matrix into 5 vectors of three-element vectors adding them to [a scalar containing] a three-element vector resulting in 5 results of three-elements each.

In example 5 we needed to add each vector in 'm' to each scalar in 'v'. We needed to modify <+> to take vectors on one side and scalars on the other:

$m + \overset{\circ}{\circ}1\ 0\ v$ $m+[1]v$ (APL+Win)

Here we are saying: "add each vector in 'm' (the 1 on the same side as 'm') to each scalar in 'v' (the 0 on its side)"

There are two numbers to the right of the rank op to specify the rank of each argument. When they are the same, or if the function is monadic, one number may be used as in the first example.

Up to three numbers may be specified to denote monadic, left and right dyadic ranks but since these cannot be stored with a user's function definition it's unlikely you'll ever see code with all three.

6: ravel each plane of a 3d array

$,\overset{\circ}{\circ}(2)7\ 3\ 4\ \rho\ 5\ 6$ $,[2\ 3]7\ 3\ 4\rho 5\ 6$

The parentheses are used to separated the 2 from the '7 3 4' here. Or you could surround the right argument with parentheses:

$7\ 12\ \equiv \rho,\overset{\circ}{\circ}2\ (7\ 3\ 4\ \rho\ 5\ 6)$

or you could use the *pass* function (or any function that does not alter the argument):

$,\overset{\circ}{\circ}2\leftarrow 7\ 3\ 4\ \rho\ 5\ 6$

otherwise '2 7 3 4' would be the argument to $\overset{\circ}{\circ}$ and $,\overset{\circ}{\circ}2\ 7\ 3\ 4$ would be applied to $\rho 5\ 6$ instead. Not the same thing.⁶

APL2 allows *ravel* to be applied to other consecutive dimensions as well. SAPL does not.

$\mathbb{Q},\overset{\circ}{\circ}2\leftarrow 2\ 3\ 1\mathbb{Q}7\ 3\ 4\rho x$ $,[1\ 2]7\ 3\ 4\rho x$

or

$,\overset{\circ}{\circ}2\leftarrow\mathbb{Q}2\ 1\ 3\mathbb{Q}7\ 3\ 4\rho x$

⁶ actually it wouldn't work as $\overset{\circ}{\circ}$ would issue a length error

7: enclose each line of a table

```
<ö1 table
```

Going back to example 4, we could re-code it as

```
(<ö1 m)+ö><v                =>(c[2]m)+c v
```

and example 5 as

```
(<ö1 m)+ö>v                =>(c[2]m)+v
```

If the frame size needs to be specified (as opposed to the cell size) simply use negative (complementary) rank. For example, monadic function *table* ($\bar{\tau}$) is defined as

```
,ö-1-1/x
```

meaning "ravel all but the first axis of the argument", making sure 'x' is at least a vector (1/x). For a 3d array all planes (rank 2) are raveled, for a matrix all vectors (rank 1) are raveled and for vectors all scalars (rank 0) are raveled.

8: create an array from the shape of another.

Here we wish to create a table of 'abc's with the frame of another:

```
'abc'→ö1 x                ((-1+px),3)ρ'abc'
```

9: apply rank to your own functions.

Here we have a function, <R2fn> which takes a matrix (and nothing else) as argument and returns ten numbers. To use it on higher rank arrays we tell APL to use matrices as cells ($x \leftarrow 2 \ 3 \ 4 \ 5 \rho 199$):

```
2 3 10=ρR2fnö2 x          =>R2fn''c[-2+1ρpx]x
```

Cut ($n\bar{\circ}$)

This is a different *function*. It uses $\bar{\circ}$ again but this time with a *left* numeric argument.

This is the same as *Dyalog's* dyadic $\bar{c}$ with variants. APL+Win uses $\bar{\square}$ penclose instead. Both of these work along the *last* axis unless brackets are used. SAX works along the *first* axis.

In *Dyalog*, $b \in x$, with 'b' boolean, cuts pieces of 'x', using 1s in 'b' as delimiter to indicate the start of a partition, and encloses them.

In *SAX*, $b \overset{\circ}{\circ} x$ does the same with $n=1$. 'b' specifies where to cut and $n=1$ means the start cuts where $b=1$. Variants exists and $n=2$ means that each cut ENDS on a 1. Ex: b is 0 1 0 1 0 0 1 0, $\square io$ is 1

$b \overset{\circ}{\circ} \in 18$	$b \in 18$
(2 3 4 5 6 7 8)	(+answer)
$b \overset{\circ}{\circ} \in 18$	$(1 + (\vee / b), b) \in 18$
(1 2 3 4 5 6 7)	

The monadic case assumes the first cell is a delimiter. For ex, with $s \leftarrow ' , az , bc , def '$:

$1 \overset{\circ}{\circ} s$	$(s \in 1 + s) \in s$
(' , az ' > ' , bc ' > ' , def ')	(+answer)

To exclude the delimiter use a negative left argument:

$\neg 1 \overset{\circ}{\circ} s$	$1 + \neg (s \in 1 + s) \in s$
(' az ' > ' bc ' > ' def ')	(+answer)

Function <vtom> revisited:

This function has to be the function most rewritten in APL. The < used so far can be changed to any monadic function. If we use *pass* we get

$\neg 1 \overset{\circ}{\circ} s$	$\neg 1 + \neg (s \in 1 + s) \in s$
az	
bc	
def	

As usual the pieces are reassembled to fit the largest.

Cut also works on higher rank arrays but always on the first dimension. Ex. $b \leftarrow 1 \ 0 \ 1 \ 0$ and $m \leftarrow 5 \ 3 \rho \ 121$:

$b \overset{\circ}{\circ} (+ /) m$	$\neg + / \neg b \in [\square io] m$
------------------------------------	--------------------------------------

To cut on another axis use rank (here the last one):

$b \overset{\circ}{\circ} (+ /) \overset{\circ}{\circ} 1 \ m$	$\neg + / \neg b \in m$
---	-------------------------

More examples

10: suppose we are given a list of series (a vector of vectors) of resistors and wish to find the overall resistance of each series (formula $\div + / \div$) we could use

```
 $\div \circ (+) \circ (\div \circ 1) \circ > list$                        $\div \circ (+) \circ \div '' list$ 
```

We use $\div \circ 1$ because $\div$ is a scalar (rank 0) function and composition operators propagate the rank of the first function. Had we not done so $+ /$ would also apply to each scalar and we would not get the result expected.

Because the inverse propriety of $\div \circ 1$ is not defined we cannot do

```
 $+ / '' (\div \circ 1) \circ > list$ 
```

11: compare two matrices of the same width, line by line. This is the same as $v1 \circ . = v2$ for matrices:

```
 $m1 \circ . (= \circ 1) m2$                        $(\subset [\square io] m1) \circ . \equiv \subset [\square io] m2$ 
```

12: this is the same problem as 2 except that 'm' is of various higher ranks arrays. Were it not for the rank operator the solution would be harder to come by.

```
 $n \circ \circ (\div \circ 2) '' > m$                       (solution too complex in APL2)
```

Dot (.)

This operator behaves as in other APLs except that it can also be used monadically. Its definition is fairly complex for the general case but it should be enough to know that $- . \times$ returns the determinant of a matrix. Ex:

```
 $100 = - . \times 3 3 p 3 4 7 6$ 
```

Select (})

This operator takes a selection function, like $1 0/$ or $2 3\{$, applicable to the right argument and merges the left argument within it. In APL2 this merging occurs within a variable. In SAX a result is returned instead:

```
 $x \leftarrow 9 \ (2''\{) \} 13$                        $x \leftarrow 13 \ \circ \ (3 \supset x) \leftarrow 9$ 
```

Here, $2''\{$ selects the 3rd element from 13 which is replaced by 9.

We can do specific items or whole lines. Here the selector function is " $b/$ ":

$x \leftarrow 8 \quad b/ \} 2 \quad 3 \rho 16$

$x \leftarrow 2 \quad 3 \rho 16 \diamond (b/x) \leftarrow 8$

Conclusion

SAX is definitely different from APL2 when it comes to using operators. Sometimes it is easier to use, sometimes APL2 is easier. J programmers will be at (better) ease with this material as most operators are already defined this way in J.

I understand both worlds. I wish we could define our own operators though. Later maybe.

For questions and remarks I can be reached at danb@dsuper.net.

Dan Baronet
Montreal, Canada.

Functional Implementation of Moments Based Region Descriptors in J

C. Anne Chamley

Abstract

For many years, the field of computer vision has been concerned with accurate ways of describing shapes. One manner of describing shapes is to use statistical region based descriptors. Moments invariants are often used for this purpose, and are a well respected method of description. The following article describes the use of functional programming, using the J language, to implement six moment based descriptors for the description of icebergs in blob coloured images.

Introduction

The theory of computer vision relies upon the ability to accurately describe objects or phenomena. These descriptions can then be used for pattern matching, or object recognition, for example. Both of these tasks rely on accurately describing the concepts in an image, since any further processing uses these descriptions.

In this article, we will be examining the implementation of a set of region based descriptors, which are derived from moments theory. A moment is a statistical measure of the properties of an object, and different properties can be derived from the moments of an object. This paper will illustrate the implementation of six moments based descriptors, which use the zeroth, first and second order moments.

A discussion of the theoretical basis of moments theory is beyond the scope of this paper, and will not be discussed here. For more information, refer to Prokop and Reeves (1992).

Moments techniques are particularly suited to image analysis because they can be adapted to use an array based approach. Since many data analysis packages have optimised array operations (e.g. IDL), it is convenient and efficient to have techniques that can exploit these array optimisations.

Reading the image

Description

The first step is to read in the image. The image is input as a text file, where each row is separated by a carriage return and line feed. These files are of variable length, both in the number of rows and the length of each row. Each image is a blob coloured image, produced by the SARTRACKER system (Williams *et al*, 1998). Images are included in the appendix for further information.

Functions

The unprocessed file can be read in using the following monadic functions, taking the file name as the argument:

```
filename =: ,~& 'c:\windows\desktop\'
read =: 1!:10<@filename
```

The input file is a text file, where each row is separated by a carriage return and line feed (ASCII codes 13 and 10, respectively). To perform the moments calculations on the image, it must first be converted to a numerical array, which can be easily achieved using the ". or 'do' function. However, this function does not produce the desired result unless the text file is first stripped of the carriage return and line feeds.

Two basic functions were developed to accomplish the task; get1 and take1. The get1 function takes a copy of the file until the next carriage return character. The take1 function then removes the next line from the array, including the carriage return and line feed. The result of this two step process is that we have the first row of the array, free of line feeds and able to be changed from text to numeric data.

```
get1 =: {.- i.&(a. {- 13)
take1 =: }.- (+~&2@ i.&(a. {- 13))
```

The next step is to use these functions repeatedly, in order to strip the end lines from every row of the image. Since we do not know how long each row will be, or how many rows there are, we need to use an approach that will continue reading data until there is no data remaining. When used with the right argument of infinity, the power operator can perform this function. The plan then, is to produce a fork to box the unprocessed text file, and to catenate an empty box to the front of this.

This is implemented as the boxed read (br) function:

```
br =: ;~&' '@(<@read)
```

In order to move rows from one side of the box to the other, a simple head and tail function were implemented:

```
tl =: ,@(>&).)
hd =: ,@(>&{.)
```

These can then be used in a fork which retrieves one line of the text file:

```
gol =: (hd,getl@tl) ; (take1@tl)
```

Which can then be used with the ^: _ , as described above, to read in the entire text file.

```
readfile =: ".&hd@gol^:_@br)
```

Results

The functions to read the file produce large arrays of data. For readability, the images used in the functions have been included in the Appendix. As a consequence, the results of these functions will not be shown here, as they are the basis of the other functions in this paper.

Other functions

Description

At this stage, it is convenient to describe a few simple utility functions, which are later used to compose more sophisticated functions.

Functions

The first of these gives the number of elements in an array:

```
nele =: # readfile
```

The next two functions, as the names suggest, give the number of rows and columns, respectively, in an image.

```
rows =: +/&{ = &13 @ (a.&i.) ) @ read
cols =: nele % rows
```

Finally, the `numberg` function gives the number of icebergs in an image:

```
numberg =: #@~.@readfile
```

Results

```
rows 'test1.txt'
22
rows 'test2.txt'
23
cols 'test1.txt'
16
cols 'test2.txt'
31
nele 'test1.txt'
352
nele 'test2.txt'
713
```

Calculating the moments

Description

Moments are a well established statistical measure, used traditionally to show the amount of variance in a statistical sample (Prokop and Reeves, 1992). However, Hu showed that moments theory can equally be applied to images, in order to gain information about the spatial properties of regions within the image (Hu, 1962). The moments for an unoccluded object in a discrete image can be given by:

$$m_{pq} = \sum_{y=0}^{M-1} \sum_{x=0}^{N-1} x^p y^q g(x, y) \quad (\text{Sonka et al, 1993})$$

Where : M is the number of columns

N is the number of rows

g is the binary image of an object.

Functions

To consistently use the array based approach, we first construct an array containing the x and y co-ordinates of the image. This is done by using the `i.` operator to make an array that contains every integer between 0 and $n-1$, where n is the number of items in each row. This is determined by dividing the number of

elements by the number of rows. Similarly, the y array is calculated by using the i. to make an array from 0 to n-1, where n is the number of rows.

```
xarr =: cols | i.&nele
yarr =: cols <. @%~ i.&nele
```

At this point, the image is a one-dimensional array. We can easily translate this into a two-dimensional array by reshaping the array by the number of columns, but it is more convenient for the moment to leave the image as a one-dimensional array. However, to perform the moments calculations, we need to produce a binary image of each iceberg. The way that this has been achieved is to use a fork which produces a binary image for each ID number indicated by taking the nub of the image.

```
lab =: readfile = "_ 0 -.@readfile
```

The moments is calculated by producing a fork, where the left component is the x array to the power of p, and the right component is a fork that multiplies y to the power of q by the value of the image at each pixel. The right argument of the top level fork is the name of the image file, and the left argument is a vector containing the order of the vector to be calculated. For example, 0 0 moment 'image.txt' will give the zeroth order moment of the image in the file "image.txt".

```
moment =: +/10(((xarr@)) ^ {.@() * ((yarr@)) ^ {.@().@() * _ 1 lab@))
```

The result of this function is a vector, where each element is the moment of a region in the image. The identifier of the region can be obtained by the nub or the image, using the ~. operator. For example, ~.@readfile 'test1.txt' gives the vector 1 2 3 4 5, meaning that any region based statistics will refer to the regions in this order. Similarly, ~.@readfile 'test2.txt' produces 1 4 2 3. All other region descriptors discussed in this paper produce vectors which should be interpreted in a similar manner.

Results

```
0 0 moment 'test1.txt'
72 183 44 17 36
```

```
0 0 moment 'test2.txt'
104 490 44 75
```

Area

Description

The first region based descriptor to be calculated from the moments is also the simplest. The zeroth order moment gives the area of a region. Therefore, the area can be obtained by:

$$m_{00} = \sum_{y=0}^{M-1} \sum_{x=0}^{N-1} x^0 y^0 g(x, y) \quad (\text{Prokop et al, 1992})$$

Functions

An examination of this formula shows that this gives a simple count of all pixels in the region. This is implemented by a fork, which binds the vector 0 0 to the moment function, forcing it to take 0 0 as the left argument.

```
area =: 0 0&moment
```

Results

```
area 'test1.txt'
72 183 44 17 36
```

```
area 'test2.txt'
104 490 44 75
```

Centre of Mass

Description

The centre of mass (COM) can be described as the point about which all mass is evenly distributed – the centre of the object. The centre of mass (denoted by $\bar{x}$ and $\bar{y}$) is obtained through the first order moments, using the following equations:

$$\bar{x} = \frac{m_{10}}{m_{00}} \quad \bar{y} = \frac{m_{01}}{m_{00}} \quad (\text{Prokop et al, 1992})$$

Functions

This can be achieved by formulating a visually complex, yet quite simple fork:

```
com =: (2&,@ numberg)$((1 0&moment % 0 0&moment),
(0 1&moment % 0 0&moment))
```

The result of this function is a two-dimensional array, where the first row of the array gives the x location of the COM for each shape, and the second row is the y co-ordinates of the COM for each shape. Thus, each column gives the complete COM for each shape. These can be indexed in the same way as the moments data - via the nub of the image.

Results

```
com 'test1.txt'
 7.5      8.2459      5.77273      9.29412      4.97222
10.5      9.43716      8.02273      15.0588      16.7778
com 'test2.txt'
15      14.4735      10.0455      21.3467
11      10.7939      10.6364      12.56
```

Central moments

Description

The central moments of an image can be used to calculate more advanced descriptors, and are denoted as μ_{pq} . The central moments are formed by calculating the centre of mass, then translating the objects such that the centre of mass is coincident with the origin of the image. The moments calculated on this normalised image (using the equation for moments as given above) are known as the central moments (Prokop *et al*, 1992).

Functions

As discussed above, the COM can be used to reposition the object so that the COM is coincident with the origin of the image. In keeping with our array based approach, we can simulate this by altering the x and y arrays, according to the value of the COM.

```
cxarr =: (xarr - "1 0 {.@com)@] ^ {.@[
cyarr =: (yarr - "1 0 (, @).@com)) ^ {.@).@[
```

These altered x and y coordinates can be used to calculate the central moments of an object, by substituting the altered x array for the original x array, and likewise with the y array. Thus:

```
mu =: +/"1@(cxarr*cyarr*lab@)]
```

The central moments are denoted by μ_{pq} .

Results

```
1 0 mu 'test1.txt'
0 1.65201e_13 1.06581e_14 _1.24345e_14 2.04281e_14
1 0 mu 'test2.txt'
0 _1.7053e_13 _1.95399e_14 _7.10543e_14
```

Orientation Axis

Description

The orientation axis gives some sense of direction to the object. The formula below describes ϕ , which is the angle between the x axis, and the principal axis, major or minor, which is closest to the x axis.

$$\phi = \frac{1}{2} \tan^{-1} \left(\frac{2\mu_{11}}{\mu_{20} - \mu_{02}} \right) \quad (\text{Prokop et al, 1992})$$

Functions

Implementing ϕ in J requires only slight complexity, using a fork of the form:

```
f1 =: +:@(1 1&mu) -:@(_3&o.%) 2 0&mu - 0 2&mu
```

Results

```
f1 'test1.txt'
0 _0.118375 0.0137031 _0.0341932 0.531458
f1 'test2.txt'
0 _0.0495822 0.310555 _0.378673
```

Image ellipse

Description

The image ellipse describes the ellipse which most closely fits a shape - that is, an ellipse about which all the first order moments are the same as the actual object. The image ellipse is defined in terms of a major and minor axis, which can be calculated using the following formulae:

$$\alpha = \left(\frac{2 \left[\mu_{20} + \mu_{02} + \sqrt{(\mu_{20} - \mu_{02})^2 + 4\mu_{11}^2} \right]}{\mu_{00}} \right)^{\frac{1}{2}} \quad (\text{Prokop et al, 1992})$$

$$\beta = \left(\frac{2 \left[\mu_{20} + \mu_{02} - \sqrt{(\mu_{20} - \mu_{02})^2 + 4\mu_{11}^2} \right]}{\mu_{00}} \right)^{\frac{1}{2}} \quad (\text{Prokop et al, 1992})$$

Where α is the major radius and β is the minor radius of the ellipse. Obviously, the approximate length and width of the region will be twice the value of the major and minor radii (respectively).

The intensity of the image ellipse is a measure of how well the ellipse fits the object. This is a very simple calculation, and is specified by:

$$I = \frac{\mu_{00}}{\pi\alpha\beta}$$

Functions

The calculation of the principal axes was implemented using two rather complex forks:

```
top = (+: @+: @*: @mu-4 1 1 %: @+ 2 0&mu *: @-0 2&mu)
major =: 0 0&mu %: @%- 2 0&mu +: @+ 0 2&mu + top
minor =: 0 0&mu %: @%- 2 0&mu +: @+ 0 2&mu - top
```

The intensity also makes a simple fork, using the major and minor functions:

```
I =: 0 0&mu % major o.@* minor
```

Results

major 'test1.txt'					
16.4283	12.045	5.64905	2.63637	4.4969	major
'test2.txt'					
23.5421	17.4863	5.19964	6.35354		
minor 'test1.txt'					
12.7584	9.036	2.6229	2.21923	2.60758	
minor 'test2.txt'					
18.6486	13.2773	2.83953	4.12198		

Conclusion

A functional approach to moments based descriptors provides a clear illustration of the simplicity of both the descriptors and the J language. The functions implemented in this paper illustrate how moments theory can be put to practical use in image processing, using blob coloured images. Further work in this area could include functions which would implement pattern matching over a number of images, using the region based descriptors implemented in this paper.

References

1. Holmes, W.N. "Functional Calculation" (series of articles to be offered for publication) 1996.
2. Hu, M.K. "Visual pattern recognition by moments invariants" *IRE Transactions on Information Theory* IT-8, 179-187, 1962.
3. Sonka, M, Hlavac, V., and Boyle, R. *Image Processing, Analysis and Machine Vision*. London: Chapman & Hall, 1993.
4. Williams, R.N., and Crowther, P. "Application of a Technique, based on Local Image Homogeneity, to delineate Spatial Objects in Satellite Images" In Publication, IJRS 1998.
5. Prokop, R.J., and Reeves, A.P. "A survey of Moment-Based Techniques for Unoccluded Object Representation and Recognition" *CVGIP: Graphical Models and Image Processing* vol. 54, no. 5, September 1992.

Appendix A – Images

Two images were used to test the functions described in this paper. The preprocessing of the images was completed by the SARTRACKER program (Williams, Rees and Young - see Williams *et al* 1998). The images depict icebergs in the Southern Oceans surrounding Antarctica, and the original images were synthetic aperture radar images from ERS-1. The images shown here are segmented, and each shade of grey represents a numerical value, depicting a region to which the pixel belongs. There are two regions in these images that do not depict icebergs. These are the border, and the background. Although these are treated in the same manner as icebergs, a cursory examination of the images reveals that they are not icebergs.

Image "test1.txt"

[illegible]

Fast Fourier Transforms, Diffraction Patterns, and J

by Ned W. Allis (allisn@lafayette.edu),
Jeffrey P. Dumont (dumontj@lafayette.edu),
Flynn J. Heiss (flynnheiss@yahoo.com),
Clifford A. Reiter (reiterc@lafayette.edu)

The fast Fourier transform package available for J is very useful for simulating diffraction patterns. Here we use it to replicate a classic optical transform and explore its power in creating diffraction patterns of fractal images and chaotic attractors. These types of images are already visually stimulating, and we found that the diffraction patterns could also yield remarkable images. We will define some J functions which are helpful for viewing the results of the J Fourier transform package.

The diffraction patterns of crystals have been the focus of research since the discovery of x-ray diffraction in 1912. Diffraction patterns and their relationship to Fourier transforms are discussed in detail in [6]. Renewed interest in diffraction patterns has arisen due to the discovery of quasicrystals which exhibit classically forbidden symmetry in their diffraction patterns. Senechal gives a good introduction to quasicrystals [11].

The Fast Fourier Script in J:

The fast Fourier transform package can be obtained from the J website [4]. Our J examples assume that the `fftw` add-on has been installed and the `fftw.ijs` script has been loaded. The package is based on the code written originally by Frigo and Johnson [2]. The script includes both the forward and reverse transforms as predefined functions `fftw` and `ifftw`. A detailed description of the mathematics involved can be found at [2], but here we offer a basic explanation. The `fftw` function defined in the script takes a complex-valued array X of any dimension d as its right hand argument and returns another d -dimensional complex-valued array Y according to the following definition:

$$Y(i_1, i_2, \dots, i_d) = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} \dots \sum_{j_d=0}^{n_d-1} X(j_1, j_2, \dots, j_d) \omega_1^{-i_1 j_1} \omega_2^{-i_2 j_2} \dots \omega_d^{-i_d j_d},$$

where $\omega_s = e^{2\pi\sqrt{-1}/n_s}$, for all $s \in \{1, 2, \dots, d\}$.

The reverse transform is computed using:

$$Y(i_1, i_2, \dots, i_d) = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} \dots \sum_{j_d=0}^{n_d-1} X(j_1, j_2, \dots, j_d) \omega_1^{i_1 j_1} \omega_2^{i_2 j_2} \dots \omega_d^{i_d j_d}.$$

Notice that the only difference between the forward and the reverse transforms is the minus signs within the exponents. We now give a simple example of the use of the `fftw` function.

```
load 'c:\j403\system\packages\fftw\fftw.ijs'
]m=: 2 2$1 2 1j1 3j1
1 2
1j1 3j1
fftw m
7j2 _3
_1j _2 1
```

Computations using `fftw`, although rapid, can be very memory intensive. Most of the input arrays we work with are approximately 1000 by 1000 pixels. The following function

```
ts=: 6!:2 , 7!:2@:]
```

measures time and space usage. In Table 1, we list the time and memory that it took to perform an `fftw` operation on our 400 MHz PC. We note that the transform is significantly faster on arrays whose size is a power of 2, although the transform on a slightly larger array uses the same amount of memory. It also seems that the limit on our system of `fftw` on square matrices is 2047 by 2047.

Size	Time	Memory
64 by 64	0.016 sec	1.42 megs
128 by 128	0.062 sec	5.65 megs
256 by 256	0.390 sec	22.55 megs
512 by 512	1.578 sec	90.19 megs
513 by 513	3.141 sec	90.19 megs
1024 by 1024	9.531 sec	360.725 megs
1920 by 1920	30.74 sec	721.435 megs
2047 by 2047	51.656 sec	721.435 megs

Table 1. Time and memory involved in `fftw` computations.

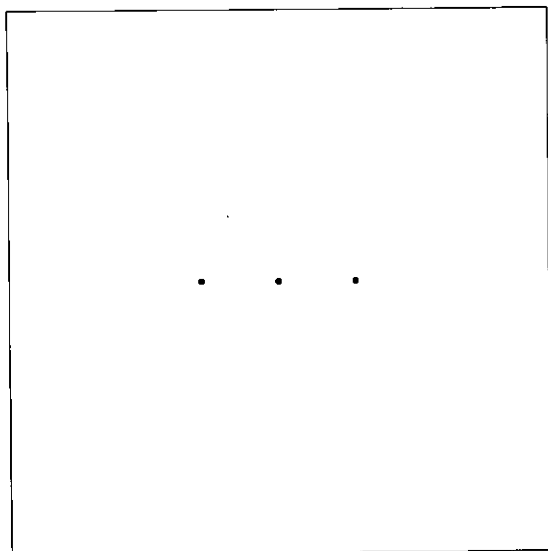


Figure 1: Three apertures

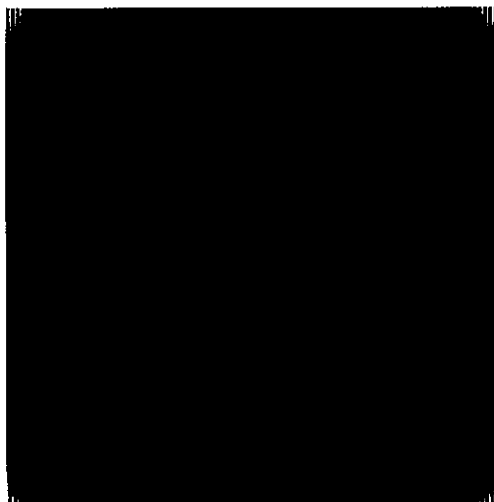


Figure 2: The magnitude of the Fourier transform of Figure 1 as given by the `fftw` function with `dis`

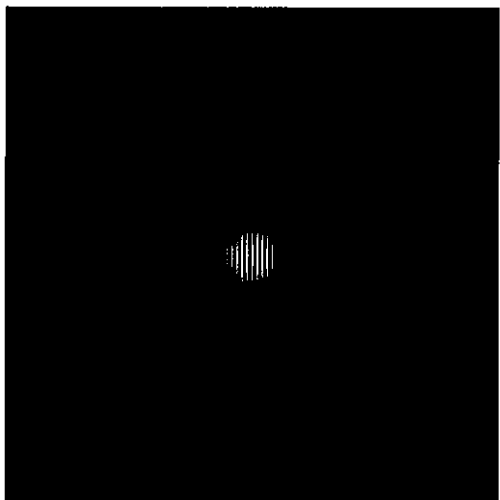


Figure 3: Figure 2 as recentered by the HR function

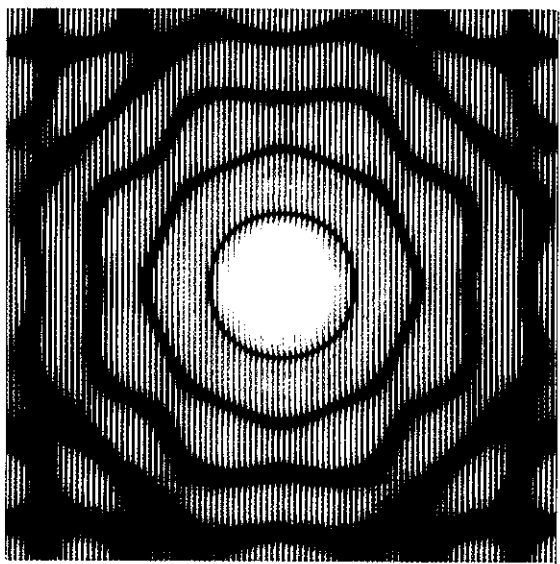


Figure 4: The magnitude of the Fourier transform of Figure 1 as given by the cile function and recentered according to the HR function

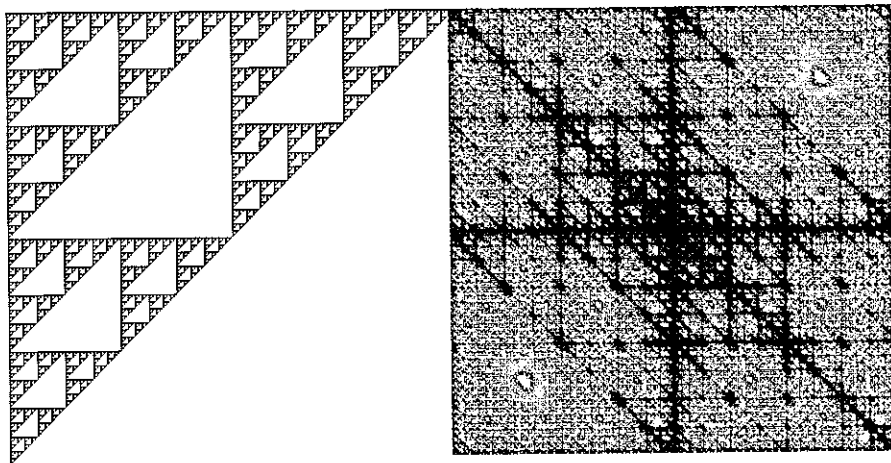


Figure 5: A Sierpinski triangle and the magnitude of its transform

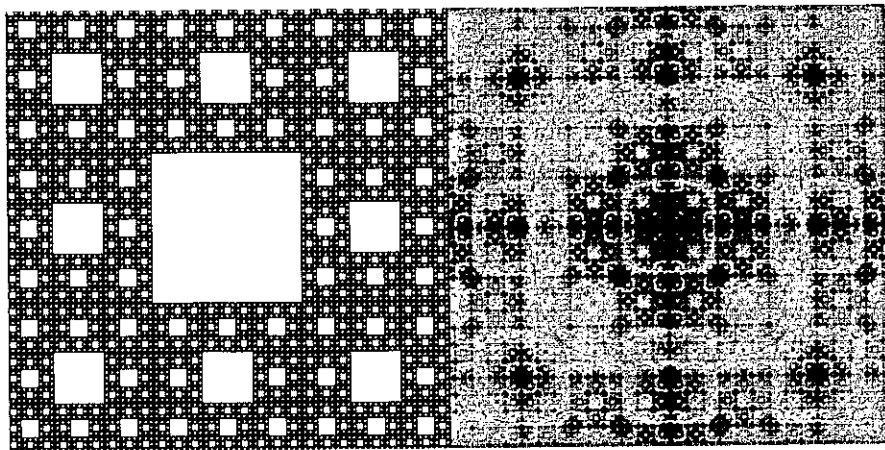


Figure 6: A Sierpinski carpet and the magnitude of its transform

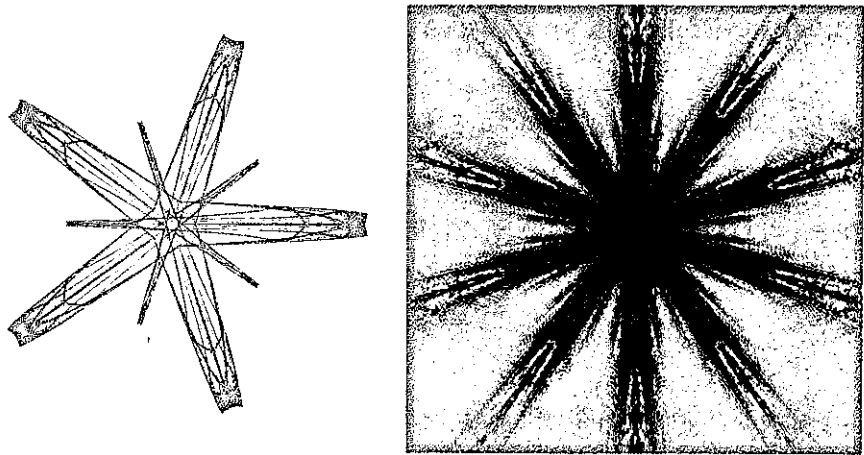


Figure 7: A chaotic attractor with dihedral 5-fold symmetry and the magnitude of its transform

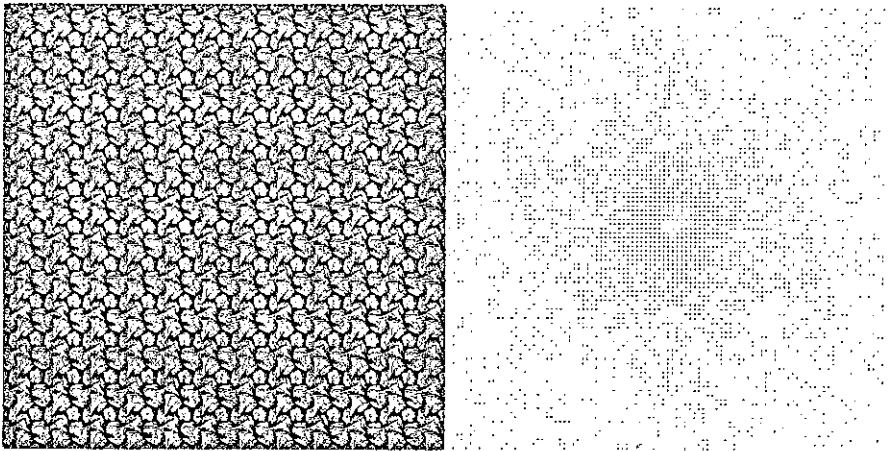


Figure 8: A chaotic attractor with forbidden symmetry and the magnitude of its transform

Diffraction Patterns

As a first application of `fftw` we replicate one of the experiments in the Atlas of Optical Transforms [3]. The input, seen in Figure 1, consists of three circular apertures arranged horizontally, and can be found on the left side of plate 10 in the Atlas of Optical Transforms. The corresponding diffraction pattern can be found in the same position on the right side of plate 10. To recreate that input, we use

```
a=:100 100$0
b=:_100 _100{.66 66{.255*25>+/"1*:, "0/~i:15
B=:_700 _700{.500 500{.(a,.a,.a) ([,],[,])b,.b,.b
```

Here, `a` gives a 100 by 100 array of 0's, `b` gives a circular region of 255's embedded in a 100 by 100 array of zeros, and `B` gives us the desired array displayed in Figure 1. We then compute the magnitude of the complex-valued Fourier transform. Next we use the `dis` function to make these values discrete in order to create an 8-bit greyscale bitmap.

```
max=:>./@,@]
min=:<./@,@]
maxout=:0.99996*@[
dis=:[:<.(maxout*]-min)%max-min
```

Here we demonstrate the use of this function:

```
x=(1+i.20)%19r5p1
Sin=:1&o.
data:=(Sin x)%x

10 dis data
9 9 9 9 9 9 8 8 7 7 6 6 5 4 4 3 2 1 0 0
```

Applying the `dis` function to the magnitude of our transform, we write out the bitmap using `writebmp8` as defined in `raster3.ijs` [10] using a greyscale palette, `g256`.

```
tr=:256 dis |fftw B
g256=:3#"0 i.256
(g256;tr) writebmp8 'figure2.bmp'
```

Figure 2 almost gives us the expected diffraction pattern. It turns out that the corners of our images look similar to what we should see as the centre and vice versa.

Thus, we use the following functions to re-centre our image.

```
hr=: <.@-:@# |. ]
HR=: hr"1@:hr
```

The function `hr` rotates an array half way along its length while `HR` applies `hr` to both axes of a matrix. Applying `HR` to the data shown in Figure 2 yields Figure 3, which closely resembles the expected diffraction pattern found in the Atlas of Optical Transforms; while this image seems distressingly black, it actually shows slightly more contrast than the image in the atlas. In fact, we can see an extra ring.

At this point, we maximise the contrast to better understand the diffraction pattern. This leads us to the use of the following function from [8]:

```
cile=: $@] $ ((/:@/:@] <.@: * (%#)),)
```

Here we demonstrate the use of the `cile` function:

```
10 cile data
9 9 8 8 7 7 6 6 5 5 4 4 3 3 2 2 1 1 0 0
```

Notice the difference between the outputs of the `cile` and `dis` functions applied on the same data. The output of the `cile` function distributes the data into groups of 2 while the `dis` function has an abundance of 9s but only one 2. Using this function on the output of the Fourier transform, `HR 256 cile |fft w B`, provides us a detailed look at the fringes in the diffraction pattern (see Figure 4).

Transforms of Sierpinski Fractals

Other visually attractive diffraction patterns are created by using Sierpinski fractals as input. We created both the Sierpinski triangle and carpet with the following function given in [7].

```
ggl=: +./"1@([ (+./@(+./)@(</~@i.
    @[*.*."1/-@]))"2>@>@{#@(<#<@(<"1@]))})
```

Figure 5 displays a Sierpinski triangle on the left created with

```
st=: 2 ggl (n#3)#: i.2^n=.9
```

Its Fourier transform is located on the right side in Figure 5. Here we maximise contrast with the `cile` function. The palette is black at one end, white at the other end, and blends between red and magenta in the middle. Note that the Fourier

transform, like the input array, displays self-similarity, a common feature of fractals. Figure 6 displays a Sierpinski carpet on the left created by

```
sc=:2 ggl (n#3)#:i.3^n=.5
```

and shows its Fourier transform on the right, which also displays fractal-like qualities.

To give an example of a 3-dimensional Fourier transform, we create a Menger Sponge as follows.

```
ms=:3 ggl (n#3)#:i.3^n=.4
```

We then take the Fourier transform of the sponge and display each plane of both the Menger Sponge and the transform in an animation which may be found at [9].

Transforms of Chaotic Attractors

We will see that chaotic attractors and their diffraction patterns may be visually appealing. We start with an attractor with rotational symmetry. Such images have diffraction patterns that also contain rotational symmetry which is either equal to or twice that of the original image. For example, the left side of Figure 7 displays a chaotic attractor with 5-fold rotational symmetry, which was created by the methods discussed in [5]. Its transform, located on the right, has 10-fold symmetry, arising from the centrosymmetric property of the Fourier transform [11] combined with the original symmetry type carried over by the transform.

In search of quasicrystal-like chaotic attractors, we also tried taking transforms of chaotic attractors with almost forbidden symmetry created with techniques discussed in [1]. Figure 8 gives an example of a chaotic attractor with almost pentagonal local symmetry and its transform, which consists of points on a square lattice. These points are so focused that they need to be enlarged. We use the following J function to make them more visible.

```
tess=:4 4&((>./@,);.-3)
```

This results in a 7-fold magnification of the lattice points on the right side of Figure 8.

Acknowledgement: This work was supported in part by NSF grant DMS-9805507. The chaotic attractor in Figure 7 was joint work with a co-author from [5].

References

- [1] Dumont, J. and Reiter C. *Chaotic Attractors near Forbidden Symmetry*, Chaos Solitons and Fractals, to appear
- [2] Frigo, M. and Johnson, S. *FFTW Homepage*, <http://www.fftw.org>
- [3] Harburn, G. , Taylor, C. A. and Welberry, T. R. *Atlas of Optical Transforms*, G. Bells & Sons, Ltd, (1972)
- [4] J Software, <http://www.jsoftware.com>
- [5] Jones, K. and Reiter, C. A. *Chaotic Attractors with Cyclic Symmetry Revisited*, Computers and Graphics, to appear
- [6] Lipson, S. G. , Lipson H. and Tannhauser D. S., *Optical Physics*, Cambridge University Press, (1995)
- [7] Reiter, C. A. *Sierpinski Fractals and GCDs*, Computers and Graphics, 18 (1994) 885-891
- [8] Reiter, C. A. *Fractals, Visualization, and J*, Iverson Software, (1995)
- [9] Reiter, C. A. <http://www.lafayette.edu/~reiterc/fft/>
- [10] Reiter, C. A. <http://www.lafayette.edu/~reiterc/j/>
- [11] Senechal, M. *Quasicrystals and Geometry*, Cambridge University Press, (1995)

Ned W. Allis
Lafayette College, Box 8210
Easton, Pa 18042, U.S.A.

Jeffrey P. Dumont
Lafayette College, Box 7756
Easton, Pa 18042, U.S.A.

Flynn J. Heiss
57 Main Street
Geneseo, NY 14454, U.S.A.

Clifford A. Reiter
Department of Mathematics, Lafayette College
Easton, Pa 18042, U.S.A.

TECHNICAL SECTION

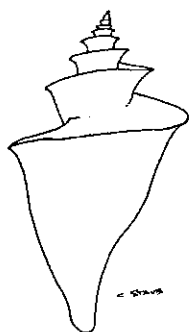
This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: Making Compressed GIFs in APL+Win and Dyalog APL	Adrian Smith	119
PROPER Capitals	Adam Kertesz	125
At Play with J: We'll Cross that Bridge when we Come to it	Gene McDonnell	126
Setting up a www-server with Dyalog APL and IIS	Janne Mäenpää	131
Sending MIDI Messages to a Sound-card with J	Martin Neitzel	130

Renaissance Data Systems

All APL Books in Print



We are pleased to announce that RDS is now on the web at www.aplbooks.cnchost.com with a display of our entire catalog of APL Books in Print. There are even some classic publications which we have received permission to reproduce. Subjects include APL and Mathematics, Learning APL, and APL for the Professional Programmer. We also carry a few books of interest from the rest of the world!

You may contact us using email at rendata@aplbooks.cnchost.com and place orders at orders@aplbooks.cnchost.com. Credit card payment is available for international customers.

Please note our new mailing address:

**Renaissance Data Systems
P. O. Box 313
Newtown, CT 06470.**

Hackers' Corner: Making Compressed GIFs in APL+Win and Dyalog APL

*by Adrian Smith, with assistance from Gert Glantz
(gert.glantz@tietoenator.com)*

Disclaimer

Unusually for a Hackers' Corner, this article requires you to go out and buy some additional software to make it all work. However it does cover some interesting techniques which the +Win programmer needs to recover the bits from a picture object and save the result as a bitmap file. This is courtesy of Gert Glantz, and came via Carl House who uses much of the HTML code published in previous *Vectors* to maintain a large and complex web-site. The BMP conversion code may be downloaded from the APL2000 website.

Why Bother?

The problem with web graphics is gradually receding as 'modern' formats like .PNG replace the old lingua franca of the Compuserve GIF. However GIF is still by far the most commonly recognised filetype, and is very efficient in storing simple block graphics, where the run-code compression can work well. The snag is that the algorithm used to compress GIFs has been retrospectively claimed as copyright by Unisys, who are vigorous in pursuing any software development company which dares to post GIF files on its web site. The answer is to find a software package (such as PaintShopPro) which is licensed, and to make your GIF files with this. Unfortunately this is a very labour intensive solution if you are automatically generating a large stack of pictures (like the climate charts on the Causeway website) as PSP has no command-line interface.

Carl House and I had reached roughly this point in the weeks leading up to Orlando, when he discovered that JASC (the PSP people) also sell a package called Image Robot [1] which is essentially a command-line, scriptable PaintShopPro-5. Now all we have to do is to generate our image (in +Win we use a 'Picture' object), capture the bits, generate a standard .BMP file and launch Image Robot from the command-line. Well, it works a treat, and as far as I know is both legal and decent, as well as honest.

Stage-1: First make your BMP

Fortunately, Dyalog does this bit for us, so what follows is only necessary for APL+Win. Dyalog readers may skip ahead to the next subhead!

```

v z←Image2BmpFile im;h;cmap_entries;bits_per_pixel;offbf;offbc;ih;iw
[1]  a Creates the contents of a BMP-file from an image
[2]  a im=image created with "Get" method or "image" property
[3]  a z=file contents
[4]  a| 1998 9 18 11 6 37 a Gert Glantz, Enator Sweden
[5]  a Please send comments and suggestions to: gert.glantz@enator.se
[6]  (ih iw)+pim a Image-dimensions
[7]  cmap_entries+0 a No of entries in colormap
[8]  bits_per_pixel+24 a No of bits per pixel
[9]  h+(14+40)p[tcnui a Initialize file header
[10] offbf+0 a Fileheader offset
[11] offbc+14 a Infoheader offset
[12]
[13] a Bmpfileheader
[14] h[offbf+12]←'BM' a Always 'BM' in the 2 first bytes
[15] h[(offbf+2)+14]←I2C 14+40+(cmap_entries*4)+(ih*iw) a bfSize = file size
[16] h[(offbf+10)+14]←I2C 14+40+cmap_entries*4 a bfOffBits = header size
[17]
[18] a Bmpinfoheader
[19] h[offbc+12]←S2C 40 a bcSize = size of infoheader
[20] h[(offbc+4)+14]←I2C iw a bcWidth = bitmap width
[21] h[(offbc+8)+14]←I2C ih a bcHeight = bitmap height
[22] h[(offbc+12)+12]←S2C 1 a bcPlanes = always 1
[23] h[(offbc+14)+12]←S2C bits_per_pixel a bcBitCount
[24] h[(offbc+24)+14]←I2C 0 a xPels/M
[25] h[(offbc+28)+14]←I2C 0 a xPels/M
[26] h[(offbc+32)+12]←S2C cmap_entries a bcClrUsed
[27] z←h
[28]
[29] a Bmpcolormap
[30] a z←z,(cmap_entries*4)p??? a Not used
[31]
[32] a Data
[33] z←z,Image2Bmp im a Convert image to bitmap and append
v

```

This explains itself most adequately from the comments; however it does show one very useful idea – including your email address in the function header. As I said in the disclaimer, I picked up this code in Orlando (actually at the airport on the way home) and started digging into it to prepare this article; obviously I needed to seek the author's permission, so I simply pasted the address from the above comment line and fired off a mail. The answer was back in less than 2 hours. On the principle that APL is inevitably open-source, I would suggest that the implementors consider adding timestamping (*visible* timestamping), author's name and email contact as leading comment options on saving any function.

Next up are a couple of little utilities for getting integers packaged up as bytes:

```

v z+S2C n;[]IO
[1]  a char+S2C int -- Convert SHORT integers to character pairs
[2]  []IO+0 o z+[]AV[,0+0 256 256τ,n]
v

v c+I2C n;[]IO
[1]  a char+I2C int -- Convert integers to character
[2]  []IO+0 o c+[]AV[,0256 256 256 256τ,n]
v

```

This neatly avoids problems with `[]dr` which fails on booleans. Now all we need is to convert the actual bits to make the body of the file:

```

v z+Image2Bmp im;maxb;blk;s;res
[1]  a Creates 24-bit bitmap from 32-bit image
[2]  a im=image created with "Get" method or "image" property
[3]  a z=bitmap
[4]  a| 1998 10 11 19 38 18 a Gert Glantz, Enator Sweden
[5]  a Please send comments and suggestions to: gert.glantz@enator.se
[6]  z+' ' o s+pim
[7]  im+,eim          a Bitmaps are stored "bot to top"
[8]
[9]  a Convert 32-bit integer to 24-bit (=3 chars)
[10] a Stored in BGR order in stead of RGB
[11] maxb+10000      a Block size to avoid WS FULL
[12] :while 0<pim
[13]     blk+(pim)\maxb
[14]     z+z,,082 []dr ~8+[2](blk,32)p11 []dr blk+im
[15]     im+blk+im
[16] :endwhile
[17]
[18] z+((1+s),3*1+s)pz
[19]
[20] a Row length must be a even multiple of 4
[21] :if 0*res+4|1+pz
[22]     z+z,((1+pz),4-res)p[]av[256]
[23] :endif
[24]
[25] z+,z          a Return bitmap as vector
v

```

Now to try it out ... we need a form with a picture on it so the *RawGui* code from *RainPro* looks a good place to start hacking.

This needs quite a large workspace to run in - I settled on 12M as the minimum working size required to snap and convert a typical chart.

```

v bits←RawGui PG;bb;SCL;ppt
[1]  a Sample function to show a Rain chart on a picture
[2]  a Hardcode bounding box (x y x y) at standard setting
[3]  a Get this from PG in a real application!!
[4]  bb←0 0 432 324
[5]  a Initialise resource tables if not already present
[6]  :if 0=Onc 'psbrush' o psbrush←0 2p0 o psfont←0 2p0 o :end
[7]  a Set the extent to match your desired viewing size.
[8]  OIself+'tgt' OIWI 'Create' 'Form'('where' 2 2)('extent' (1.6×324 432+16 8))
[9]  OIself+'tgt.pic' OIWI 'New' 'Picture'('where' (0 0, 1.6×324 432+16 8))
[10] OIWI 'scale' 5
[11] a Wipe the slate
[12] OIWI 'Draw' ('Brush' 1 (255 255 255)) ('Clear')
[13] a Save scaling command <SCL> (r c h w) ...
[14] SCL←c('Scale'),bb[4 1],1 -1*-f#2 2pbb
[15] a Calculate Pixels per Point to get fonts sized right
[16] ppt←.5+.*((OIWI 'scale')[1+4 3])+1-f#2 2pbb
[17] a Run our PostScript interpreter with these parameters ...
[18] a Use a final 0 here for 16 or 256 colour systems
[19] SCL ps_exec PG 'tgt.pic' ppt psacmap 1
[20] a Grab the bits to we can save them to file
[21] bits←OIWI 'image'
[22] a Clear resource tables (optional)
[23] psFree
v

```

To exercise it ...

```

Population
psView PG a to see it
qq←RawGui PG
bmp←Image2BmpFile qq
'qwe.bmp' OIcreate -1
bmp OIappend -1
OIuntie -1

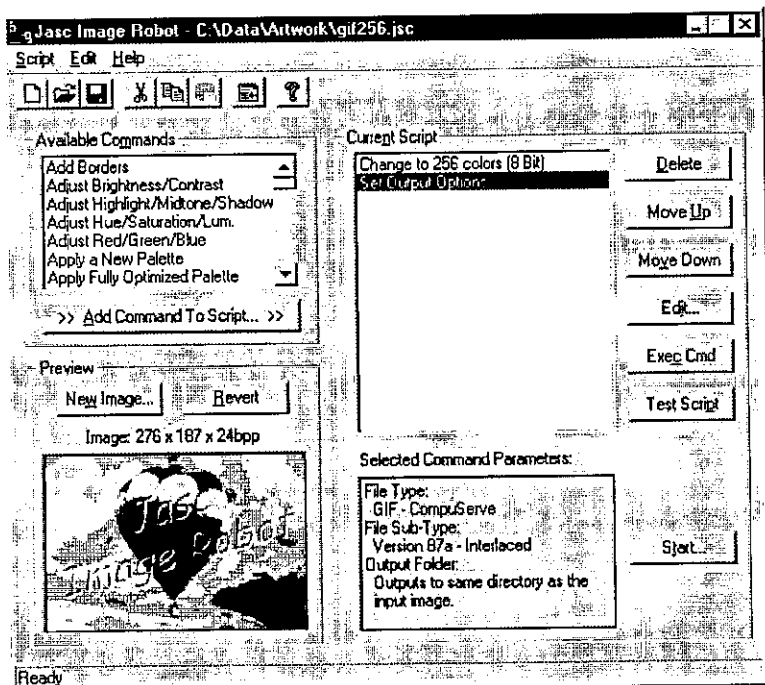
```

Double-click the file in Explorer, and sure enough it loads up in Windows Paint, so we can sign off stage-1. Now to reduce the byte count from a little over 1M to something reasonable.

Stage-2: Write your Script

Writing the script is almost a no-brainer here: we need to cut the image down to 256 colours and save it as the simplest available GIF format (no transparency information is required). Save the script, get a DOS prompt in the directory where you put that bitmap and run:

```
C:\TODO>C:\util\irobot\IRobot c:\data\artwork\gif256.jsc /r *.bmp
```

The resulting reduction in byte-count is pretty spectacular:

```
C:\TODO>dir qwe.*
```

```
Volume in drive C is SYSTEM
Volume Serial Number is 07CF-0A15
Directory of C:\TODO
```

```
QWE      BMP      1,067,142  07/01/00  23:20  qwe.bmp
QWE      GIF       9,415  08/01/00  0:05  qwe.gif
          2 file(s)      1,076,557 bytes
          0 dir(s)      1,120,272,384 bytes free
```

Stage-3: Roll it out ...

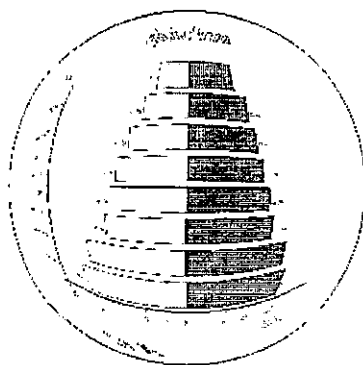
Say we have a selection of timeseries to be generated as charts and saved down to the website in compressed GIF format. Probably we would dump the bitmap images into a working directory (such as c:\todo) and set the output folder in the script to a suitable target. We can give ImageRobot a specific list of files to work

on, but the easy option is to start with a clear directory and simply convert the lot with a wildcard filespec as above. So in both Dyalog and APL+Win we have:

```
Qcmd 'C:\util\irobot\IRobot c:\data\artwork\gif256.jsc /r c:\todo\*.bmp'
```

The only difference being that in +Win we see the DOS box pop up, unless we insert a left argument of 2 to run the command in the background. Dyalog waits for the command to complete; this is not possible in +Win which continues processing immediately. Probably we would add a logfile parameter to the batch command and check for the existence of the file as a secure way of knowing that the job had run OK:

```
2 Qcmd 'C:\util\irobot\IRobot c:\data\artwork\gif256.jsc /r c:\todo\*.bmp  
/l gif256.log'
```



Just for fun, here is the chart I have been using as my example, with a little spherical distortion applied for good measure ...

```
C:\TODO>type gif256.log  
Reading qwe.bmp  
Executing Script Commands...  
Executing Script Line 1 of 3  
Executing Script Line 2 of 3  
Executing Script Line 3 of 3  
Script Complete  
Saving .\qwe.gif  
Output file exists. Overwriting.  
Batch processing complete.  
1 File(s) successfully converted.
```

As I said in the introduction, it works a treat! Probably the best \$89 you will ever spend. From now on, the RainPro climate demos are definitely going on the web site as compressed 16-colour GIFs for the benefit of all those poor souls who still run Netscape-3.

Reference

- [1] Jasc Image Robot 1.2 can be purchased direct from www.jasc.com for \$89, or with manuals and CD for around \$135 including shipping to Europe. This compares well with the benchmark UK price of £138, so web purchase is recommended!

PROPER Capitals

From: Adam Kertesz

Nov 1999

Here is a little utility which capitalizes the first letter in a text string and any other letters in text that follow any character other than a letter.

```

v R←A PROPER B;ALLC;BOOL;FIRST;IND;K;LC;N;P;UC;X;IO
[1]  a Adam Kertesz, 03/20/1998+05/21/1999
[2]  a      To change text into "Proper" format
[3]  a      B is a character vector to change
[4]  a      A is a two part exception list, both vector of vectors
[5]  a      If not exists: empty
[6]  a      If exists: [1] remain as is , e.g. APL→APL, WfWG→WfWG
[7]  a      [2] should be all lowercase, e.g. AND→and
[8]  IO←0
[9]  :IF 2≠IO NC 'A' ∘ A+2p←''
[10] :ENDIF
[11] (A N)+2+A ∘ B+,B
[12] UC←[AV]([AV] 'A')+126 ∘ LC+[AV]([AV] 'a')+126 ∘ ALLC+UC,LC
[13] BOOL←B∉ALLC ∘ FIRST+1 ∘ R+''
[14] :WHILE 0<pB
[15]   X←+/\BOOL=BOOL[0] ∘ K+X+B ∘ B+X+B
[16]   :IF BOOL[0]
[17]   :ANDIF ~(C)K∈A
[18]     IND←ALLC:K ∘ P+((C)K∈N)∉FIRST ∘ K[0]←UC[26|IND[0]] ∘
       K[P+1pK]←LC[26|P+IND]
[19]   :ENDIF
[20]   R←R,K ∘ FIRST+FIRST>BOOL[0] ∘ BOOL+X+BOOL
[21] :ENDWHILE
v

```

```

PROPER 'this is a TITLE'
This Is A Title

```

```

PROPER '76budGet'
76Budget

```

```

PROPER 'ATHENS, georgia'
Athens, Georgia

```

```

LIST1←''
LIST2←'THIS' 'IS' 'A'
(LIST1 LIST2) PROPER 'THIS IS A TITLE'
This is a Title

```

```

LIST1←'APL' 'COBOL'
LIST2←'ARE' 'AND'
(LIST1 LIST2) PROPER 'APL AND COBOL ARE LANGUAGES.'
APL and COBOL are Languages.

```

```

(LIST1 LIST2) PROPER 'ARE APL AND COBOL LANGUAGES?'
Are APL and COBOL Languages?

```

At Play With J: We'll Cross that Bridge when we Come to it

by Eugene McDonnell

Bjorn Helgason, from Iceland, submitted this problem to the Internet J Forum:

Four people, A, B, C, and D, come to a bridge at night, with only a flashlight to guide them. The time each one takes to cross the bridge is: A in 1 minute, B in 2 minutes, C in 5 minutes, and D in 10 minutes. The bridge will only support two of them at a time, and the time to cross is, of course, that of the slower walker. The flashlight must be carried on any crossing. They want to get across the bridge as quickly as possible. Since they have a palmtop computer with J installed, they write a program that tells them what the minimum time is, and in what order they should go forth and back over the bridge. Your problem is to write an equivalent program. To help you get started, the program found that the minimum time is seventeen minutes.

The first response to Bjorn's problem said that there was no need for a program to be written, because it could be solved in one's head; and complained, furthermore, that seventeen minutes was impossible; it was demonstrable that the minimum time was nineteen minutes: the fastest one, A, going across first with B and the flashlight, leaving B on the other side, coming back with the flashlight to go across with C, then coming back with the flashlight again to go across with D: two plus one plus five plus one plus ten, making nineteen, and that the order was unimportant so long as A was the constant companion. It couldn't be done more quickly, since the fastest man was always the one to go back. However, Helgason kept insisting that seventeen was the minimum, and offered to send a private message to the complainer giving such a solution.

After a fair amount of back and forth between Helgason and the disbeliever, the issue was resolved when Kirk Iverson finally submitted a solution from Toronto which gave the seventeen-minute solution for the problem. He admitted that he wasn't sure whether the program would give the minimum answer for all possible combinations of number of people per crossing and times for each to cross, but it would handle many cases. [Yes, Kirk is related to Ken; he's a nephew.] The disbeliever was converted, of course, and made a handsome apology.

See whether you can arrive at a solution, either to this specific problem, or to the more general one solved by Kirk. When you're satisfied that you either can solve the problem, or have thrown up your hands in frustration, or else are just too lazy to give it any more thought, you can go on to read Kirk Iverson's solution. From here on, it is his text, slightly amended, with additional comments by me shown in square brackets.

First, the "compiled" code, in case anyone wants to run this without seeing what it does:

```
NB. ---- copy into ijs window and execute -----
".(noun define)-.LF
bridge=(.<./@(>@{2&{}}){(.@],(1&{.@],&.><@[]),(2&{.@]-&.><@[]],3&{
.@],<@[]]]^:(+@#@(>@{1&{}}))@((((>@{.@[-.]]);>@{:@[]}).@(([-.[-.
]))/))@(>@{0&{}})(([<.&#@])&{.])"1(,,:|. )@(/:~)@(>@{1&{}})(([>@{-(>@{
2&{}}@]<@{<./}>@{:@[]]+.#@(>@{:@[]]=#@(>@{1&{}}@[]])){(.@],(1&{.@]-
&.><@[]],(2&{.@],&.><@[]],3&{.@],<@[]]]^:(+@#@(>@{1&{}}))^-&.&{({
.;};.'"~) :.((+/@:(>./@>);])@{3&{.}}))@,
)
NB. -----
```

[I speculate that Kirk first wrote a series of verbs to accomplish the objective, then obtained this unreadable mess by applying the fix adverb (f.) to the main verb. Doing this replaced all the subordinate verbs by their definitions, yielding the aforesaid mess. He probably then obtained the character string corresponding to the fixed function, using the 5!:5 form of the foreign conjunction, and displayed this in a field 62 characters wide, giving the six lines shown above, and copied them. The line following the first NB. line contains three items that are defined in a script loaded when a J session starts: the variable *noun* has the value 0; the adverb *define* has the value : 0 (explicit definition script form) and the variable *LF* is the linefeed character, defined as 0{a. The (noun define) in the first line after the comment line (beginning with NB.) permits entry of multiple lines of text. Kirk pasted in the six lines, then typed a right parenthesis on the next line, and hit enter, thus causing entry to terminate, and the (noun define) was replaced by the six lines shown. The -.LF removed the linefeeds from this text, and the ". executed the line, causing a verb named *bridge* to be defined.

I copied the *bridge* verb, and pasted it into a J session, and it worked as advertised.]

The left argument to *bridge* is the maximum number of people who can walk on the bridge at the same time; the right argument is a list of the length of time each person needs to cross.

The result is a boxed list of total elapsed time, followed by all moves:

```

      2 bridge 1 2 4
+-----+-----+
|7|1 2|1|4 1|
+-----+-----+

```

The result signifies 7 minutes total time; 1 and 2 cross; 1 returns; 4 and 1 cross.

[This is your last chance to try figuring out how to solve our problem, because the solution Kirk obtained is now going to be shown.]

NB. -----
 NB. Crossing the bridge.

NB. Rules

NB. Move all people from one side of bridge to other.
 NB. Each person has an individual time it takes to cross.
 NB. At most MAXLOAD number of people on bridge.
 NB. Torch must always be with a group on the bridge.
 NB. Speed of a group is the speed of the slowest member.

NB. Strategy

NB. Overall strategy is to pick a good group to go over, and
 NB. then have the fastest person to return with the torch. Wash,
 NB. rinse, repeat. I call the guy to return the torch the "runner".
 NB.
 NB. We attempt to move the slowest people together as a group,
 NB. rather than split them up to slow down all groups. We move
 NB. the slowest ones over whenever there is a suitable runner
 NB. to return, i.e., none of this slow group will have to return.
 NB.
 NB. If there is no good runner, we select the fastest to go over
 NB. and act as runners. We only include enough runners which are
 NB. necessary to bring in the rest of the people.

NB. Notes

NB. Iteration is done by repeatedly applying a function to
 NB. the full set of data until it results in everyone moved.
 NB. Data is maxload;unmoved;moved;move0;move1;move2;...

NB. Verbs to access pieces from the data

maxload=: >@(0&{) NB. maximum number of people on the bridge
 unmoved=: >@(1&{) NB. people not yet moved
 moved=: >@(2&{) NB. people already moved
 moves=: 3&}. NB. record of all moves
 more=: *@##unmoved NB. are there more to move?

```

NB. move/unmove move people across bridge
NB. x. is list of people to move; y. is data
move=: {.@] , (1&{@] less each <@[] , (2&{@] , each <@[] , 3&{@] , <@[]
unmove=: {.@] , (1&{@] , each <@[] , (2&{@] less each <@[] , 3&{@] , <@[]

```

```

NB. Strict <less> Similar to -. but removes from x only the number
NB. of matching items found in y. All items in y are expected to be
NB. found in x, and the items in the result are in the same order as
NB. they appear in x. Universe is ~.x, and is catenated in front of
NB. y, therefore count (#/.~) of each returns the counts of the
NB. respective items. The difference (incremented to compensate for
NB. the catenation of the universe onto y) is used to copy items from
NB. the universe.

```

```

less=: universe #- [ >:@-&count universe , ]
universe=: ~.@[
count=: #/.-

```

```

NB. Pick the group to go across. Runners are the fast group to
NB. shuttle the rest over; waddlers are the slower group put together
NB. to capitalize on the slowest of the bunch.

```

```

groups=: sort@unmoved (runners ; waddlers) ]
runners=: [ maxtake- required <. maxload@]
required=: 1: >:@>. #@[ <:@>.@% <:@maxload@]
waddlers=: maxload@] maxtake |.@[

```

```

pickslow=: nogoodrunner +. lastgroup NB. 0-runners; 1-waddlers
nogoodrunner=: moved@] <:@fastest slower
lastgroup=: #@slower = #@unmoved@]

```

```

NB. fastest/slowest in a group

```

```

fastest=: <./
slowest=: >./
slower=: >@{:@[
maxtake=: ([ <. #@]) {. ] NB. take, but don't overtake

```

```

pick=: groups (pickslow >@{ [ ] ]

```

```

NB. forward - pick group to go and move them
NB. return - pick fastest runner and move back, if more to move
forward=: pick move ]
return=: (fastest@moved unmove ])^:more

```

```

NB. iterate repeats the forward/return action until there are no more.
iterate=: return@forward^:more^:_

```

```

NB. assemble the argument into the data structure; inverse
NB. computes total trip time from moves, and returns that along
NB. with the moves.
assemble=: ({. ; ] ; ''~) :. ((+/@:(slowest@>) ; ] )@moves)

```

```
NB. maxload play speeds
bridge=: (iterate&.assemble)@,
```

```
NB. -----
```

[And now for our problem:]

```
2 bridge 1 2 5 10
+---+---+---+---+
|17|1 2|1|10 5|2|2 1|
+---+---+---+---+
```

The total time is 2+1+10+2+2, or 17, as was required. An alternative would have 2 return the first time, and 1 the second time, and this also is minimum.

Kirk's J implementation consists of two dozen or so verbs, all in tacit form, so this is a functional programming solution. The iterate verb continues until there is no more to be done, using power to the limit, denoted by infinity (^: _).

As this article goes to press, it is recognized that Kirk's solution is not guaranteed to give the minimum result for all cases; for example, it fails for 1 10 11 12; but it was the first solution that gave the minimum result for the given case 1 2 5 10.

Vector Back Numbers

Back numbers of Vector are available from:

British API Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Setting up a www-server with Dyalog APL, IIS, ISAP and NT

Janne Mäenpää (janne.maenpaa@stonesoft.com)

Introduction: The Basics of an APL Web Server

This article concentrates on building a dynamic www server on Microsoft's Internet Information Server and Dyalog APL. The latter takes care of all dynamic elements, and a third party program ISAP by Lingo Allegro handles the communication between the two.

A dynamic APL web server can be created with only APL, or in combination with third party programs. A solution using IIS is, in my opinion, more flexible and ready-for-use than the web server included with the Dyadic interpreter. User privileges, user logs, encryption and integration with the operating system are taken care of. IIS also allows several sites on one machine, even several IP numbers. One great thing about a less exotic server is that it can be maintained by everyday webmasters. No APL needed!

In an IIS+ISAP+APL solution Dyalog APL receives data about all user's HTML operations linked to ISAP.DLL (post, get, button, URL) via a shared variable. APL pushes user-ready HTML pages back to ISAP→IIS→user via the same variable. HTML pages coming from APL can either be static or dynamic. Server can also contain static pages and elements that do not need APL interaction. Usually these are help pages, pictures etc.

The Case of Foreign Trade Statistics

The Finnish foreign trade statistical database Ultika has been around since the beginning of the decade. The programs are provided by TietoEnator, data is produced by Finnish Customs. The database contains more than 2.5 billion numbers, and thus is one of the biggest public statistical databases in Europe. Before the introduction of a web interface Ultika was accessed using 3270 terminals. The database is still on a VM/CMS mainframe under IBM APL2 and ADI. The web interface is on an NT machine, interacting with VM using TCP/IP inside TietoEnator networks. Www-Ultika was launched on December 1998; static (Finnish) pages can be experienced at <http://Ultika.Tieto.net/>.

The Basics of Dynamic HTML

HTML utilises a few basic tools from the graphical user interface: text fields, checkboxes, radio buttons, pull-down menus and buttons. In Ultika buttons have been replaced with picture maps that are used like buttons. The basic GUI tools turn into variables on an HTML page, and get values according to user interaction. An HTML page is defined to be a form with an appropriate tag that also includes data about what is to be done when exiting the form. Exit from the form can happen in three ways:

- by clicking a link (links do not need forms to work)
- by clicking a picture map (like Ultika buttons) or
- by pressing a button

Upon exiting, the browser sends the server, among other things, the form variables along with their values. The server replies with headers and the resulting HTML page.

Dynamic IIS-ISAP-APL Server

A www server receives headers and data about user actions from the browser. In this case IIS delivers the data to Dyalog APL via Lingo's ISAP. The APL server action is based on namespaces. An HTML form tag or link contains the name of the server namespace as the value of a variable called `WS`, to be used when exiting the page. These server namespaces contain an APL vector `_COM`, including the namespace functions to be executed.

An example of a link from an HTML page (not form) to APL:

```
<a href="ISAP.DLL?WS=EXECUTE&OPERATION=IOTA10"> 1 to 10 </a>
```

The text "1 to 10" is a link on the HTML page, and by clicking it the data on href are transferred to IIS server by the browser. IIS transfers the data to ISAP.DLL, and ISAP pushes it to the shared variable `_IN` in the EXECUTE namespace. `_IN` is a two-dimensional nested matrix. The first column contains the names of the variables (`WS` and `OPERATION` in this case) and the second corresponding values (EXECUTE and 10), all as text. A form may contain several variables with the same name.

In the EXECUTE namespace the answer is formulated to an HTML text vector and sent to the browser. The developer does not need to worry about user control or take care of simultaneous users, ISAP does that. ISAP creates a copy of the

namespace used, and after the transaction deletes the copy. In this #.CLI0 (#.CLI1 etc.) namespace the user is treated as the only person ever to have used the service, so it is very difficult for the developer to get confused with user channels.

An Example of Server Services

All data transferred from IIS to APL are in the picture. ISAP creates several global variables to the namespace used. These variables can be used when creating an HTML-answer to the user.

The figure consists of four screenshots of APL dialog boxes, each showing a different set of data related to an HTTP transaction.

- CLI0._ALL_HTTP - Dialog**: Shows a list of HTTP headers and their values.

[0]	HTTP_ACCEPT	image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, image/png, */*
[1]	HTTP_ACCEPT_LANGUAGE	en
[2]	HTTP_CONNECTION	Keep-Alive
[3]	HTTP_HOST	jcm
[4]	HTTP_REFERER	http://jcm/ultika/isap.dll?WS=ALKU
[5]	HTTP_USER_AGENT	Mozilla/4.51 [en] (WINNT; 1)
[6]	HTTP_CONTENT_LENGTH	214
[7]	HTTP_CONTENT_TYPE	application/x-www-form-urlencoded
[8]	HTTP_ACCEPT_ENCODING	gzip
[9]	HTTP_ACCEPT_CHARSET	iso-8859-1,*,utf-8
- CLI0._IN - Dialog**: Shows a list of HTML variables and their values.

[0]	NS	POHINTA
[1]	LOMAKE	P1802
[2]	TIKETTI	93448809351434040393242
[3]	MAATO1	ru se
[4]	ALUESUHHAUSO1	2
[5]	UUOSI	1995
[6]	UUOSI	1996
[7]	UUOSI	1997
[8]	UUOSI	1998
[9]	KUUKAUSI	12
[10]	KUUUALIALKU	Tammikuu
[11]	KUUUALILOPPU	Tammikuu
[12]	KUYKSIKKO2	2
[13]	KUYKSIKKO2	4
[14]	OK.x	67
[15]	OK.y	10
- CLI0._REMOTE_IP - Dialog**: Shows the remote IP address.

[0]	131.207.6.53
-----	--------------
- CLI0._SECURE - Dialog**: Shows the secure flag.

[0]	0
-----	---
- CLI0._COM - Dialog**: Shows the first function to be started.

[0]	0
-----	---
- CLI0._PORT - Dialog**: Shows the server gate number.

[0]	80
-----	----
- CLI0._PATH_IR - Dialog**: Shows the server virtual directory.

[0]	C:\inetpub\wwwroot
-----	--------------------

Number one contains http headers sent by www browser. APL variable contains information about file formats understood by the browser, language, connection type etc. Number two is the _IN containing all HTML variables with values, three is _COM variable on top, the name of the first function to be started. Under _COM are various variables: user IP, info about encryption, server gate number, server virtual directory and server actual directory.

Practical Work in Practice

Setting up an Ultika-type web server is cheap, easy and fun. IIS is free and the ISAP demo version equals the price. When installed, ISAP sets up its demo site (the workspace, run time engine, HTML pages, ISAP programs) under IIS as one of its web sites. The developer can rather easily build his site on this basis. It is very advisable to use the tools in the root of the ISERV demo workspace. The workspace is a replica of Lingo's web site.

Our equipment is:

- NT 4 server (workstation for 10 or less simultaneous users)
(www.microsoft.com)
- Dyalog APL (run time is enough for web site) + workspace
(www.dyadic.com)
- Microsoft Internet Information Server 3 or newer (preferably IIS4)
(www.microsoft.com → NT server option pack, can also be installed on an NT workstation)
- Lingo Allegro ISAP 3 or newer (preferably ISAP 4 or 5)
(www.lingo.com)

A good help designing the pages is Microsoft FrontPage, a WYSIWYG HTML-editor integrated to IIS.

Learning the Hard Way, Hints and Tips for the Developer

Here are a few tips learned the hard way. Learning them has made my life easier, so I am sharing them with the APL community... Solutions described here are rather obvious, but realising them might take time and effort. At least for me they did.

- **Dynamical bases.** The dynamic elements of all HTML-pages should be marked in the base-HTML clearly. In Ultika all dynamic parts are marked with a modified HTML comment tag: `<!--**-->`, one in the beginning of the dynamic part and one in the end. Between the two is the default value of the dynamic element. In the worst case scenario the tags go through APL unmodified, but that does not hurt the browser and the user gets some values. In the web server project the HTML pages can be separated to a sub-project and taken care of by a web designer. When basic HTML pages are read to the APL server workspace, the non-dynamic elements can be separated to specific variables, like in the elements of nested APL arrays.

- **Modularity.** Ultika has dozens of similar pages with only a little variation. To avoid maintaining all these individual pages, a modular system was introduced. Modular elements or modules are put to a few HTML bases according to user needs. These modules are selections of years, months, countries, product codes etc. Actually each Ultika module corresponds to a mainframe 3270-selection of the old Ultika interface.
- **User recognition.** There are two ways of creating a www-session with state that is able to recognise the user every time she/he interacts with the server: using a cookie or a ticket hidden in the HTML page. Because some users can or will not accept cookies, Ultika uses a ticket. A Ticket is a simple, unique string of numbers created with the user id and the seconds passed since I was born. This ticket is put to every HTML page in a hidden variable ticket, for example: `<input type=hidden name="TICKET" value="67678867">`
In the Ultika server workspace every user who is logged on has his own namespace containing user information. This info is about the statistic data the user is browsing etc. Every ticket is also in a nested matrix containing the ticket, the name of the corresponding namespace and the absolute seconds the ticket was last used. Whenever a ticket gets to be older than 30 minutes, the user ticket and namespace are deleted. If IIS passes an HTML-page to APL with a TICKET-variable not on the list, the user receives a notification about the session being not valid any more. Compared to a cookie, a ticket does nothing in user's machine, so the paranoid people can relax about Internet safety.

Happy Developer

In my opinion the IIS-ISAP-APL solution is very effective. Getting started is easier using a pure-APL solution, with IIS and ISAP the developer has to study the logic of two extra programs. When the basics are clearly in mind, the actual web server programming is fast and effective. IIS-ISAP co-operation is considerably fast and stable, and when IIS crashes it normally does not crash the NT server. ISAP includes lots of programs that a pure-APLer would need to write anyway, not to mention the service level of IIS. Both ways have their pros and cons, and like in all life, there is more than just one way of doing things right.

Sending MIDI Messages to a Sound-Card with J

by Martin Neitzel (neitzel@gaertner.de)

Today's PCs are usually equipped with a sound-card which is MIDI-capable, that is, it will understand the protocol defined by the Musical Instrument Data Interchange standard. Here is some simple code to interface to a MIDI device using J.

The MIDI Utility Definitions

The Microsoft Windows system already contains all the basic routines to deal with MIDI devices. There are actually two sets of routines. The high-level "multimedia player" routines allow you to run prepared MIDI files. MIDI files typically have several voices and contain timing information. The other set of routines, and these will be our topic, are the low-level routines. They allow us to send tiny protocol units directly to the sound-card which is just fine in an interpreter's interactive environment:

```
<midi.ijs required files>=
```

```
require 'dll format'
coclass 'midi'
```

```
NB. The winapi declarations have moved around a bit historically.
NB. and its not always possible just to say 'require winapi'. So:
winapi_path =. monad define
  select. ({.~ 1.&'/' ) 9!:14''
  case. '4.01';'4.02'      do. 'system/examples/winapi/winapi'
  case.                    do. 'system/packages/winapi/winapi'
  end.
```

```
)
```

```
'z' load winapi_path''
```

```
<midi.ijs dll calls>=
```

```
midiOutGetNumDevs =: 'midiOutGetNumDevs' win32api
NB. Two are buggy in the J system distribution:
midiOutGetDevCapsA =: 'winmm midiOutGetDevCapsA i i *c i'acd
midiOutOpen =: 'winmm midiOutOpen i *i i i i'acd
midiOutShortMsg =: 'midiOutShortMsg' win32api
midiOutReset =: 'midiOutReset' win32api
midiOutClose =: 'midiOutClose' win32api
```

Two of the DLL calls are incorrectly declared in the supplied API specs for win32api, so we have to do it ourselves.

Alas, to find these routines and their documentation, you still have to acquire some Microsoft products: A "Microsoft Developer's Network" CD-ROM is essential for the documentation - but still not enough. In addition, you'll need the C header files to find all the actual values of the symbolic constants mentioned in the documentation.

Almost all of the functionality of a MIDI device can be controlled by "MIDI short messages". They consist of a command byte followed by up to three parameters. Our workhorse for the experiments will therefore be `midioOutShortMsg`, disguised by a simple J utility `sm` which takes care of the message encoding and the currently opened MIDI device.

The "General MIDI" standard assigns various piano, string, or wind instruments to numbers ranging from 0 to 127, and sometimes the whole 8-bit-byte range is supported by a MIDI device. Note numbers are simply a chromatic numbering, using 60 for the middle C, 61 for middle sharp C, and so on. Loudness (or, more accurately, "velocity") uses 127 as medium loud and 255 as "fortissimo". (Mind you: effective loudness depends on your volume settings. Check them in case you hear nothing during your experiments.)

In the `sm` definition below, `".@('moh' '_)'` will cause a deferred evaluation of the variable `moh`, the current Midi Output Handle as returned from the last `midioOutOpen`. The sub-clause `256&#. @|.` is a neat way to put the up two three Short Message codes into the proper byte order and merge them into the `int` required by the Windows API.

```
<mido.ijs short message routines>=
```

```
sm=: midioOutShortMsg @ (".@('moh' '_)' ; 256&#. @|.)
```

```
chprg   =: sm @ (192&.)           NB. cmd192 instr-nr(y.)
noteon  =: sm @ ((1)& 144 0 125) " 0 NB. cmd144 note(y.) loudness
noteoff =: sm @ ((1)& 128 0 125) " 0 NB. cmd128 note(y.) loudness
```

To play a note, you have to send a "note on" message, wait a bit, and then send a "note off" message. With a piano sound, one might think a "note on" message would suffice, since the note will decay naturally. However, until you finally emit the "note off" message, precious resources might still be tied up in your soundcard. Also, switching to an "organ" type sound will make the purpose of "note off" messages drastically clear.

To ease the three steps of on/wait/off for each note, the `dp` verb will play a note with a given duration. It uses the `6! : 3` delay foreign with sub-second resolution. Unit time here is a tenth of a second, and the default duration is set at 2 tenths. Of course nobody is prevented from using a duration of `0.1`, resulting in a milli-second note - very quick!

Exercise here: make a prediction just from looking at the definition what `dp` would do with non-scalar or boxed arguments.

```
<midi.ijs play verb>=
dp =: verb define "0 0
    2 dp y.
:
    noteon y. =. >y.
    6! : 3 x. * 0.1
    empty noteoff y.
)
```

Initializing the MIDI Device

The above provides a lot of definitions. As the last step of `midi.ijs`, we will open one of the devices using the low-level routines. This way, users can directly proceed with using `dp` and have some fun.

Before we present the code going into the `midi.ijs` script, we start with a simpler session trace showing what is involved with opening the device, step by step. We switch a bit back and forth between session logs and `midi.ijs` code, so watch the headers.

First, we have to open the proper MIDI device and to obtain a handle for it. `midiOutGetNumDevs` will return the total number of devices in the system able to receive MIDI messages. Even though my laptop used for these experiments has only one physical sound-card, I'll get 3 MIDI devices reported:

- the MIDI-controlled sound generator of the sound-card
- the external MIDI port of the sound-card (disguised as a game port), possibly leading to one or more connected MIDI synthesizers, drum machines, and similar gizmos
- a software-emulated MIDI synthesizer accepting MIDI messages and generating WAV data for any cheap-o sound-card based on high quality instrument samples

Another MIDI output "device" often seen is the "MIDI Mapper" of the Windows system. This is a pure software driver used to compensate for some lack of standardization from the early MIDI days: the code numbers for instruments have not always been fixed, and the "MIDI mapper" allows you to re-assign instrument numbers on-the-fly, before the messages finally reach the true device.

The numbering of the devices is *not* fixed. It can change from reboot to reboot on a machine. In order not to end up with speaking to an orphaned game port, you'll have to list the names of the reported devices and choose the proper one.

`mididOutGetDevCapsA` returns a rich structure. We use a bit of magic hackery here just to extract the name of a device. Another article describes how the complete story can be uncovered.

```
<midid.js dll calls>+ =
```

NB. A wrapper to return the device name given a device number:

```
mididOutGetDevName =: monad define "0
    caps=. mididOutGetDevCapsA y. ; (100# ' '); 100
    NB. The following relies on some "magic knowledge"
    NB. about the layout of the "capabilities" C structure:
    name=. ((.- i.&(0{a.)) (8+i.32) { > 2{caps
)
```

```
<session trace>=
```

```
] ndev =. > mididOutGetNumDevs ''
3
(; mididOutGetDevName)"0 i. ndev
+-----+
|0|ESS MPU-401      |
+-----+
|1|ESFM-Synthese (220)|
+-----+
|2|YAMAHA SGMP Driver|
+-----+
```

The "MPU-401" device refers to the external game/midi port of the system, whereas "ESFM-Synthese" refers to the on-board FM MIDI chipset of the sound-card. The odds are good you'll find something similar in your system, too. The Yamaha SGMP Driver is the software-emulated "sound-card". Let's use the built-in hardware synth, i.e. the device number 1 from the list above. (The MPU driver would be useless unless there's a synthesizer hooked up to the external port.)

The first argument to `mididOutOpen`, `(,9)`, is a bit tricky. The parameter type is `*i`, for "a pointer to the space receiving the handle of the opened device".

Fortunately, there is no need to involve complicated `mema/r/w/f` calls to get the memory. We are not dealing with a structure which will be referenced again later on. Instead a simple temporary array will do the job, therefore the `ravel` comma. But a `(,0)` or `(,1)` would not be adequate, because that gives only a list for one (j) boolean. We really need an integer like 9 (or 54321 or 2 or `_42`) here to create the space capable to hold the (integer-like) handle.

The second argument in the list is 1, the device number of our ESFM-Synthesizer. Just accept the three remaining zeroes as constants for the moment.

```
<session trace>+=

] r =. midiOutOpen (,9) ; 1 ; 0 ; 0 ; 0
+-----+
|0!_2104923004|1|0|0|0|
+-----+
moh =: 0{1 pick r

chprg 0      NB. acoustic grand piano
+-----+
|0!_2104923004|192|
+-----+

dp 60      NB. play a single note (middle C)
NB. Bing!
```

In our script, we are doing just the same. The only difference is that we will pop up a selection dialog and take it from there.

```
<midi.ijs initialization code>=

ndev =. > midiOutGetNumDevs ''
devlist=. midiOutGetDevName each i. ndev
mo_dev_idx=: 1 pick (<:ndev) wselect 'Select the MIDI device' ; <devlist

NB. parameters are:
NB. *device_handle (will be set) ; device_index;
NB. callback_function; instance; flags (int!)
] r =: midiOutOpen (,9) ; mo_dev_idx ; 0 ; 0 ; 9-9
] moh =: 0{1 pick r

chprg 0      NB. acoustic grand piano

NB. Announce that we are up & running:
2 2 2 8 dp 51 51 51 48      NB. yaddadda-dah!
```

The final section of the code makes the high-level functions visible to general applications by planting referents into the `z` locale. Utilities requiring MIDI knowledge remain confined to the `midi` class.

```
<midi.ijs exports>=
```

```
NB. export major tools:
```

```
export =: monad define
    dp_z_      =: dp_midi_
    chprg_z_   =: chprg_midi_
    midireset_z_ =: midiOutReset_midi_ bind moh
    midiclose_z_ =: midiOutClose_midi_ bind moh
    empty ''
)
export ''
```

This simple set of routines hides all the details of the MIDI system. In particular, the output handle `moh` becomes completely invisible. Users of the `mid` class only have to remember a final call to `midiclose`.

This closes our `mid.ijs` script.

From MIDI Technology to Simple Music Theory

This, finally, is the fruit of our labour: a short sample session using just the high-level definitions and concentrating on the notes from a musician's perspective. It does look impressive here as it is written: while you can read the J input sentences here, the output is "only" audible. So run the code!

```
<session trace>+=
```

```
require 'contrib/windows/packages/music/midi'
```

```
maj =: 0 4 7
```

```
min =: 0 3 7
```

```
10 10 20 dp 66 68 69 +each maj;min;maj      NB. box notes for chords
```

```
NB. Elvis was here:
```

```
C=:60
```

```
riff =. 0 12 10 7
```

```
blues =. 0 0 0 0 5 5 0 0 7 5 0 7
```

```
(dp- # $ 6 3"-) C + 0 ,-, blues +/ 2 # riff
```

```
NB. Use midireset whenever you have to reset the device,
```

```
NB. for example if you experience "hanging notes".
```

```
NB. Also, a call is well advised before closing the device:
```

```
midireset''
```

```
+-----+
```

```
|0!_2104923004|
```

```
+-----+
```

```
NB. When you are done with everything:
```

```
midiclose''
```

```
+-----+
```

```
|0!_2104923004|
```

```
+-----+
```

Where To Get This Code

This text is available in a variety of forms. The true source of it is a single file written for a simple literate programming tool. This file has been postprocessed into various forms. You are probably looking at one of the following right now:

You might hold it in your hands as a printed Vector article, containing both descriptive text and the J code.

You might be browsing though the hypertext version with full blown cross-references, served at <http://juggle.gaertner.de/bnp/midi.html>.

Very likely however, you'd be interested in just the `midi.ijs` script file as extracted from this text. This is available, too, along with the original literate programming version and a `music.ijs` script providing simple tools for studying simple harmony theory (scales, chords, and all that).

All these files are still evolving, and therefore they are excellent candidates for maintenance in the J source repository.

It is the module `j/contrib/windows/packages/music` at the CVS repository `:pserver:anoncvs@juggle.gaertner.de:/usr/local/cvs`. If this looks all Greek to you, turn to the intro page at <http://juggle.gaertner.de/cvs/>. It provides anything you need to know, including hands-on tutorials and references to the software for your system. If you don't want to bother with CVS, however, you can download all the files introduced here (and the source of this article) with your web client from:

<http://juggle.gaertner.de/cvs/cvsweb.cgi/j/contrib/windows/packages/music>.

Acknowledgements

I'm indebted to Eric Iverson for explaining the finer points of the DLL interface to me and for his comments on this text.

Martin Neitzel
Gärtner Datensysteme
Braunschweig,
Germany
email: neitzel@gaertner.de

Index to Advertisers

APL Booklist	118
Dyadic Systems Ltd	6
Strand Software	2
Vector Back Numbers	130

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue $n+1$ and discuss themes for issues $n+2$ onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" – an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Maltings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01853-696760
Fax: 01853-697719
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0870-1010-469
Email: HMW@4extra.com

Insight Systems ApS
Nordre Strandvej 119G
DK-3150 Hellebæk
Denmark
Tel: +45 70 26 13 26
Fax: +45 70 26 13 25
Email: info@insight.dk
Web: www.insight.dk

APL2000
Rapid Application Development
6610 Rockledge Drive
Suite 502
Bethesda MD 20817
USA
Email: sales@apl2000.com
Web: www.apl2000.com

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: sales@soliton.com

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: +31 347 342 337

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1999/2000 Committee

Chairman	Anthony Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Secretary	Ajay Askoolum 01737-771643 ajay@askoolum.freemove.co.uk	42 Hanworth Road, Redhill, Surrey RH1 5HT
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o Vector Administration, Brook House, Gilling East YORK YO62 4JJ
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 100430.740@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane, YORK YO30 6DF
Education	Dr Ian Clark 01388-605544 100021.3073@compuserve.com	Unit R21, Auckland New Business Centre, St. Helen Auckland, Bishop Auckland, Co. Durham DL14 9TX
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Richard and Adam Weber (0121-3546550), Anthony & Sylvia Camacho, Ray Cannon (01252-874697), Marc Griffiths, Bob Hoekstra (01483-771028), Jon Sandles (01904-612882)	

Typeset by APL-385 with MS Word for Windows

Printed in England by Short-Run Press Ltd, Exeter